DLLBC: One Grammar, Four Arrows

A mock-paper on the Dependent Low-Level Borrow Calculus — draft

Abstract. DLLBC, the Dependent Low-Level Borrow Calculus, puts full dependent types and Rust-style mutable borrowing in one language and checks both with one symbolic interpreter: a single term grammar, four evaluation arrows (runtime read and write, comptime read and write), and a checker that runs function bodies on symbolic arguments, so that borrow checking and dependent type checking are two readings of one execution. The design rests on an identification — LLBC's symbolic refinement of a matched scrutinee *is* Agda-style dependent pattern matching, one substitution mechanism serving both checkers — completed by a *branch equation* rule that reifies the same fact where abstraction has replaced substitution; and, at the boundary, on an inversion of the Aeneas pipeline: where Aeneas synthesizes a backward function per borrow, DLLBC lets the signature declare it (the ↳ obligation is its type, a declared `back` the function itself) and audits the body against the declaration at return. Checked and green in the mechanization at the pin: the complete rule set of the seven figures, each rule tied to its implementing function and exercising test; *three* in-place quicksorts, which between them make the paper's comparison of two verification architectures a measurement rather than a prediction — one type-checking as an implementation of its pure model `sortRangeL`, conformance reduced to conversion, and two carrying no declared backward specification anywhere in their call trees, whose return type *is* their postcondition (sortedness of the exit snapshot, with count-preservation against the entry) and whose every proof step is built in the body from its callees' postconditions, with no pure model of the partition or the sort in either development at all; the **carve**, the calculus's first rule that consumes evidence — a lazy reorganization splitting an array into segments, licensed by a comptime `Le` proof, after which two range borrows coexist because they are literally different subterms of one value tree and no disjointness judgment exists anywhere in the system; a declared decreasing-argument guard that closed a live unsoundness — an unguarded recursive declaration proved any postcondition from itself. Its scope is exactly: a **single**, **declared** argument position, checked for strict structural decrease against that parameter's current snapshot, on **self**-calls only, with mutual recursion rejected outright rather than supported and no general well-founded measures; with a sufficiency hypothesis discharging the out-of-fuel path, recursion so guarded is total. Also checked at the pin: a differential harness relating the symbolic checker to concrete execution, validated by turning a removed unsound inference back on and watching it go red; and a machine-verified invariant that refinement propagates knowledge and never state. The differential is load-bearing rather than reassuring: it found the first divergence whose *polarity* runs the other way, where the checker was right and the **machine** was broken — something no amount of checking could have revealed, and which a soundness statement phrased as "the checker over-approximates" cannot express. And because the two direct-proving quicksorts share a specification and no code, running them against each other is evidence about the specification rather than about either program: eleven shared inputs, agreeing with each other and with a reference sort. Three measurement sets appear, and no two are comparable: a profiler-guided substitution fix dropped the *conformance* audit 465× (84,121 ms to 181 ms; the full suite from 38m49s to roughly 13–20 s); separately, the two `List` direct-proving quicksorts differ by about a thousandfold (21.8 s positional, 21 ms whole-list) with no performance work between them, how the problem is posed dominating what the checker costs; and the cross-differential's eleven inputs are a count of test cases, not a time. Open, and stated as such: the pure model's own correctness, which the conformance route rests on and the direct route no longer needs; a body cannot yet name the value a consumed binder held, which costs the direct route four staged proof-builders that do no mathematical work; the v0 kernel is type-in-type with no consistency claim; and three identified audit-strategy holes must be closed before any soundness statement can quantify over declared specifications. No soundness theorem is claimed.

Status of this document. This is a *mock paper*: a paper-shaped audit of the DLLBC mechanization, written by extracting the rule figures from the implementation at pinned commit `9d92a894` of the Ochre repository and holding every claim to what is checked there. It is not peer-reviewed and the calculus is work in progress. Claims carry a uniform status convention — GREEN checked in the mechanization at the pin; IN FLIGHT under construction; PROPOSED decided but not mechanized; OPEN a known gap — and untagged claims in the rule figures are green. The findings ledger (`NOTES.md`, alongside this document) is part of the artifact: it records every doc-vs-implementation divergence the figure extraction surfaced, including the ones this paper reports as open holes.

The figures were first extracted at `122bb424` and have been re-extracted twice since, each time because the artifact's subject changed rather than its detail: once when a second verification architecture landed, and once when arrays, range places and the carve arrived — the latter adding Section 10.7, the first new judgment form since the original extraction. Where a rule moved, the figure was re-read against the implementation rather than carried forward. The findings ledger keeps each extraction's numbering sealed and records, per era, what closed and what remains.

Contents

1 Introduction

Type systems are the most widely deployed form of static verification, and the guarantees they offer span a wide range: from freedom from runtime type errors, through Rust's ownership-based memory safety, to the full functional correctness of Lean, Agda, and Rocq. The strongest of these guarantees rest on *dependent types* — the mechanism by which a type may mention a value, so that a signature can state "this function returns a sorted permutation of its input" and the compiler checks it as a condition of compilation.

There is a large and consequential intersection between software that demands low-level control and software whose correctness is critical: operating-system kernels [1], cryptographic primitives [2], network stacks [3], allocators. Yet the languages that supply the control (C, C++, Rust) lack the type-theoretic machinery for dependent verification, and the languages that supply dependent types (Lean, Agda, F*) lack control over layout, allocation, and in-place mutation. A program that needs both is written twice — once in the systems language, once in the proof assistant — with a translation between them that must itself be trusted or verified.

DLLBC, the Dependent Low-Level Borrow Calculus, is a core calculus built to test one way of closing this gap: put dependent types and Rust-style ownership in *one* language, and let *one* symbolic interpreter serve as both the borrow checker and the dependent type checker. The move that makes this possible is to erase the syntactic wall between terms and types. There is a single syntactic category; types are terms of universe sort; and the ownership constructs — borrow, loan, dereference — are ordinary term formers within it. Type checking is then not a discipline layered over evaluation but the *same evaluator run on symbolic inputs*: to check a function body, the interpreter runs it over symbolic arguments, watching ownership flow and computing types by the very reduction rules the runtime uses. Borrow checking and type checking become two readings of one execution.

1.1 The headline artifact

The calculus is driven by a target program: an imperative, in-place quicksort over a mutable borrow, written in the shape a Rust programmer would write it. The mechanization verifies it three times — twice under the two architectures Section 7 compares, and a third time over a different container — and that family is the paper's central artifact.

The first is *conformance*: the quicksort carries a signature tying it to a pure specification, and it *type-checks as an implementation of its pure model* `sortRangeL` — a check that fuses the ownership audit (the borrows are used safely) with dependent conversion (the mutated value matches the model's result). What that route leaves open is the model's *own* correctness, that `sortRangeL` returns a sorted permutation, which reduces to ordinary lemmas in the pure fragment. OPEN

The second removes the model. Each function is described *only* by its return type — a postcondition over the value the borrow holds at exit — and every proof step is built inside the imperative body from its callees' postconditions. Its quicksort's return type is sortedness of the exit list paired with count-preservation against the entry list, and there is no pure model of the partition or of the sort anywhere in that development; the properties one wants are proved where the mutation happens. GREEN This is the route the project chose, and carrying it to completion is what the paper reports.

The third keeps the second's architecture and changes the data structure: an `Array n T` whose sub-ranges are *places*, so a recursive sort hands its callee a sub-slice rather than the whole array and two indices. GREEN It matters for two reasons beyond the program. It removes a structural cost the `List` versions both pay — a list segment is not a place, so a contract must describe the part the callee must not touch — and, because it shares a specification with the second and no code at all, the two can be run against each other, which is a check on the specification rather than on either program.

Two caveats bound all three. Recursion is guarded by a *declared* decreasing argument, and with a sufficiency hypothesis discharging the out-of-fuel path the guarded recursion is total — but the guard is per-declaration and mutual recursion is rejected rather than supported. And because the v0 mechanization collapses the universe hierarchy to type-in-type — surrendering strong normalization to Girard's

paradox — everything here is "verified modulo Girard." Consistency is deferred project-wide, and the hierarchy returns with it.

1.2 The money test

The claim that *both* machineries are indispensable is best made by a program that only their fusion rejects. Take a length-indexed vector paired with its length as a dependent Σ-type, borrow it mutably, and push an element by mutating both fields in place — but *forget* to increment the stored length:

```
fn push (e : T, v : &mut Σ (l : Nat). Vec T l) = {
  match v { Pair(l, xs) => {
    *xs := VCons(*l, e, *xs);   // xs now holds a Vec T (S l)
    // *l := S(*l);             ← forgotten
  }}
}
```

No borrow checker rejects this: every write is memory-safe, the two field borrows are disjoint, nothing dangles. No *pure* dependent type checker rejects it either, because there is no mutation for one to watch. It is caught only by the single interpreter that, at the function boundary, audits the mutated value against the type it now owes — and that type must *compute*. The second field is expected at `Vec T *l`; with the length left at `l`, the deref `*l` is the unknown snapshot σ_1, the expected type is stuck at `Vec T` σ_1, and no constructor inhabits a stuck type, so the field — which now carries a `Vec T (S` σ_1`)` — fails to convert. The rejection is not a bespoke rule but conversion having nothing to say yes to (Section 10.6). Restore the increment and the two types meet by arithmetic. This program — accepted the instant the update is present, rejected the instant it is dropped — is the calculus's evidence that ownership and dependency do distinct, non-redundant work inside one checker.

1.3 Contributions

- **The calculus and its presentation** (Section 2): one grammar and four evaluation arrows — runtime read and write, comptime read and write — that carry all behavioral variation, organized by a construct/arrow table that serves as the map of the language.
- **The founding identification, and fording** (Section 3): the observation that the symbolic refinement of a matched scrutinee in LLBC [4] *is* Agda-style dependent pattern matching — one substitution mechanism — together with its architectural payoff. The kernel implements only the unification *solution* transition and its occurs check; no-confusion, constructor injectivity, UIP, and conflict discharge are a *derived library* inside the calculus, built from the identity eliminators and mechanically checked (Section 10.4, Section 10.6). The account is completed by *branch equations*: substitution rewrites a scrutinee's occurrences, but where the scrutinee is a stuck spine the checker must first *abstract* it, and abstraction is what leaves a body unable to name what it just learned. Binding the branch's own equation restores it, and costs nothing at the splits where substitution already sufficed.
- **Declared-and-checked backward specifications.** Where Aeneas [5] synthesizes a backward function to describe what flows back out through a borrow, DLLBC lets the programmer *declare* that description in the signature and checks the body against it. A lying declaration is rejected; the checked conformance of quicksort to `sortRangeL` is the payoff.
- **The carve: proofs licensing a reorganization** (Section 10.7). An `Array n T` former with index and range *place steps*, so that a sub-range is a place and a range borrow is an ordinary `&mut`. The rule that splits an array into segments is the calculus's first to *consume evidence* — it fires only on a comptime proof of `Le (add lo cnt) n` — and afterwards the two sub-borrows are disjoint because they are literally different subterms of one value tree, so no disjointness judgment exists anywhere in the system. Rust ships this capability and checks it at run time; the contribution is moving that check to compile time and deleting the failure mode, which is possible because there is a comptime fragment to pay the proof from.

- **The case study, three times** (Section 5): the in-place quicksort type-checked end to end as an implementation of its pure model; re-posed and re-proved with no model at all, its postcondition discharged inside the bodies; and then re-posed again over arrays, where a recursive sort hands its callee a sub-slice instead of the whole structure plus two indices. The second required three things the first never did — branch equations, a Σ-eliminator so that a caller can take a returned certificate apart, and the recursion guard below — and each is a place where removing the model exposed something it had been hiding.
- **A declared decreasing-argument guard, closing a live unsoundness.** Because a call is checked against a signature alone, a self-call was admitted at the function's own declared return type with no side condition: Hoare's recursion rule with its side condition deleted, under which `fn bad () -> Id Nat Z (S Z) { bad() }` was accepted. The hole was unreachable under the conformance architecture and immediate under the direct one, which is itself the paper's sharpest evidence for building both (Section 8).
- **Empirical metatheory:** a validated differential harness relating the symbolic checker to a concrete evaluator, and verified invariants of the checker itself — notably the discipline that refinement propagates *knowledge* (constructor shapes, equation solutions) and never *state* (holes, loan markers, mutation results), which instrumentation found violated exactly zero times and which is now asserted at its single substitution site. Two results promote the harness from assurance activity to part of the claim. It found a divergence whose *polarity* is inverted — the checker right, the machine broken — which is not something a design argument can reach, because the machine is what such an argument reasons from (Section 6.2). And two implementations of one specification, sharing no code, now check each other (Section 6.3), which is evidence about the *specification* that no single verified program supplies.
- **A measured comparison of two verification architectures** (Section 7): routing correctness through a pure model (the conformance route — effectively the Aeneas pipeline rebuilt inside one language) versus proving *propositional* postconditions directly in imperative bodies over exit snapshots. Both are landed on the same problem — though not, as Section 5 is careful to say, as the same program — so the comparison rests on two working developments rather than a projection; two of its findings — that the direct route does not merely relocate the pure model, and that it is cheaper to check by a wide margin — were not arguments the project had when it chose that route.

1.4 Roadmap and lineage

Section 2 presents the grammar, the four arrows, and the first programs run on the machine; Section 3 develops the identification of refinement with dependent matching and the fording library it makes possible. The remainder of the paper takes up the boundary discipline and the audit at return (Section 4), works the quicksort case study three times (Section 5), reports the differential harness and the verified invariants (Section 6), sets the two verification architectures side by side (Section 7), and closes with lessons (Section 8) and related work (Section 9). The complete rule set — seven figures, each ending in a rule-to-implementation-to-test correspondence table — is Section 10.

DLLBC is the mutation-stage vehicle of a longer program. It supersedes an earlier staging — a pure core with equirecursive subtyping and self-types [6], then an ownership layer, then mutation — by carrying ownership and dependency together from the start, which is where their interaction, the object of study, actually lives.

2 The Language at a Glance

DLLBC has a single syntactic category of terms. Types are terms of universe sort, and the borrow machinery — mutable borrow, dereference, and the borrow type — consists of ordinary term formers within it. The familiar stratification into a "pure" language of types and a "runtime" language of programs is not drawn syntactically; it is recovered *semantically*, as the fragment on which certain of the four arrows are defined.

2.1 Values are trees; there is no heap

Programs execute against an *environment* Ω mapping variables to *values*, and a value is a tree: a constructor applied to values, plus a few ownership-tracking forms. There is no heap and no addresses. A mutable reference is modeled by *moving ownership of a subtree to the reference* — the value becomes $\text{borrow}_m\,\ell\ v$, carrying its payload v under a fresh loan identifier ℓ — and leaving a marker $\text{loan}_m\,\ell$ where the subtree used to sit. The symbol $\bot$ marks a vacant slot: not a value of any type but the *absence* of one, excluded from every read-shaped rule.

This value semantics buys the calculus its central property. Types may mention program values, but only as *snapshots*: the (possibly symbolic) value a variable held at a fixed moment, written σ for an unknown one. A snapshot is a copy, never a location, so no mutation can make a type stale — the one guarantee the design exists to provide. Because a borrow *carries* its payload, a deref `*b` appearing in a type is a pure projection of that payload, $*\text{borrow}_m\,\ell\ v \rightsquigarrow v$, never a store lookup that could read the wrong thing.

Two regimes coexist over these values. Inside a function body, borrowing is *contract-free*: the checker is a symbolic interpreter running the very rules the runtime runs, and it simply watches each payload change. Contracts appear only at *boundaries* — function signatures, and borrows stored under a type constructor — the opaque places the interpreter cannot watch through. There is exactly one place a body's borrows are ever *judged* against a type: the audit at return.

2.2 The four arrows

All evaluation and checking is organized by four judgment forms over the one grammar. Two are runtime, two comptime; two read, two write.

	read	**write**
runtime	$\Omega \vdash t \Rightarrow v \dashv \Omega'$ — consume-read: evaluate t to a value with move semantics	$\Omega \vdash t \Leftarrow v \dashv \Omega'$ — destructive write: fill the location t denotes with v
comptime	$\Omega \vdash t \rightsquigarrow v$ — comptime read: evaluate in the borrow-free fragment	$S \vdash x \twoheadleftarrow t \dashv S'$ — comptime write: refine the snapshot layer by substitution

Table 1: The four arrows. Ω is the runtime environment; the comptime read leaves no Ω', being effect-free; the comptime write ranges over the whole checker state S.

The comptime pair is not a second semantics. It is the *same machine restricted to the borrow-free, assignment-free fragment*: comptime read threads Ω over pure entries, but excludes the constructs that touch the loan state (minting a borrow, consuming through one) and excludes assignment outright, so its only footprint on Ω is the fresh pure entries its `let`s introduce. "No side effects at the type level" is thus a *fragment property*, not a separate rule set — and it is what an eventual erasure pass will rely on, since types must be deletable without disturbing the state the runtime sees. The comptime *write* $\twoheadleftarrow$ is the checker's refinement judgment, fired on entry to a match branch; it has no surface syntax at all — no program text elaborates to it.

Which arrows are defined on which constructs is the skeleton of the language. Table 2 reproduces it. Two regularities organize everything that follows. *Down the columns:* the $\rightsquigarrow$ column is the $\Rightarrow$ column minus the borrowing rows, minus assignment, and minus consumption — the comptime fragment made visible. *Across the rows:* several constructs mean different things under different arrows — `*t` reads destructively, locates non-destructively, or projects a snapshot; `match` destructures or reborrows by the scrutinee's mode; a variable is a move, a write target, a snapshot read, or a refinement site — one grammar, with the arrows carrying all the behavioral variation. The rule figures (Section 10.1, Section 10.2, Section 10.3, Section 10.4, Section 10.5, Section 10.6) are the unpacking of this table, one region at a time; the prose below runs the machine on a handful of programs to fix intuitions first.

construct	$\Rightarrow$	$\Leftarrow$	$\rightsquigarrow$	$\leftsquigarrow$	note
`x`	✓	✓	✓	✓	$\Rightarrow$ is the move; $\Leftarrow$ fills a vacant slot; $\rightsquigarrow$ reads the snapshot; $\leftsquigarrow$ *is* refinement
Type_i	✗	✗	✓	✗	comptime value; no runtime representation
$\Pi(x : \tau) \to \tau'$	✗	✗	✓	✗	as above
$\lambda(x : \tau).t$	✓	✗	✓	✗	no capture, so formation is inert: $\Omega' = \Omega$ even under $\Rightarrow$
$t\ t'$	✓	✗	✓	✗	$\Rightarrow$: the call rule; $\rightsquigarrow$: β/ι-reduction
`T`	✗	✗	✓	✗	a comptime value
`T.c`	✓	✗	✓	✗	constructor values exist in both worlds; $\Rightarrow$ consumes the fields
`match t {…}`	✓	✗	✓[1]	✗	$\Rightarrow$: two modes by scrutinee type, branch entry refines by $\leftsquigarrow$ and may bind the branch's equation (Section 3); $\rightsquigarrow$: ι-reduction, stuck on symbolic
let $x = t; t'$	✓	✗	✓	✗	sequencing in both fragments
$t := t'; t''$	✓	✗	✗	✗	$\Rightarrow$: RHS by $\Rightarrow$, target by $\Leftarrow$; excluded from the comptime fragment
`&mut t`	✓	✗	✗	✗	mints a loan; `&mut *x` is reborrow
&mut $(s : \tau \hookrightarrow S)$	✗	✗	✓	✗	the borrow *type* is comptime, though borrow *values* are not
`*t`	✓	✓	✓	✓	the peel, arrow-generic: takes, locates, projects, or refines the payload
`a[i]`, `a[lo ; cnt]`	✓	✓	✓	✗	index and range *place steps* (Section 10.7); a range being a place is what makes a sub-slice borrowable. Both fire a **carve** when the place is first navigated

Table 2: The construct/arrow table: the map of the language.

2.3 First programs

We now run the machine, annotating Ω after each step. One standing convention governs the traces: *reorganization is lazy*. The environment may be rewritten between two steps — a loan ended, a displaced value dropped — but the checker does so only when some rule's premise demands it, never eagerly. The demand does the work.

Moves, and the vacant slot. A bare use of a variable is a $\Rightarrow$-read: it consumes.

```
let x = 3;          // Ω = x ↦ 3
let y = x;          // Ω = x ↦ ⊥, y ↦ 3
```

The second line is $\Omega \vdash x \Rightarrow 3 \dashv \Omega[x \mapsto \bot]$: the value moves out and $\bot$ is left behind. A second use of `x` is simply stuck — no read rule fires on $\bot$ — which is how use-after-move becomes the absence of a derivation rather than a special error. A vacant slot is refillable, and writing onto a slot that is *not* vacant forces a drop first, inserted lazily. One refinement earns its place from the mechanization's brush with index arithmetic: a $\Rightarrow$-read of an *index-kind* value — a concrete `Nat`/`Bool`/`Unit` tree, any pure-former value, or a symbolic one of such a type — is a *copy*, the owner keeping it (Section 10.1, R-Copy). On such values the ownership machinery is doubly vacuous; on data proper — `Cons`-trees, pairs, user constructors — a move stays a move, keeping Rust's line that silently duplicating aggregates hides cost.

Borrowing, writing through, and ending. Minting a borrow moves the payload to the reference and parks a loan marker at the source; writing *through* the borrow locates the payload without consuming the reference; and the loan ends only when a later demand forces it.

[1]The $\rightsquigarrow$ cell for `match` is aspirational. Comptime ι-reduction of `match` on the pure fragment is not implemented in the v0 mechanization; type-level case analysis holds today only through the recursors that `match` would *elaborate* to (the CIC eliminators of Section 10.3), pending the substitution discipline for binders. It is the one cell where this table runs ahead of the mechanization.

```
let x = 3;          // Ω = x ↦ 3
let b = &mut x;     // Ω = x ↦ loan_m ℓ, b ↦ borrow_m ℓ 3
*b := 7;            // Ω = x ↦ loan_m ℓ, b ↦ borrow_m ℓ 7
let y = x;          // demand for x forces End-Mut ℓ first:
                    // Ω = x ↦ 7, b ↦ ⊥;  then the move: x ↦ ⊥, y ↦ 7
```

The write `*b := 7` peels one indirection under ⇐ and replaces the payload in place; no contract governs it, because inside the body borrowing is contract-free. The last line is where laziness pays out: the move needs to read `x`, but `x` holds a loan and no rule applies, so the environment reorganizes first. G-EndMut (Section 10.2) plugs the borrow's current payload back where the marker sits and kills the borrow — itself just a ⇐-fill aimed at the environment's own bookkeeping — after which the move proceeds. Nothing marked *when* ℓ had to end; the demand for `x` did.

Take and refill. Under ⇒ the peel `*b` moves the payload out *through* the borrow, leaving a hole; the only rule then applicable at `b` is the ⇐-fill back through it. This is the in-place update idiom the calculus is built to make routine:

```
let x = Cons(3, Nil);
let b = &mut x;            // Ω = x ↦ loan_m ℓ, b ↦ borrow_m ℓ (Cons 3 Nil)
let tail = *b;             // Ω = …, b ↦ borrow_m ℓ ⊥, tail ↦ Cons 3 Nil
*b := Cons(7, tail);       // Ω = …, b ↦ borrow_m ℓ (Cons 7 (Cons 3 Nil)), tail ↦ ⊥
```

Between the take and the refill, `b` holds a hole — a state both Rust (error E0507) and LLBC [4] forbid, because a panic in the gap would end the loan with the owner recovering garbage. DLLBC has no panic and the hole cannot escape: ⊥ satisfies no read rule, and no function boundary can be crossed while an argument borrow holds one. No list node was copied, and under compilation the value's journey out and back is no journey at all.

Where the rules of ownership are. The reborrow `&mut *b` borrows a borrow's payload, suspending the parent until the child's loan ends; composed with drop it produces the calculus's classic stress test, the *self-reborrow* `b := c` after `let c = &mut *b`, which DLLBC deliberately *rejects* — the chain to be dropped runs through the very value being written, so no ending rule fires and drop is stuck (Section 10.2). Other calculi in this family accept it at the price of ghost variables; declining to pay keeps drop a total procedure and Ω free of inaccessible entries. The section's lesson survives the rejection: the borrow machinery has *no rules of its own* beyond bookkeeping. Every step above is ⇒ for the moves, ⇐-fill for every write, and two reorganizations — G-EndMut and drop — aimed at an environment that remembers who owes whom.

3 The Founding Identification

Inductive values are read by `match`, and `match` is the calculus's *only* eliminator — there is no field projection, no tag test, no destructor syntax. Field access *is* a match; borrowing the second field of a pair *is* `match b { Pair(_, snd) => … }`. Because a body's arguments are symbolic, matching must work on values the checker knows the type but not the value of, and it is here that a single observation organizes the whole design.

3.1 Refinement is dependent pattern matching

Inside a function body an argument is a symbolic value σ: an entry of the environment's pure layer whose type the checker knows and whose value it does not. When the checker matches such a scrutinee and enters the branch for constructor C, it performs a *comptime write at the scrutinee* — the judgment $S \vdash x \twoheadleftarrow C\ \overline{\sigma} \dashv S'$ (Section 10.4, M-OwnedSym/M-BorrowSym) — which substitutes $C\ \overline{\sigma}$ for σ in *every* entry and *every* type across the whole checker state at once. The environment's knowledge is upgraded wholesale: after entering the `S` branch of a match on `n : Nat`, every occurrence of the old snapshot has become `S` σ' for a fresh predecessor σ'.

This is dependent elimination in the Agda style: by *unification and substitution*, not by threading equality proofs through a motive. And it is *verbatim* what LLBC's symbolic interpreter [4] already does when it matches a symbolic scrutinee. The identification — that LLBC's symbolic refinement *is* dependent pattern matching, one substitution mechanism serving the borrow checker and the type checker alike — is

the founding observation of the calculus, and it explains a restriction that would otherwise look arbitrary: the scrutinee *must be a variable*. Substitution is defined on variables; an arbitrary expression has no substitutable identity to refine. The same fact fixes what refinement may carry — constructor shapes and equation solutions, facts true of the value timelessly, and never a hole, a marker, or a mutation's result, which are facts about a slot at a moment (Section 10.3, and the invariant reported in the empirical section).

In a body, this refinement is the runtime symbolic match, and it is fully in force. Case analysis at the *type* level — the comptime $\rightsquigarrow$-reduction of `match` on the pure fragment — is the one piece deferred to a later substitution discipline for binders (the qualified $\rightsquigarrow$-match cell of Table 2); the identification below concerns the mechanism that is implemented and load-bearing today.

3.2 The second layer: branch equations

Substitution is a complete account of branch knowledge for one kind of body and an incomplete account for another, and which kind you are in is decided by whether the body has to *produce* evidence. Under the conformance architecture (Section 7) it never does: the body carries zero proof obligations, so having every occurrence of the scrutinee rewritten is everything a body could want, and the identification above is the whole story. Under direct propositional proving the body must hand its caller a proof, and there the account runs out — at exactly one place, and for an instructive reason.

The place is a *stuck* split. When a match scrutinee whnf-reduces not to a bare σ but to a neutral spine — `leb` σ_{x} σ_{p}, the shape every comparison-driven algorithm produces — there is no substitutable identity to refine, so the checker first *generalizes*: it mints a fresh σ_{b} : Bool and abstracts the spine to it across the whole state (Section 10.3, X-Gen), after which the branch refines σ_{b} to `True` as an ordinary symbolic split. Everything that *already* mentioned the spine now reads `True`. But a body that writes `leb x p` *after* the split re-derives the spine from scratch, and what it derives is a term the abstraction never saw; nothing rewrites it, and the body learns nothing from the branch it is standing in. The natural thing to write, `Refl : Id Bool (leb x p) True`, is rejected — and rightly, on the calculus's own terms: `Refl` inhabits an identity only when its endpoints *convert* (Section 10.6, T-Ctor), and `leb` σ_{x} σ_{p} does not convert with `True`. That non-conversion is precisely what made the scrutinee stuck in the first place; had the endpoints converged there would have been nothing to generalize.

So the mechanism has two layers, and the imperative fragment had only the first. Motive abstraction — generalization, then refinement — handles *occurrences*. *Knowledge* is a separate obligation, and the pure fragment had always discharged it with a convoy motive, carrying `Id Bool` ⟨spine⟩ b through the elimination so each arm receives the equation as an argument. The rule below gives the body's `match` the same second layer:

$$\text{match}\ \ h : x\ \left\{\ \overline{\text{br}}\ \right\}$$

binds, in every branch, an equation h : Id τ ⟨the scrutinee's pre-split value⟩ ⟨this branch's constructor⟩. One binder serves the whole match, since only its *type* varies per branch. It is *declared*, not inferred: without the binder nothing is bound and nothing is minted, so no existing program changes. And it is sound for exactly the reason the substitution is sound — the branch is entered precisely when the scrutinee evaluates to that constructor, so the equation is a fact about the value, true at entry and true forever. That is the same standard the identification above imposes on what refinement may *carry* — constructor shapes and equation solutions, never a hole, a marker, or a mutation's result — now applied to what it may *reify*. The rule adds no new kind of knowledge; it makes the branch's existing knowledge citable.

The interesting part is what it costs, because the answer is: nothing, exactly where it was already free. The equation's content is entirely determined by what happened to the pre-split value. At an ordinary symbolic split the $\rightsquigarrow$ refinement (X-Shape) has already rewritten that value to this branch's constructor *everywhere*, so the equation's two endpoints are literally identical and h is inhabited by `Refl` — no symbolic value minted, no Γ_σ entry, nothing carried. On a concrete scrutinee, likewise. Only at a stuck split is the equation a genuine hypothesis, and only because generalization is the one thing that makes

a pre-split value *unnameable* afterwards. The implementation follows the distinction exactly: X-Gen returns the normalized pre-abstraction spine alongside σ_{b}, and the branch setup mints a single fresh σ typed `Id Bool` $\langle$spine$\rangle$ $\langle C\rangle$, registered in Γ_σ *before* the refinement fires so that it is swept along with every other σ-bearing component of the state — the same discipline that keeps obligations and the pinned return type current, and therefore no new update path. (One narrowness is inherited: X-Gen mints only `Bool` generalizations, so the hypothesis form available today is always an `Id Bool`.)

The shape is standard — Lean's `match h : x with`, Rocq's `destruct … eqn:` — and saying so is the point rather than a caveat: the calculus reaches the same construct from the other side. In a conventional elaborator the equation is *always* a real hypothesis, because nothing substitutes into the goal on the programmer's behalf; here substitution does the work almost everywhere, and the equation is needed only in the one case where abstraction has *replaced* substitution. The two-layer decomposition is thus not a design choice imported from prior art but a distinction the symbolic interpreter forced, and the prior art is what confirms the resolution.

What it buys is the relational partition, and through it the case study's headline. The partition's postcondition bounds its two parts (`Ub p (*v)` for the part it keeps, `Lb p hi` for the part it returns); its body branches on `leb x p` and must therefore *produce* `Le x p` on one side and `Le p x` on the other, from the branch it took. Both come from the equation and from nothing else — `leb_true_le x p h` on the `True` side, and `leb_false_gt` weakened by one on the `False` side. Neither derivation exists without the rule, and the mechanization keeps the pair adjacent on purpose: the same body with `Refl` in place of h is the rejected wall test, and the same body with h is accepted, one binder the only difference between them. Downstream of that partition is the whole-list quicksort whose return type is `Sorted (*v)` together with count-preservation against `old *v`, carrying no declared backward specification anywhere in its call tree.

3.3 Fording: a minimal kernel and a derived library

The deepest payoff of the identification is architectural, and it shows in how the calculus handles *equations*. Matching a proof of an identity type is the refl-match: matching `Refl` at `Id A a b` whnf-unifies the two endpoints. If one endpoint is a *flexible* σ — a substitutable symbolic not occurring in the other side — it is *solved* to the other, and the scrutinee's own snapshot is then refined to `Refl` (Section 10.4, M-Refl-Flex Section 10.3, X-Sol). The occurs check is real: a σ appearing on both sides is not flexible. This *solution transition*, together with its occurs check, is the *entire* unification vocabulary the kernel implements. When both endpoints whnf to rigid heads, the match is simply *stuck* — there is no rule.

Everything a full dependent-pattern-match unifier would additionally do — no-confusion between distinct constructors, injectivity of a constructor, uniqueness of identity proofs, discharge of a conflicting equation as absurd — is *not* a kernel operation. It is a *derived library*, written inside the calculus as ordinary well-typed terms over the two identity eliminators (Paulin-Mohring `j` and Streicher `k`, Section 10.6, T-Elim-J/T-Elim-K): constructor injectivity is a term `injProof` built from `j`; UIP is `uipProof` built from `k`; no-confusion for `Nat` is a `j`-spine landing in a reducing code type that computes to $\bot$. These are *mechanized facts*, checked in the same kernel as every other program, with no machine rule beyond the eliminators' own typing. This is *fording*: rather than teach the kernel a unification algorithm, the calculus keeps the kernel to substitution-plus-occurs-check and recovers the algorithm's consequences as a library of provable identities. The metatheoretic surface shrinks accordingly — there is one transition to trust, not a case analysis of unification steps.

3.4 The recursive-family encoding

Native indexed families arrive with the fording kit; until then, an indexed type such as `Vec` lives in the calculus as its *recursive-family encoding*, computed from its index by large elimination:

$$\mathrm{Vec}\ T\ Z \equiv \top, \quad \mathrm{Vec}\ T\ (S\ n) \equiv T \times (\mathrm{Vec}\ T\ n),$$

with `VNil` and `VCons` as notation for the unit value and pairing, and the index argument vanishing. The load-bearing property is that the type *computes under a stuck index*: `Vec T (S σ)` unfolds to `T × (Vec T σ)` even while the length σ is unknown, which is exactly what large elimination provides and exactly what a checker running over symbolic arguments needs. (One consequence is presentation-specific and

does not survive to native families: with no index argument at the constructor, more write orders type-check than the indexed form would force; that sharpening returns with native `VCons`.)

3.5 The money test, mechanized

The mechanism behind the money test of the introduction is now nameable. A constructor application is checked by *inhabitation against a type that must compute* (Section 10.6, T-Ctor): the expected type is reduced toward canonical form, and the constructor's fields are checked against the arguments its signature then demands. A *stuck* type — one whose head will not reduce because it awaits an unknown snapshot — admits no constructor; the check is a table lookup that misses, and the audit returns false. This is not a rule against forgotten updates; it is the ordinary constructor-checking rule meeting a type the program left un-computed. In the Σ-paired vector push, the omitted `*l := S(*l)` leaves the second field expected at the stuck `Vec T` σ while the value on hand is a `Vec T (S` σ`)`; the two do not converge, and the boundary audit rejects. Supply the update and the expected type computes to a form the value inhabits. The dependent guarantee is carried entirely by *what conversion has to say yes to*, and the ownership machinery — suspension, contract-free siblings, the single boundary audit — is what makes the in-place mutation that provokes the check safe to write in the first place. The two are indispensable together, and they meet in one interpreter.

4 Ownership across boundaries

Inside a function body, borrowing is contract-free. The checker is a symbolic interpreter running the very rules the program will run, so it simply watches each move, write, and reborrow go by; no borrow needs to announce what it will become, because the checker already knows. Contracts exist for the one thing the checker cannot watch: **opacity**. There are exactly two opaque sites. A **function boundary** is checked once, against a signature, and every caller sees only that signature — never the body. And a **borrow stored under a type constructor** — an element of `List (&mut T)`, a field of a pair — needs a type that stands on its own, apart from the environment that would otherwise track it. Only at these two sites does a borrow's type carry an obligation.

This section is the contract layer. It builds up in three moves: the shape of the obligation (the &mut ($s : \tau \hookrightarrow S$) type); how a call installs one and later collects it (calls as wires, and then, when borrows entangle, as groups); and how a body discharges it (the audit at return — the *single* check in the whole borrow story). The rules live in Section 10.5, the reorganizations they lean on in Section 10.2; we cite rule names and do not restate them.

4.1 The obligation as interface

$$\&\text{mut}\ (s : \tau \hookrightarrow S)$$

reads: exclusive access to a value of type τ, and across this boundary a value of type S is owed. The binder s names the payload *at entry*, for use in S — which is checked at *exit*, when the entry payload is otherwise long gone. The obligation is type-changing: S need not be τ. A push signature

```
fn push (n : Nat, e : T, v : &mut (Vec T n ⇝ Vec T (S n))) = …
```

tells the caller, from the signature alone, that the vector behind `v` is one longer at exit. We write &mut ($\tau \hookrightarrow S$) when S ignores the binder s, and plain &mut τ when moreover $S = \tau$.

This is the paper's first inversion of the Aeneas pipeline [5], and the cleanest. Where Aeneas *synthesizes* a backward function per borrow — a pure function describing what flows back through it — DLLBC moves that description into the signature and makes it the programmer's to state: the $\hookrightarrow$ obligation is the backward function's *type* (B-Seed-Borrow installs it at function entry, instantiating the binder s at the entry snapshot). A structuring observation, worth stating because it recurs in the case study: the Σ-paired presentation of a length-carrying vector, &mut $\Sigma(l : \mathrm{Nat}).\ \mathrm{Vec}\ T\,l$, sidesteps $\hookrightarrow$ entirely — it owes back exactly the type it received, the length change riding *inside* the invariant type rather than *across* it. Both forms are legitimate; the Σ form buys an invariant obligation at the price of carrying the index in the data, and $\hookrightarrow$ earns its keep when the changing index lives outside the borrow, in the telescope.

Snapshots in signatures. A signature is a telescope: each argument's type may mention earlier arguments. For borrow arguments, later types reach the payload with the comptime deref $*b$, evaluated ($\rightsquigarrow$) wherever the type is consulted, so it denotes the entry snapshot:

```
fn nth (b : &mut List T, i : Fin (len *b)) -> &mut T
```

Out-of-bounds is unrepresentable: `Fin (len` $*$ `b)` replaces the error monad. The division of labor is exact — $*b$ means *the payload now* (projected wherever the type is read), while s means *the payload at entry* (bound once, for the one position that must outlive its moment). The projection is sound precisely because this is a value semantics: a borrow *carries* its payload, so $*\,(\text{borrow}_m\,\ell\ v) \rightsquigarrow v$ is a pure projection, never a store lookup and so never stale (its one proviso: a suspended borrow, its payload mid-reborrow, has no meaningful snapshot, and the projection is stuck until the reborrows end).

One convention the mechanization pinned down and B-Seed encodes: a later parameter's type mentions an earlier argument by its *runtime variable*, resolved to the snapshot by $\rightsquigarrow$ each time the type is consulted — the telescope performs no cross-parameter substitution at seed time. A pure binder substituted eagerly at seed time would sit unsubstituted under weak-head evaluation, read as a rigid neutral, and be mistaken for rigid at a refl-match that should have seen it as flexible. The caller side mirrors this (B-Inst): each checked actual is bound into the running instantiation before the remaining parameter types are consulted, so `Fin (len` $*$ `b)` at a call site means the `b` just passed. A practical corollary the mechanization hit: because arguments are consumed left to right, a later argument may not mention a borrow an earlier argument of the same call already consumed — a bounds proof about $*v$ must be `let`-bound *before* the call that consumes `v`.

4.2 Calls as wires, then as groups

A call is checked against the signature alone — recursion forces this, the checker cannot unroll a call to itself, and all calls get the same treatment uniformly. The rule (B-Call) is: consume the arguments; for each argument borrow, annotate its loan with the owed type from the signature, instantiated at the actuals; mint a fresh symbolic result. In the simplest case each returned or retained borrow corresponds to exactly one argument loan, and the $\hookrightarrow$ obligation says everything — a **wire**. Ending a wire (Section 10.2, G-EndOwed) is where the caller learns what the callee did: a fresh value arrives at the owed type, an existential opened at loan-end, and *how much* the caller learns is exactly how much the signature says. Under &mut `List T`, only that a list came back; under the `Vec` signature above, its precise new length. The spec is the type; precision is bought by strengthening it.

Some functions break the one-borrow-one-loan correspondence:

```
fn choose (c : Bool, x : &mut T, y : &mut T) -> &mut T = match c { True => x, False => y }
```

The returned borrow points into `x` or `y` depending on `c`, and no per-borrow promise can say which. Treating the loans independently is *unsound*: if `x`'s loan could end while the returned borrow still lived, the caller would recover `x` while an exclusive borrow into (possibly) `x` was still active. What must be recorded is the **grouping** and an ending **order**. A call whose borrows entangle mints a **loan group** (B-Call): a node tying the loans it captured to the borrows it issued. Its whole content is its ending discipline (Section 10.2, G-EndGroupOpaque): every issued borrow ends first, then the group ends atomically, releasing each captured loan with a fresh existential. The order *is* the soundness argument made structural — `x` cannot recover while the result lives, because the group holds `x`'s loan and cannot end until the result's does. A wire is simply the degenerate group, one captured and one issued, so the two sections are one mechanism at two precisions.

This is where DLLBC pays for having *no lifetime annotations anywhere*. Rust ties a returned reference to its inputs by naming lifetimes and threading them through the signature; DLLBC names nothing. The loan group *is* the tie, minted automatically from the signature at the call, and the ending is demand-driven — nothing in the program marks *when* a group ends; a later demand for a captured owner forces it. The honest cost is precision. A single group ties *all* captured loans to *all* issued borrows and releases them together; a system with named lifetimes could free a captured owner whose borrow provably never reached the result, where the group must hold it until every issued borrow dies. The atomic release also

over-approximates the concrete dynamics, where each captured loan ends only when its own owner is demanded — a symbolic environment can hold a released existential where the concrete one still holds a suspended loan, an obligation any simulation relation must reconcile (Section 10.2 records it). The trade is deliberate: no annotation burden, the coarsest sound tie.

Self-calls need a side condition. Checking a call against the signature alone has one consequence that is invisible until a return type carries real content: a *self*-call is admitted at the function's own declared return type. Under the conformance architecture that is harmless, because the declared `back`'s conversion is where the work happens; under a propositional postcondition it is Hoare's rule for recursion with its side condition deleted, and every false postcondition proves itself (Section 8 gives the two-line witness). The side condition is a *declared* decreasing position `[k]`, and the checker being a symbolic interpreter is what makes it cheap: a self-call is admitted only when the actual at position k is a strict structural subterm of that parameter's *current snapshot*, which inside `match n { S(m) => … }` is already `S` σ_{m} while the actual is σ_{m} — ordinary structural comparison, no measure and no termination checker. Three scoping decisions come with it. The index is declared rather than inferred, because "some argument decreases at each call" is unsound (two branches can each shrink a different coordinate while growing the other, forever) and because paths are explored independently, so no inferred index could be committed to across them. A *borrow* parameter may decrease through its payload snapshot — the guard in its most literal form, and the only thing that shrinks in a list cursor, sound because snapshots are entry-knowledge that mutation never rewrites. And mutual recursion is rejected outright by call-graph reachability: a per-declaration guard would let `f` → `g` → `f` admit each other's postconditions with nothing decreasing anywhere, the same hole through two doors.

4.3 The audit at return

The callee's side is symmetric, and it is the only check in the whole borrow story. Before returning, every argument borrow must hold a value of its owed type, verified by conversion (B-Audit-Val, and its borrow-returning reshaping B-Audit-Borrow). One step hides inside "holds a value": the audit is itself a demand, and it *collapses first*. The suspension tree a borrow-mode match built — field loans parked in the parent, unobservable through the whole body because nothing before the boundary can demand them — is ended here by G-EndMut (Section 10.2), the boundary being the canonical demander of an argument borrow that has no owner to demand it. Then conversion judges the collapsed payload. The audit is *collapse-then-convert*: it assembles the final value out of the body's strong updates, reassemblies, and holes, and checks the assembled thing once, against the type the signature owes. A hole ($\bot$) satisfies no type, so a function cannot return one, and a broken invariant left mid-body must be mended by here — a body may pass through any number of states its signature could never describe, because the signature only ever speaks about this one.

The return type itself is fixed at *entry*, while the parameters it may mention are still live (B-Pin): a dependent return type over a consumed parameter means that parameter's *entry* value, since re-reading at return would find $\bot$. One position is deliberately excepted, and it is the propositional main line's central move (Section 7): a *borrow* parameter's payload deref reads the **exit** snapshot, with `old` $* v$ as the entry operator. The justification is the loan structure itself — with no lifetimes the callee's access ends exactly at the audit, which already computes the collapsed final payload, so "the value at the end" names one well-defined tree — and caller-side the call mints one symbolic value shared between the loan's eventual release and the returned evidence's subject, so a returned proof *attaches* to the recovered value. Both readings are implemented at this pin; earlier versions of these rules pinned entry wholesale and flagged the difference as a gap. One admission rule rides along (B-ExFalso): a branch whose result is an eliminator applied to a verified inhabitant of $\bot$ is dead, and the audit admits it at *any* return type — the $\bot$-witness is the only thing checked. This is how a bounds-checked cursor's `Nil` branch "returns" a borrow it cannot have: it does not, and need not.

4.4 Declared backward specs

Opacity is real: after a write through the result of `choose`, the caller cannot prove which input received it — that is exactly what the group forgot. The general remedy indexes precision by the status of the

callee's *backward flow*, what the group does at its end, and the trichotomy is the familiar one from proof assistants. It may be a **parameter** — the opaque group above, an abstraction boundary where the caller learns only types; a **definition** — transparent, the flow being definitionally the callee's body, full precision, no annotation; or a **spec** — a stated obligation, checked against the body and hiding the rest. This document's core is the parameter case; the other two are sharpenings of the group-end rule.

The spec case completes the Aeneas inversion. If $\hookrightarrow$ is the backward function's *type*, a declared `back` is the backward function *itself*, moved into the signature and audited against the body that claims to implement it, rather than synthesized from it. The check is almost free, and for a structural reason: the suspension tree the body actually built — the captured borrow's final payload, with the issued loans' markers read as holes — *is* the backward function the body implements. Auditing the declaration is therefore one conversion, the declared `back` applied to its holes against the resolved tree (B-BackN). At the caller, a spec-carrying group ends by *computing* each captured release — the declared back applied to the surrendered payloads — instead of minting a fresh existential (Section 10.2′s G-EndGroupBack, consuming the spec that Section 10.5′s B-Call instantiated). And composition falls out of the same resolution: the tree resolves *through sub-calls' declared backs*, so a captured loan of a sub-call resolves to *that* call's back applied to its issued borrows — exactly LLBC's backward-function composition, here as a checked declaration rather than a synthesized term. Composition is all-or-nothing: a spec-carrying function's whole call tree must be spec'd, since an opaque sub-group leaves an unresolvable marker no spec converts against (recursion is fine — a recursive call resolves through the function's own declaration).

That declared backs must be *audited* and cannot be *inferred* is the load-bearing soundness fact, and it earns a statement of its own.

Backward flow is not a function of the signature. Consider `through (b : &mut List T)` $\rightarrow$ `&mut List T = b`, which returns its whole argument borrow, and an `advance` of the *same* signature that returns a field reborrow of the tail. At the caller, the two demand different releases: ending `through`'s group should restore the owner to the value written through the result, while ending `advance`'s should restore only the field that was reborrowed and leave the rest. The signatures are identical; only the bodies differ. Any rule that reads a constrained backward flow off the signature is therefore unsound — and an early version of this calculus did exactly that, inferring the constrained wire from the signature, and was refuted by this pair. The correction is not a cleverer inference but a different discipline: backward flow must be *promised* (a declared `back`) and *audited* (B-BackN against the body). A promise the checker verifies is sound where an inference the checker guesses is not.

The principle has since decided a case it was not written for. The array era's carve (Section 10.7) must decompose a segment's extent, and where that extent is a telescope parameter's symbolic value, solving the decomposition by refinement silently constrains the function's *callers* — a body carving `[Z ; i ; S j]` out of `Array n` was accepted, and so was a caller passing $n = 2$ with $i = j = 5$, which execution then got stuck on. Different mechanism, different milestone, same sin: a body imposing a cross-boundary constraint that its signature does not record. The ruling followed from the precedent rather than from fresh analysis — the carve may not refine a parameter's extent, and must instead have the decomposition hold by conversion or *cite* it as a checked equation. What makes the precedent worth stating as a principle rather than a fix is that it arrived pre-argued for a rule nobody had imagined when it was set.

The audit has a value-returning dual, and stating it precisely matters because its reach is limited. An in-place mutator that returns a plain value issues no borrows, so its declared back is the *zero-hole* case — the spec applied to no holes, converted directly against the argument borrow's resolved suspension tree (B-Back0). Until this dual was implemented, a value-returning body's back went unchecked at its own audit; a lying variant of a partition scan passed because only the borrow-returning branch ever looked. The lesson generalized to a testing discipline — a negative test per rule *branch*, not per feature. The dual's reach is precise: it checks a back authored *as the raw tree* — a scan that is literally the

composition of its sub-calls' declared backs converts — while a back that *reformulates* the tree (a swap declared as swapL $ij * v$ against the set-based composition it actually implements: semantically equal, never definitionally so) falls outside conversion's reach and is left to the differential harness to validate. Complementary, not redundant. The principled close is deferred: admit a reformulated back with a *cited bridging equation* and have the audit rewrite along the proved `Id`, so reformulation becomes checked rather than trusted. PROPOSED

4.5 Opacity, twice discovered to be doing work

Everything above treats opacity as a cost: the thing contracts exist to compensate for, and the reason a caller learns only what a signature says. The array era found it twice in the opposite role, and both times the finding inverted a claim this project had written down.

The first is Section 6.2′s: a call re-mints the callee's payload at the declared type, and that re-mint turned out to *repair* an executing machine that had lost the structure its borrows were pinned to. Forgetting was correct and remembering was broken.

The second decides a program's shape. A body that peels an element must first match its own length, which rigidifies the extent; but a rigid extent cannot then be carved at a symbolic offset. So one body cannot both choose a split point and carve at it — not as a transparency-versus-abstraction trade, but as a dead end. The way out is a function boundary, because the re-mint hands the array back *uncarved and with a flex length*, which is exactly the state a carve needs. The array partition is a separate declaration for that reason and not for modularity, and the design note's own advice — carve inline, reach for a function only when you want abstraction — was withdrawn rather than qualified.

The corrected statement generalizes past arrays: **carving is a within-body mechanism, and function boundaries are where its guarantees stop and start again.** Cross a boundary to change rigidity regimes. That a calculus's opacity mechanism should be load-bearing for expressiveness, rather than only for modularity, is not what Section 7′s framing would predict, and it is worth holding onto as a caution against reading opacity purely as loss.

4.6 The precision spectrum, and its open holes

The spectrum — opaque parameter, checked spec, transparent definition — is the organizing picture, and the checked parts of it are green in the mechanization: the wire and opaque group ends, the ↳ audit, and the declared-back audit in both its borrow-returning and value-returning forms all discharge on the test corpus. What a full soundness statement would additionally need is *not* yet green, and the extraction recorded three audit-strategy holes precisely so that no soundness claim quantifies over declared specs before they are closed. Stating them is a feature of this paper, not an embarrassment.

These three are current behaviors of the green mechanization, stated as the limitations they are. First, a declared back on a call that captures *two or more* borrows is silently *ignored*: the group-end matches its spec branch only against a single captured loan and otherwise falls through to the opaque arm, degrading to an existential without rejecting. A declared promise silently unused is a bug class — the fix is to reject or to support, never to drop. Second, there is a read/write asymmetry on group-captured owners: *reading* such an owner demand-ends its group, but *overwriting* it is rejected as in-flight, because drop kills the captured borrow without routing through the group's ending. The locatability phrasing that governs ordinary ends does not predict the split. Third, and most soundness-relevant: both declared-back checks take the first qualifying obligation and pass *vacuously* when none qualifies — and the value-returning check also passes when the argument borrow was consumed whole into a sub-call, so a declared back can go entirely *unaudited* on such a path. None of the three is exercised by a current test; each is an identified hole in the audit strategy that any soundness statement over declared specs must close first. The conformance architecture that Section 7 measures rests on exactly this declared-back machinery, which is one reason the paper treats it as a baseline to be scrutinized rather than a settled result.

5 Case Study: A Verified In-Place Quicksort

The preceding sections introduced the four arrows one region at a time; this one assembles them into a real algorithm — three times. The target is an in-place quicksort over a mutable borrow. The mechanization verifies it under *both* of the architectures Section 7 sets side by side, which is what turns that comparison into a measurement; and then verifies the second architecture again over a different *container*, which is what turns the differential harness into evidence about the specification rather than only about the checker.

The first pass, through the end of §5.6, is architecture A. The program is the one a Rust programmer would recognise — a swap-based Lomuto partition, recursion on sub-ranges, no allocation and no rebuilding of sublists — and it is checked not as a bespoke development but as an ordinary *implementation of a pure model*: each imperative function carries a declared backward specification (Section 10.5), and conformance is conversion. We build it from the bottom: the slice representation, the borrow-returning cursors, the bounds proofs that ride the telescope, the smallest self-contained instance of the architecture, the partition, and the recursion.

The second pass (Section 5.7) is architecture B, and it is the project's mission result: the same problem with *no declared backward specification anywhere in the call tree*, each function described only by a return type that is its postcondition, and every proof step built inside the body from its callees' postconditions. Reading the two in sequence is the point — the first shows what a pure model buys and what it costs, the second what is left when you take it away.

The third pass (Section 5.8) keeps architecture B and changes the *container*: an `Array n T` whose sub-ranges are places, so that a recursive sort hands its callee a sub-slice instead of the whole structure plus two indices. It is not a third architecture. It is the second one again, over a representation that removes the frame problem §5.1 describes — and because the two share a specification and no code, running them against each other is a check on the specification that neither could perform alone (Section 6).

Every listing in this section is quoted from the mechanization at the pin, and every acceptance and rejection claim it makes is a green `native_decide` proposition unless a status tag says otherwise.

5.1 The slice, as a borrow and a bound

DLLBC has no heap — no addresses, no aliasing through pointers — which is exactly what lets a type mention a value's *snapshot* and never go stale (Section 2). A "mutable slice", then, cannot be a pointer-and-length into a backing store; it is represented as what a slice fundamentally *is*, a fat pointer written honestly: a mutable borrow of the underlying list, paired with a comptime length bound. The pair `(v : &mut List Nat, n : Nat)` is the calculus's slice, and a sub-range is named not by a second borrow but by an offset `lo` and a count `cnt` measured against the same `v`. Suffix sub-slices are tail reborrows (`&mut *v` peeling one level, Section 10.2); prefix recursion rides the bound rather than a prefix borrow. This is a deliberate scoping of the claim: true random-access arrays with $O(1)$ indexing are a compilation-story question, outside the calculus, and the segment vocabulary here (`len`, `take`, `drop`, all of which compute) is `List`-shaped. What the calculus *does* claim, and what the rest of this section checks, is the load-bearing half: $O(1)$-extra-space *swap-based mutation through borrows*, with the segment's untouched remainder pinned to its entry snapshot by the ↳ obligation.

The scoping has a cost that the two `List` developments in this section pay, and Section 5.7 returns to it: because a list segment is not a *place*, a sub-range can never be handed to a callee as a borrow, so a contract must describe the whole list including the part the callee must not touch.

That cost is what the third development removes. A flat `Array n T` former, an index and a range step in the place grammar, and a lazy *carve* reorganization that fires only when handed a comptime proof of `Le (add lo cnt) n` make a range into a place — so two range borrows of one array coexist not because the checker believes an aliasing argument but because they are literally different subterms of one value tree, and the suspension machinery of Section 3 already knows what to do with those. The frame is then preserved by construction rather than described by a contract. GREEN It is built, and Section 5.8 is

the third quicksort: the same propositional architecture as §5.7, over a container where a sub-slice is a thing you can point at.

5.2 Cursors, and the disjointness problem

Element access is a borrow-returning recursion. The bounds-checked `nth` walks the list behind a borrow and hands back a mutable borrow of the requested cell:

```
fn nth (v : &mut List Nat, i : Nat, p : Le (S i) (len *v)) -> &mut Nat {
  match v {
    Nil => botElim Unit p,
    Cons(hd, tl) => match i {
      Z => &mut *hd,
      S(k) => nth(&mut *tl, k, p)
    }
  } }
```

There is a problem the moment a swap needs *two* cells at once. Calling `nth` consumes its borrow argument: after `nth(v, i, p)`, the borrow `v` is gone — captured into the call's loan group (Section 10.5) — so a second `nth(v, j, q)` has no `v` left to pass. Two sequential calls cannot yield two simultaneously-live cursors; the first consumes the slice. This is the calculus's version of the constraint that in Rust forces `split_at_mut`, and the resolution is the same shape: a single call that issues *two* borrows from *one* captured loan.

```
fn nth2 (v : &mut List Nat, i : Nat, j : Nat,
         pij : Le (S i) j, p2 : Le (S j) (len *v)) -> Σ (x : &mut Nat) → &mut Nat {
  match v {
    Nil => botElim Unit p2,
    Cons(hd, tl) => match i {
      Z => match j {
        Z => botElim Unit pij,
        S(jjv) => Pair(&mut *hd, nth(&mut *tl, jjv, p2))
      },
      S(k) => match j {
        Z => botElim Unit pij,
        S(jj2) => nth2(&mut *tl, k, jj2, pij, p2)
      }
    }
  } }
```

`nth2` is this calculus's `split_at_mut`: one call captures the single argument loan and issues the two borrows of the returned pair, tied together in a loan group whose ending discipline keeps them exclusive of the owner until both die (Section 10.5). The two cursors are disjoint *by construction* — they point into distinct fields of the same `Cons`-spine, held apart by the suspension tree — and this disjointness is what makes the swap through them a genuine in-place exchange rather than a copy. With the pair in hand, the swap is the take-and-fill idiom applied twice: `let t = *ei; *ei := *ej; *ej := t`, three destructive writes through the two reborrows, no list node copied.

5.3 Bounds proofs that ride the telescope

The cursors above carry proofs, and it is worth seeing how little those proofs cost. The precondition `p : Le (S i) (len *v)` is an ordinary telescope entry whose type mentions the payload snapshot through the comptime deref `*v` (Section 10.3); it is refined per branch exactly as the scrutinee is. Three facts make the discipline lightweight. First, the `Nil` branch is a $\bot$-*discharge*: there, `p` has type `Le (S i) 0`, which reduces to $\bot$, so the branch is dead and is closed by `botElim` — the ex-falso admission rule (B-ExFalso, Section 10.5), which lets a provably-dead branch "return" a borrow it could never actually hold. Second, the recursive call passes the *same* proof unchanged: `Le (S (S k)) (S j')` `equiv` `Le (S k) j'` holds *definitionally*, so descending the list needs no transport lemma — the bound simply reduces along the recursion. Third, an out-of-bounds access is rejected not inside `nth` but at the *call site*, because there is no inhabitant of the false bound to supply:

```
let x = Cons(1, Cons(2, Cons(3, Nil)));
let bb = &mut x;
swap(bb, 0, 4, (), ());   // p2 : Le (S 4) (len [1,2,3]) = Le 5 3 = ⊥
let y = x;
```

This program is rejected with "does not have its parameter type": the actual `()` supplied for `p2` would have to inhabit `Le 5 3`, which whnf-reduces to ⊥, and ⊥ has no inhabitant. At concrete, in-bounds calls the same proofs are the trivial `()` — `Le 1 2` and `Le 3 3` both reduce to ⊤ — so the cost of dependency is paid only where a bound is genuinely at stake. `Fin`-style safety, with the error monad replaced by a type that cannot be constructed when the access is illegal.

5.4 The smallest complete instance

Before the partition, one function isolates the entire architecture in five lines. `certSwapCount` borrows a symbolic list, swaps two positions in place, and returns a *certificate* that the swap preserved the element counts:

```
fn certSwapCount (s : List Nat, m : Nat, i : Nat, j : Nat,
      pij : Le (S i) j, p2 : Le (S j) (len s)) -> Id Nat (count m (swapL i j s)) (count m s)
      { let cert = count_swapL' m i j s pij p2;
        let b = &mut s;
        swapS(b, i, j, pij, p2);
        cert } }
```

Everything the case study is about is present here at minimum size. The body performs a genuine imperative mutation (`swapS` through the borrow `b`); the boundary machinery recovers, precisely, that after the swap `s = swapL i j σ` — the pure model of the swap applied to the entry snapshot; and the returned `cert`, a `count_swapL'` proof computed over that entry snapshot, is accepted at the return type because its subject `swapL i j s` is *definitionally* the recovered value. Imperative mutation, precise recovery of the mutated value as a pure term, and a pure lemma certifying a property of it — end to end, checked by conversion. The check is not vacuous: the negative control `certSwapCountLie`, which claims the count *grew* by one, is rejected ("does not have return type"), because the certificate proves equality and the value-returning audit converts the real return type against the declared one.

5.5 Partition: a gap-counter Lomuto

The partition is Lomuto's, with one design choice forced by the calculus. It maintains a boundary index `i` and a *gap counter* `g` — the number of scanned elements greater than the pivot — and decides at each step, from the comptime read `leb (nth (add i g) *v) pivot`, whether to advance the boundary or grow the gap, swapping the boundary and scan cells only on the `g = S g'` case. The structural reason for the gap-counter form is `nth2`: a textbook Lomuto swaps element `i` with element `j`, and its very first iteration can have `i = j`, a *self-swap*. But `nth2` cannot issue two cursors to the same index — its two borrows are disjoint by the suspension tree, and aliasing them is exactly what it forbids — so the scan is organised so that the swapped positions are never equal, and the base case is split on `i` to make the `i = 0` placement a literal no-op (`*v`) rather than the stuck `swapL 0 0 *v`.

One machine feature earns its place here. The scan branches on `if leb x pivot` with `x` symbolic, and the scrutinee does not reduce to a bare σ but to a *stuck spine* `leb σ σ_p`, which the substitution-based refinement ↜ cannot split — there is no variable to substitute. The checker *generalizes*: on an owned stuck Bool spine it normalizes the spine and abstracts it into a fresh σ_b : Bool across every σ-bearing component of the state (the X-Gen rule of Section 10.3), after which the ordinary owned-symbolic split refines σ_b to `True`/`False` per branch. Both the spine *in the scrutinee* and the same spine *in a pinned return type* refine together — which is precisely what generalizing across all σ-bearing state delivers, and a dedicated probe (`stuckProbe`, whose two `boolRec` sides converge only per branch) confirms it, together with negative controls for non-convergence and for non-exhaustiveness.[2]

[2]The generalization is currently `Bool`-specific — `generalizeStuck` mints only a fresh `Bool` symbol — sufficient for the `leb`/`if` splits the corpus needs but narrower than a fully general rule; this is recorded as an extraction finding.

The scan is declared `back = partScanL pivot k i g (*v)`, its pure model, and the callee audit checks it *per path* by conversion: the body's suspension tree — the composition of the recursive call's declared back with `swapS`'s — is definitionally that unfold. The recursion threads a length equation (`hlen : len *v = S (add k (add i g))`) whose transports are definitional totals plus a single `len_swapL` bridge in the swap case, and a lie in the declared back (the boundary and gap indices swapped) is rejected because the composed tree no longer converges. The subrange variant `partScanRange` carries the honest invariant of a sub-slice — an *inequality* `Le (add lo (S (add k (add i g)))) (len *v)`, since a sub-range does not span to the end — ported through the same identity toolkit; it too checks green.

5.6 Quicksort

The crown assembles the pieces. `quicksort(v, fuel, lo, cnt, hbnd)` sorts the `cnt` elements at offset `lo` in place, fuel-structural to mirror the pure model `sortRangeL`. The bracketed `[fuel]` is the declared decreasing position (Section 4): under this architecture it is a formality, since the declared `back`'s conversion is what carries the proof, but the same annotation is what makes the recursion of Section 5.7 admissible at all.

```
fn quicksort [fuel] (v : &mut List Nat, fuel : Nat, lo : Nat, cnt : Nat,
                     hbnd : Le (add lo cnt) (len *v)) -> Unit
      back = sortRangeL fuel lo cnt (*v)
      { match fuel {
          Z => (),
          S(f2) => match cnt {
            Z => (), S(cnt2) => match cnt2 {
              Z => (),
              S(cnt3) => {
                let pivot = nth lo (*v);
                let i = partIdxRangeL lo (S (S cnt3)) (*v);
                let g = partGapRangeL lo (S (S cnt3)) (*v);
                // hle, bl, br : the range bounds, from partScanSizeL and the
                // three length-preservation lemmas (see below)
                partScanRange(&mut *v, lo, S(cnt3), Z, Z, pivot, hle);
                quicksort(&mut *v, f2, lo, i, bl);
                quicksort(&mut *v, f2, S(add lo i), g, br)
              } } } } }
```

Three features of the calculus converge in the last three lines. The pivot's relative index `i` and right-count `g` are read from the *entry* list, before any mutation, as the pure `partIdxRangeL`/`partGapRangeL` — so the body's suspension tree (the partition's back, then the two recursive quicksort backs composed) is *definitionally* `sortRangeL`'s own unfold, and conformance is once again conversion, not a bespoke argument. The two recursive calls are *sequential reborrows of the one borrow* `&mut *v`: this only checks because a `&mut` on a place already holding a parked loan *demand-ends* the previous call's group first — releasing `*v` with its back applied — the concrete demand-driven ending rather than an atomic-at-audit over-approximation that would suspend `*v` for the whole body and block the second reborrow. A minimal witness (`twoRec`, two sequential self-reborrows with `back = *v`) isolates exactly this. Finally, the pivot index is returned, where a wrapper needs it, by a *singleton-Σ device*: `partitionRange` returns `Σ (q : Nat). Id q (partIdxRangeL lo cnt *v)`, pinning the recovered index to its pure value. The range bounds `bl`/`br` are the one place with real proof weight — a forest built from `partScanSizeL` (`i + g + 1 = cnt`) and the three length-preservation lemmas (partition and each recursive sort keep `len *v`, so bounds stated over the live `*v` transport back to the entry length `hbnd`) — but it is ordinary dependent bookkeeping, and it elaborates.

The headline holds: `checkFnOk quicksort` is green — the imperative in-place quicksort type-checks as an implementation of its pure model `sortRangeL`, with conformance reduced to conversion (Section 10.5). The from-scratch conformance `native_decide` runs in seconds rather than the pre-fix tens of minutes, after the delayed-lift `substPure` optimization, and because converting the declaration to the surface syntax preserved its underlying value byte for byte, the check replays from `native_decide`'s cache across that surface change; the performance story is told in full in Section 8.

5.7 The same problem with no model at all

Architecture B removes the `back`s. A callee's *only* description becomes its return type, a postcondition over the exit snapshot; a caller sees an opaque exit plus whatever evidence the callee returned. Four things had to change, and each is worth naming because each is a place where taking the model away exposed something the model had been hiding.

The data plan changes. Range indices were an artefact of wanting a model function: `sortRangeL fuel lo cnt` is a function of offsets, so the program was too. Stated propositionally, the natural unit is the whole list, and what makes whole-list recursion possible is take-and-refill (Section 2′s idiom) applied to *ownership* rather than to elements. Two mutators establish the vocabulary — the second of them is the one the final quicksort actually glues with:

```
fn split_off [i] (v : &mut List Nat, i : Nat, hi : Le i (len *v))
  -> Σ (ret : List Nat) → Σ (h1 : Id (List Nat) (*v) (take i (old *v)))
       → Id (List Nat) ret (drop i (old *v))

fn append_back [v] (v : &mut List Nat, w : List Nat)
  -> Id (List Nat) (*v) (append (old *v) w)
```

`split_off` leaves the first `i` elements behind the borrow and returns the rest *by value* — it cannot come back as a borrow, since its owner is local — with the returned list Σ-pinned to `drop i (old *v)`. It is the general ownership-splitting primitive, and the reason this development has one at all; the quicksort below reaches the same shape through its partition and needs only `append_back` to reassemble.

Note what these ensures are stated *in*: `take`, `drop` and `append` are *observation* functions, which say what a list *is* independently of any implementation. That is the line this architecture holds, and it is a real line — a declared `back` was a function mirroring the body's own algorithm, and none of these is. The line is drawn at the development's own vocabulary rather than at "purity": the same file defines `swapL`, which *is* a model of a swap, and uses it in the ensures of a swap leaf. What the quicksort's call tree contains is what the claim is about, and it contains `count`, `len`, `append`, `Ub`, `Lb` and `Sorted` — nothing that mirrors a partition or a sort.

The leaf becomes provable, which contradicted a prior finding. An earlier reading held that an in-place leaf mutating through pointer writes has an opaque exit, so no value-level postcondition about it is provable and delegation to a model-carrying callee is forced. Half of that is wrong, and the half matters: opacity is a property of borrows *issued by a call*, not of inline mutation. A leaf doing its own cursor work through the body's *own* match-field borrows (doc §3.3) writes into a suspension that the audit itself collapses, so its exit is a constructor tree over known snapshots. `set_at`'s base case is literally `{ *hd := x; Refl }` — a strong update through a field reborrow, where the exit `Cons x` σ_{tl} *is* `set Z x (Cons` σ_{hd}`)` σ_{tl} definitionally. Three features had been filed forward on the strength of the wrong half; two of them evaporated (Section 8 logs the retraction).

The partition returns two lists, and needs branch equations to do it.

```
fn partition [v] (v : &mut List Nat, p : Nat)
  -> Σ (hi : List Nat) → Σ (hub : Ub p (*v)) → Σ (hlb : Lb p hi)
       → Σ (hl1 : Le (len *v) (len (old *v))) → Σ (hl2 : Le (len hi) (len (old *v)))
       → Π (n : Nat) → Id Nat (add (count n (*v)) (count n hi)) (count n (old *v))
```

`*v` keeps the elements $\leq p$; the rest returns by value. There is no `partitionL` in the mechanization — no pure model of this function exists — and every one of the six conjuncts is proved *inductively in the body* from the recursive call's own ensures. The bounds are where Section 3′s branch equations become load-bearing rather than convenient: the body branches on `leb x p` and must *produce* `Le x p` on one side and `Le p x` on the other, and the same body with `Refl` in place of the equation binder is a rejected test sitting one screen away in the same file. The two length conjuncts exist only to feed the recursion below its sufficiency argument.

Recursion carries its own decrease. With the model gone, the return type *is* the postcondition, and a self-call admitted at it unconditionally proves anything (Section 8). The guard is the declared `[fuel]`:

```
fn quicksort [fuel] (fuel : Nat, v : &mut List Nat, hfuel : Le (len *v) fuel)
  -> Σ (hs : Sorted (*v)) → Π (n : Nat) → Id Nat (count n (*v)) (count n (old *v))
```

The head is the pivot; the tail is partitioned through `v`; the kept part is sorted in place through `v` and the returned part through a borrow of the local holding it; `append_back` glues. `Sorted` here is the *structural* Σ-chain down the spine, not the positional `SortedR` above — which is why the Σ-eliminator had to land before any of this could be written, a caller's whole knowledge of a callee's exit being a certificate it must be able to take apart. The sufficiency hypothesis `hfuel` is what makes the out-of-fuel path dead: at `fuel = Z` with a non-empty list it reduces to ⊥, so the branch is an ex-falso and the guarded recursion is *total*. Weakening `hfuel` to `Unit` while keeping the parameter — so that the rejection is about typing and not an unbound name — is a negative control, and the body is rejected.

One step is more than gluing, and it is the same keystone the positional development met. `sorted_append_pivot` needs `Ub p` of the left part *after* it has been sorted, while the partition bounded it *before*; `Ub`/`Lb` are Σ-chains over the spine and so are not natively permutation-invariant. The route is to cross to the multiset, where the property is `Π x. x > p → count x l = Z` and permutation-invariance is a one-line transitivity, with the two crossings as ordinary inductions. That the same obstacle appears in both encodings, and yields to the same move, is the case study's strongest evidence that it is a property of the problem rather than of either formulation.

The headline holds: `checkFnOk quicksort` is green with zero declared backs in its call tree, against lying twins for each conjunct — each chosen to survive the `Nil` path and fail on the recursive one — and against an executing differential that runs the same declaration on concrete lists and compares with a reference sort.

What the whole-list shape costs, stated plainly. It is a workaround for a structural fact, not a discovery: *a segment of a list is not a place*. A prefix of a right-nested `Cons` tree is not a subterm of it and neither is a middle; only a suffix is. So a recursion cannot hand a sub-segment to its callee, and the two architectures are two ways of paying for that. The positional one hands over the whole list plus two indices, and pays in a predicate library where every predicate is range-scoped and every lemma carries a range-fits bound. The whole-list one buys segment-shaped recursion by *moving ownership* — relinking cells and gluing with a traversal where the positional program swaps elements in place — and pays in the program. Neither is the destination; the calculus is missing the thing that would make a range a place, which is Section 8′s point about posing the problem, met from the direction that hurts.

5.8 Arrays: when a sub-range is a place

Everything above pays, twice over, for one structural fact: a segment of a list is not a place. A prefix of a right-nested `Cons` tree is not a subterm of it and neither is a middle; only a suffix is. So a recursion cannot hand a sub-segment to its callee — it hands the whole structure plus two numbers, and the callee's contract must then describe the part it must not touch. That is the frame problem, and the positional development pays it in a predicate library indexed by ranges, in subtraction-guarded reindexing arithmetic, and in length lemmas whose entire job is to transport a bound across a mutation that obviously preserves length.

The third development removes it, with one new former and one new rule.

The former. `Array n T` — a length-indexed run of element slots, with an index step `a[i]` and a range step `a[lo ; cnt]` added to the place grammar. A range being a place is the whole point: a range borrow is then an ordinary `&mut`, and needs no new kind of reference.

The rule. A **carve** is a lazy reorganization that splits an array node into segments, and it is the calculus's first rule that *consumes evidence* (Section 10.7). It fires only when handed a comptime proof of `Le (add lo cnt) n` — whose content is exactly the existence of the residue the split introduces. After it fires, the two sub-borrows are disjoint not because a checker believed an aliasing argument but because they are *literally different subterms of one value tree*, and the suspension machinery of §3.3 already handles those. There is no disjointness judgment anywhere in the implementation. The slogan is worth stating as the

design's thesis in miniature: **proofs license reorganizations; reorganizations restore structural disjointness; the borrow machinery is unchanged.**

The comparison that makes this concrete is Rust's. `get_disjoint_mut` ships exactly this capability — take several disjoint mutable subslices at once — and checks the disjointness *at run time*, returning a `Result` that can fail. DLLBC is not inventing the capability; it is moving that check from run time to compile time and deleting the failure mode. The reason Rust must check at run time is that it has no comptime fragment to pay a proof from, and having one is this calculus's premise.

What transferred, measured. The design predicted that the quicksort library would port by replacing `listRec` with `arrRec` and `Cons` with `acons`, with none of the mathematics re-derived. That is verified verbatim across the whole library — the predicates, the pivot glue, the count layer, and the permutation keystone that cost the positional development its hardest result — some two dozen definitions and proofs, each its list counterpart with the container swapped and nothing else changed. What the prediction omitted is that the array constants must *compute* the way the list constructors do, or every step of a transferred proof wants a transport lemma its list original needs none of; three ι-rules are what make the transfer mechanical rather than merely possible. One of those is a small lesson in notation: the design note's own spelling for a singleton, `arrCat (asingle p) r`, was stuck for symbolic `r` — **the chosen notation could not reach the cons view it is notation for**.

What did not transfer, and this is the sharper finding. The partition is a new program. The list partition is a relational take-and-rebuild returning two lists by value; the array version needs an in-place scan returning a pivot *index*, because the recursive calls carve at that index. There is no program to port. And the algorithm is not a free choice either: two independent *symbolic* indices into one array turn out to be unreachable, because after the first carve a second request lands in a leaf whose offset the machine minted while solving the containment equation — a value with no surface name — so `swap(a[i], a[j])` at two runtime cursors is not writable at all. Lomuto's inner loop is therefore not expressible, and what is expressible is a scan in which every element access sits at index zero of a segment the program itself carved. The design note offered "the right sub-slice is a segment with its own zero; there is no reindexing" as a convenience. It is a hard constraint, and it picks the algorithm.

The result.

```
fn quicksortA [fuel] (fuel : Nat, n : Nat, hfuel : Le n fuel, a : &mut (Array n Nat))
  -> Σ (hs : SortedA n (*a)) → Π x. Id Nat (countA x n (*a)) (countA x n (old *a))
```

§5.7′s signature, re-posed on arrays: sorted and a permutation, over the exit snapshot, in place, with zero declared backs anywhere in the call tree. It checks, and — the part that took the most work — it *runs*.

GREEN *Conformance* (architecture A) — that the imperative partition and quicksort implement their pure models `partScanL`/`sortRangeL`, checked by the declared-back audit — is green. OPEN *Model correctness* for that route — that `sortRangeL` in fact sorts and permutes — remains open, and is the cost of routing verification through a model. GREEN *The direct route* (architecture B, Section 5.7) is green and does not incur that debt: `Sorted (*v)` together with count-preservation against `old *v` is proved in the bodies, with no model of the partition or the sort anywhere in the development. Section 7 compares the two; Section 6 states what a soundness proof of the machinery *itself* would still have to establish, which neither architecture supplies.

6 Metatheory by Empiricism

The case study is checked, but the checker is not proved. No soundness theorem for DLLBC's borrow machinery exists today, and this section does not pretend otherwise. What it reports instead is the discipline the project keeps in the theorem's absence: a set of *validated empirical instruments* and *machine-verified invariants* that, between them, state with some precision what a proof would have to establish. The differential harnesses are counterexample finders that have been shown to find their counterexample; the refinement invariant is instrumented across the whole corpus and asserted at its

source; the strategy is a determinization of a nondeterministic rule set whose one known incompleteness is ledgered. The section closes with the conjecture list a future soundness proof must discharge, and with the known holes any quantified statement of it must first close.

6.1 The differential harness

The core instrument is a differential between the symbolic checker and concrete execution, in two generations. The first (`S8Diff`) tests the callee side. Its property: *if `checkFn` accepts a declaration, then every small concrete run of its body completes — is not stuck — and passes the concrete audit.* This is the simulation theorem — the symbolic checker over-approximates concrete execution — in the form of a bounded, exhaustively-checked `native_decide` proposition over a generated enumeration of bodies. Across three telescopes it generates 136 bodies, of which `checkFn` accepts 75, exercised by 238 concrete runs; all complete and audit, with no counterexample. Any accepted body with a stuck or audit-failing instantiation would fail the `native_decide` and stand as a soundness counterexample to be minimized and reported.

The second generation (`S9Diff`) upgrades to whole-program simulation, so that it catches wrong-*value* refinements and not merely stuckness. Its property: for every `checkFn`-accepted caller, the caller's *concrete* final environment (run in executing mode, where calls run the callee's actual body) is a σ-*instance* of some accepted *symbolic* path's final environment (run in checking mode, where calls use the signature rule). The instance relation is first-order matching — a symbolic σ matches any concrete value *consistently* (the same σ maps to one value throughout), constructors match structurally, loans and borrows up to canonical renumbering — which is the simulation relation proper.

The essential part is not that these pass but that they have been *validated*: a counterexample finder that has never found its counterexample is worthless as evidence. The harness carries its own positive control. With the removed, unsound `constrained`-wire inference forced back on, the `advance`-caller differential goes *red*; with it off, *green*. The `advance` cursor shares the signature of the identity-shaped `through` but writes only the tail, so the constrained refinement — releasing the owner as the surrendered tail — is a provably-wrong fact, exactly the bug class the relation is meant to catch. The finder demonstrably catches its target when the target is reintroduced, and only then.

6.2 The polarity finding: when the machine is the broken one

The differential has always been justified defensively — as insurance that the checker does not accept something execution cannot do. That justification is incomplete, and the array development is what showed it.

Every previously-known divergence between the two sides has had the checker as the *approximating* one. Section 4 records the canonical instance: a loan group releases its captured loans atomically at a fresh existential, where concrete execution ends each lazily, per owner, so a symbolic environment can hold a released value where the concrete one still holds a suspended loan. The checker is coarser; execution is the ground truth the checker is a safe abstraction of. A metatheory written from that assumption would say so, and every member of the family had confirmed it.

The array quicksort broke that pattern. It type-checked and *could not be run*: in executing mode a carving callee left the caller's array segmented, and the caller's next carve then read the wrong leaf. Checking mode never saw the fault, because a call re-mints the callee's payload as a fresh symbolic value at the declared type (Section 4′s opacity), so the checker always sees an *uncarved* array. The temptation is to read that as one more over-approximation. It is the opposite:

> The re-mint is not an approximation of what execution does. It is a **repair** of it. The checker was right and the machine was wrong — and no amount of checking could have revealed that.

This matters beyond the bug. A design document can reason about whether the checker over-approximates the machine; it cannot discover that the machine is the one that is broken, because the machine is what it would be reasoning *from*. Only running both and comparing can. That makes the differential

harness part of the central claim rather than an assurance activity attached to it — and it means the reconciliation a future soundness proof owes is not "the checker over-approximates" but a genuine two-sided relation, since at least one member of the family runs the other way.

The fault's *root* carries the transferable lesson. It was a normalization that discards a zero-extent segment and unwraps a lone survivor — reasonable, and wrong, in four independent places, because the rest of the machine depends on structure it was throwing away. Every one of the four is **unreachable symbolically**, since a residue is never *known* to be zero, and **routine concretely**, since a runtime-computed split has an empty side constantly. That asymmetry is exactly why the symbolic suite stayed green throughout. Generalized: *a normalization justified on symbolic values, applied to concrete ones* — and the reason it took four rounds to finish is that each site looked like the last one.

6.3 The cross-differential: two implementations, one specification

The harnesses above compare the checker with the machine. The array development makes a third comparison available, and it is of a different kind: two *implementations*, checked against each other.

The `List` and `Array` quicksorts share a specification — sortedness of the exit snapshot with count-preservation against the entry — and share nothing else. Not the program (take-and-rebuild versus an in-place carved scan), not the partition, not the predicates, not the container, not one line of proof. Run on eleven shared inputs, they agree with each other *and* with a reference sort. The conjunction is deliberate: a two-way agreement on a wrong answer still goes red, so the test does not reward the two implementations for sharing a mistake — and a failure to run on either side cannot masquerade as agreement.

What that buys is evidence about the *specification* rather than about either implementation. A single verified program tells you it satisfies the postcondition you wrote; it tells you nothing about whether the postcondition says what you meant. Two programs sharing only the postcondition, disagreeing, would indict the postcondition or one of the proofs. Agreeing, they are weak evidence that the shared statement is the one intended — weak because eleven inputs is eleven inputs, but it is a kind of evidence the project could not previously produce at all.

> **A fence, because this paper now reports three measurement sets and they are not comparable.** (i) The **conformance** audit's 465× speedup (84,121 ms to 181 ms) is a *checker* change — a substitution fix — measured on architecture A's program. (ii) The **positional-versus-whole-list** gap (21.8 s to 21 ms) is two direct-proving `List` programs differing in both specification encoding and partition program, with no checker work between them; Section 8 states why the honest attribution is to the pair rather than to the encoding alone. (iii) The **cross-differential** is not a performance measurement at all — it is an agreement check between two implementations, and its number (eleven inputs) is a count of test cases, not of time. Combining any two of the three into one comparison would be wrong.

6.4 Discovered preconditions

The harness does more than pass; it has told the design what the theorem needs. The clearest case is *exhaustiveness.* A symbolic `match` missing a constructor was, at one point, accepted by the checker (inductive-declaration exhaustiveness having been deferred) yet concretely *stuck* on the missing branch — precisely an accepted-but-unsafe program, the differential's target. Exhaustiveness is now checked at the symbolic match fork (Section 10.4): a branch set must cover the scrutinee type's constructors. With that, "accepted $\Longrightarrow$ concrete-safe" becomes unconditional over the generator's grammar, and the suite's non-exhaustive bodies are all rejected. The lesson is general: a precondition of the simulation theorem was made *syntactic*, moved from an informal caveat into a rule the checker enforces.

6.5 The knowledge/state invariant

The refinement arrow $\rightsquigarrow$ carries an invariant with two halves, both now machine-checked. It propagates *knowledge* — constructor shapes and equation solutions, facts true of a value timelessly — and never *state* — a hole, a loan marker, or a mutation's result, which are facts about a slot at a moment. The *reach* half: a single $\rightsquigarrow$ substitutes into every σ-bearing component of the whole checker state — the environment, the σ-context, the boundary obligations, the loan groups, the pinned return type, and the reflected backward specs (Section 10.5) — because any component that snapshots a σ-typed thing at seed time and consults it later goes stale otherwise. The *carry* half: no substitution for a σ may contain

a hole, a marker, or a mutation result — recording a take, a fill, or a swap's arrangement by rewriting a σ would make a snapshot track the present, so that a pinned `partIdxL n *v` would come to mean the index of the *already-partitioned* list, the one staleness the whole snapshot discipline exists to forbid.

Stated first as a diagnosis, the invariant became prophylaxis: instrumenting the entire mechanization found *zero* violations — no substitution ever carried a hole, marker, or mutation result — so it is verified rather than aspirational, and it is asserted at the substitution site to keep it verified as the corpus grows. (The bug that prompted the audit was elsewhere and mundane: a comptime read of a consumed variable returned $\perp$ silently, where the runtime read rejects use-after-move.)

6.6 Nondeterministic rules, and a lazy strategy

The rule figures are presented *nondeterministically*: a reorganization may fire wherever its premises hold. The implementation is one deterministic scheduling of them — a lazy, fuel-bounded strategy that fires a reorganization only when some rule's premise demands it. The determinization is explicit: a continuation-pushing pre-pass makes every `match` terminal, `explore` enumerates the derivations, and the lazy collapse is one scheduling of the reorganization-closure premise. The relationship between the two is *not* proved complete, and one gap is ledgered: there is a rules-admissible ending order — ending a live reborrow before the group it belongs to ends — that the implemented order rejects when an issued payload is suspended. Soundness is unaffected, because this is a *rejection* and not an unsound acceptance; but it is a strategy-completeness question that a full metatheory owes an answer to, and it is recorded as such rather than glossed.

6.7 What a proof must establish

The instruments and invariants above converge on a short list of obligations a soundness proof of the machinery — as distinct from correctness of the pure models, which is ordinary type theory — would have to discharge.

First, the *simulation relation* of the differential must be proved, not merely tested: that every `checkFn`-accepted program's concrete runs are instances of its accepted symbolic paths. Exhaustiveness is a precondition (now syntactic), and one modelling choice is forced by the group-end over-approximation the differential surfaced. The symbolic group-end releases each captured loan atomically with a fresh existential, whereas concrete ending is lazy and per-owner, so a symbolic environment can hold a released existential where the concrete one still holds a suspended loan. A proof must reconcile these — either by a fully-collapsed-at-observation hypothesis, or by letting a value still out on loan stand as an instance of the existential — and the differential cannot decide between them; the choice is the proof's.

The obligation is **two-sided**, and stating it one-sidedly would now be a known error. It is tempting to phrase the whole relation as "the checker over-approximates the machine", since every member of the divergence family did that until one did not (Section 6.2). A relation built on that assumption cannot express the case where the re-mint *repairs* execution rather than abstracting it, and would have to treat a genuine machine defect as a soundness counterexample. Whatever form the relation takes, it has to admit that the two sides can disagree in either direction and that which one is wrong is not determined by which is coarser.

A second obligation the array era added is easy to miss because it is an obligation on the *harness* rather than on the calculus. Arrays are the first values whose state form is coarser than their value — a carved-and-rejoined array holds a segment list where an untouched one holds a run — so the simulation relation's array case is *defined* by splitting a concrete run along the symbolic extents. That is sound exactly when the merge normalization preserves values. If it does not, the harness does not go red; it goes silently *green* on a mismatch. A nice-to-have on the obligation list has thereby become a premise of the counterexample finder's own correctness, which is the thing to know before trusting the differential over array bodies.

Second, *audit soundness* — that a passed `↳/back` audit implies the caller's recovered value genuinely satisfies the obligation — but only *modulo* the audit-strategy holes the extraction has already named, which any quantified statement must close. Section 4 names three; two of them bear on audit soundness in this sense, and it is worth separating them, because the third is a rejection rather than an unsound

acceptance and so threatens completeness instead. The two: a declared `back` on a call that captures two or more borrows is silently ignored, the group end matching a backward spec only against a single captured loan and otherwise degrading to opaque *without rejection*, so a declared promise can go silently unused; and both back checks take the first qualifying obligation and pass *vacuously* when none qualifies, so a declared `back` can go entirely unaudited on such a path. (The third, the read/write asymmetry on group-captured owners, rejects a program the rules admit — a strategy gap of the same kind as the ending-order one above.) These are holes in the audit's strategy, not in the pure models, and until they are closed "audited" means less than it appears — a soundness statement that quantifies over declared specifications must fix them first.

Third, and new at this pin, *the recursion guard*. A self-call is admitted at the function's own declared return type — signature-only checking forces that — so with no side condition the rule is Hoare's recursion rule with its side condition deleted, and any false postcondition proves itself. That hole was open and demonstrable (Section 8); it is now closed by a structural guard, and what a proof owes here is the guard's well-foundedness: that the declared position's snapshot strictly decreases along every admitted self-call, that snapshots are entry-knowledge no mutation rewrites (which is the *carry* half of the invariant above, doing load-bearing work outside the type layer for the first time), and that rejecting mutual recursion outright is what keeps the per-declaration guard from being circumvented through a second door.

The honest summary is the one this section opened with: the checker is not proved, but its behavior is instrumented, its central invariant is verified and enforced at its source, its counterexample finders are validated against the bug class they target, and the remaining gaps are enumerated rather than hidden. That inventory is what a proof would start from, and stating it precisely is the work this section claims to have done.

7 Two architectures for verification

The declared-back machinery of Section 4 admits two very different ways of verifying an imperative program, and this project has built both, to completion. They differ in what the signature promises and in where the proof burden lands. The first routes all reasoning through a pure model and keeps the imperative body proof-free. The second proves properties *directly* in the body, over the value the borrow holds at exit. Both are landed and green at the pin, on the same *problem*, which is what makes the comparison below a measurement rather than a prediction: Section 5 works each of them as a full in-place quicksort. The two are not the same *program* — how the problem is posed differs with the architecture, and Section 5.7 says how — which bounds what the comparison can attribute. The project chose the second, deliberately, as its mission; the first remains as the baseline the choice was made against.

7.1 Architecture A: conformance to a pure model

A declared back can be the whole story. Give an in-place mutator a `back` that *is* a pure function computing the same result, and the audit (Section 4, B-Back0/B-BackN) checks the imperative body against that pure model by a single conversion — the body's suspension tree is definitionally the model's unfold. The flagship instance is in the mechanization and green: the in-place, swap-based quicksort type-checks as an implementation of its pure model, with `back = sortRangeL fuel lo cnt` $* v$:

```
fn quicksort [fuel] (v : &mut List Nat, fuel : Nat, lo : Nat, cnt : Nat, hbnd : …) -> Unit
  back = sortRangeL fuel lo cnt (*v) = { … }
```

Three properties define this architecture, and all three are checked. The body carries *zero* proof obligations: partition and the two recursive calls compose their declared backs, and that composition converts against `sortRangeL`'s own unfold — conformance *is* conversion, nothing more. The caller recovers an *exact value*, not merely a property: ending the loan releases `sortRangeL` applied to the entry snapshot, so a caller that sorts a concrete list gets that concrete sorted list back. And correctness — that `sortRangeL` sorts, that it permutes — becomes a set of lemmas about the *pure model*, proved in the comptime fragment by ordinary type theory, entirely apart from the borrows. Verification splits cleanly into *conformance* (mechanical, the back audit) and *model correctness* (ordinary dependent-type reasoning). The full walkthrough is the quicksort case study; here we only record what the architecture *is*.

What it is, honestly stated, is Aeneas [5] rebuilt inside one language. Aeneas verifies Rust by extracting a pure functional model and reasoning about that; this architecture does the same, except the extraction is a declared `back` and the conformance is a conversion rather than a translation. The consequence is the point: the imperative body never carries a proof, so the interaction this calculus exists to study — dependent types *crossing* mutation, a proof about a value that is being mutated through a borrow — is never exercised where it is hard. All the hard reasoning has been exported to a pure model that could have been written in any prover. Architecture A is real, it is green, and it is deliberately the *comparison baseline* rather than the mission.

7.2 Architecture B: direct propositional proving

The mission, set by the project in favour of exactly the interaction A avoids, is to prove postconditions *directly in the body*, over the value the borrow holds at exit. In the mechanization at the pin the shape is:

```
fn quicksort [fuel] (fuel : Nat, v : &mut List Nat, hfuel : Le (len *v) fuel)
  -> Σ (hs : Sorted (*v)) → Π n. Id Nat (count n (*v)) (count n (old *v))
```

The parameter stays a plain `List Nat`; the evidence rides the return type, as a Σ of sortedness over the exit snapshot and count-preservation against the entry. Permutation is stated by counting rather than by an indexed `Perm` family — a multiset equality is what the swap and partition lemmas actually establish, and it is closed under the rearrangements a sort performs. Two snapshot readings make this well-formed, and both are implemented. $*v$ *in return position* denotes the **exit snapshot** — the value the borrow holds at the audit. It is canonical for the same reason the calculus has no lifetime annotations: with no lifetimes, the callee's exclusive access ends *exactly* at the audit, and the audit already computes the collapsed final payload, so "the value at the end" names one well-defined tree. `old` $*v$ denotes the **entry snapshot** — `old` is an operator, not a binder, sugar over the telescope's existing entry snapshot, so nothing scoped escapes its bracket. A postcondition can therefore relate exit to entry — the count conjunct above says that every value occurs as often in the exit list as in the entry list — without either reading becoming stale.

The proofs are threaded through the body from callees' postconditions. A call to partition returns evidence about the partitioned list; the caller opens that evidence and uses it to discharge its own obligations, exactly as one composes lemmas — except the "lemmas" are the postconditions of imperative calls, and the subject they speak about is a value being mutated in place. Caller-side, a single symbolic value is shared between the loan's eventual release and the returned evidence's subject, so the proof *attaches* to the recovered value rather than floating beside it.

This is the architecture that walks into the hard interaction on purpose, and the friction it meets there is the research object, not an obstacle to be smoothed over before publication. The project keeps a *pain diary*: each contortion the programmer is forced into is logged as a candidate calculus feature — a missing sugar, an absent rule, an implicit that should have been inferred. Three entries have been promoted from diary to design, and the order is instructive. The exit-snapshot reading and the `old` operator came first, and are what make the return type above well-formed. *Branch equations* (Section 3) came third and were the one the architecture could not proceed without: a body that must produce evidence has to turn "the comparison said yes" into an order fact, and under architecture A no body ever needed to.

One entry remains open, and it is the residual cost of the architecture rather than a gap in it. OPEN A body can name the value a place holds *now*, and — for a borrow parameter — the value it held at entry; it cannot name the value a *consumed* binder held, nor an intermediate exit that a later mutation has superseded. So a proof spanning two mutations must be built *before* the second and applied after, as a function of values that do not exist yet. The mechanization's quicksort carries four such staged builders; none of them does mathematical work, and all four would collapse into ordinary lemma applications given an `old` that extended to consumed things. Six independent instances of the pattern are logged across the last two milestones — enough that the diary now reads as a specification for the feature rather than a list of complaints, which is what the diary discipline is for.

7.3 The comparison

	A — pure-model conformance	**B — direct propositional**
Return type	borrow stays a borrow; a declared `back` names the pure model	borrow stays a borrow; postcondition over the exit snapshot $* v$, `old` for entry
Body proof burden	none — conformance is one conversion of the composed backs	proofs threaded through the body from callees' postconditions (Σ/ evidence plumbing)
Caller recovers	the *exact value* — the model applied to the entry snapshot	the *property* (`Sorted`, `Perm`), attached to the recovered value
Correctness proof	lemmas about the pure model, proved separately in the comptime fragment	proved in place, directly, as the body is checked
Reformulation problem	present — a back that reformulates its tree is outside conversion's reach (Section 4); bridging-equation close deferred	absent — there is no back to reformulate; the postcondition is proved directly
Body's branch knowledge	occurrence rewriting suffices — no body cites a branch fact	needs *branch equations*: the order fact is produced, not merely assumed (Section 3)
Recursion	structural on the model's own `fuel`; conformance is per-path	the declared `[k]` guard, with a sufficiency hypothesis discharging the out-of-fuel path
Cost to check	GREEN 181 ms for the conformance audit, after the substitution fix (Section 8)	GREEN 21 ms — a different program, and the encoding is why (Section 8)
Status	GREEN landed — quicksort conforms to `sortRangeL`	GREEN landed — quicksort proves `Sorted` $\wedge$ count-preservation, zero declared backs

The trade is legible in the table. A costs nothing in the body and yields the exact value, but pays in a pure model that must be written and kept in step, and inherits the reformulation gap of Section 4 — the moment a declared back is stated in any form other than the raw suspension tree, conversion may not see the equality, and the audit must fall back to differential validation. B pays in the body — every property must be plumbed through as evidence — but yields the property one actually wants, over the value one actually has, with no separate model to maintain and so no reformulation problem to inherit. B's discipline is the `ensures`-clause of a Floyd–Hoare verifier such as Dafny [7], recovered here without a separate specification language: the postcondition is an ordinary dependent type over the exit snapshot, checked by the same interpreter that runs the body. That the calculus can express either architecture from the *same* boundary machinery — a declared `back` for A, a postcondition over $* v$ for B — is itself the finding.

B has since been carried out twice, over different containers, and the second instance sharpens what the architecture costs. The array quicksort (Section 5.8) is this same architecture — postconditions over the exit snapshot, zero declared backs — on a representation where a sub-range is a place. Two things follow that the `List` instance could not have shown. The first is the cross-differential (Section 6.3): two implementations sharing a specification and nothing else, agreeing, which is evidence

about the *specification* that no single verified program can supply. The second is a correction to B's cost model. The array design predicted that making sub-ranges into places would delete positional reasoning outright; it deletes it *within a body* and hands it back at every *function boundary*, because a signature must describe a split that only the caller's carve can name. So B's ledger reads: one stratum deleted, one inherited, and one invented at the leaf's interface — smaller than the one deleted, but not nothing, and not predicted.

Two further results from carrying B to completion sharpen the trade beyond what the table shows, and neither was predicted. The first is that B does *not* merely relocate A's pure model: the mechanization's back-less quicksort has no model function of the partition anywhere, only *observation* vocabulary — `count`, `len`, `take`, `drop`, `append`, `Ub`, `Lb`, `Sorted` — functions that say what a list *is* rather than what a body *does*. An earlier reading of the evidence held that direct proving eliminated conversion-based verification while the model functions survived as the only provable exit shapes; carrying the architecture to the end refuted it, and the refutation is recorded in Section 8 because the diagnosis it replaced was a reasonable inference from a partial result. The second is that B is *cheaper to check*, by a wide margin, for a reason that has nothing to do with either architecture: the spec encoding, not the checker, dominates the cost (Section 8). Both findings favour B, and neither is an argument the project made when it chose B — which is the most one can ask of a comparison one has prejudged.

8 Lessons

Five methodological lessons carried more weight than any single rule in building DLLBC. Each was earned against a wrong first guess, and each left a mark on how the calculus is now developed.

8.1 Measure before you architect

The final assembly's conformance check — the quicksort audit that closes the case study — was, at first, unusable. A from-scratch build of the test suite, under the `native_decide` kernel reduction that is the mechanization's acceptance surface, took **38m49s** wall (2326s CPU), essentially all of it in that one check.

The instinct was that the check re-examined the same enormous proof terms many times, so it should be **cached**. Four content-addressed accelerations were implemented and measured, and every one was a wash or worse. A coarse per-call convert cache bought **no win** — the redundancy lived inside a single large normalization, below the granularity a top-level memo can see. A per-node structural memo of certified-rigid terms was a **wash** (33m41s CPU). Keying that memo on pointer identity instead — hashing structural nodes is itself $O(\text{subtree})$ per probe, the very quadratic a cache is meant to remove — **made it 40% worse** (48m17s CPU): pointer identity misses the fresh structural duplicates that substitution mints per occurrence, exactly the hits the structural set had been converting into wins. Interning nodes at their construction sites **recovered nothing** (49m11s CPU) and, worse, destabilized the suite into a `SIGSEGV` across eight modules. The residual truth these four negatives established is a general asymptotic one: on **unshared** trees, content-addressed acceleration re-introduces the quadratic it targets, and no addressing scheme repairs a cost that is **construction**, not lookup.

What did work was refusing to architect further and profiling instead. `perf(1)`, run against a compiled bench harness that isolated the check from the suite's elaboration (a two-minute iteration in place of forty), attributed **93% of all cycles to one line**: the eager de Bruijn re-lifting inside substitution. The textbook recursion re-shifts the substituend at **every** binder it crosses — $O(\text{binders} \times |\text{substituend}|)$ — and at quicksort scale the substituends are 10^5-node **closed** proof values, so each shift is a structurally-identity rebuild (Val.shiftPure alone was 62% of cycles, with 28% more in the allocator and reference-count churn on the copies). The repair was a 52-line diff: carry the number of binders crossed and lift only at an actual occurrence, skipping even that when the substituend is closed. The isolated check fell from 84,121ms to 181ms — **465×** — and the full suite from **38m49s to roughly 13 seconds**. The moral is not that caching is bad; it is that the four negative results were **load-bearing**. They are what proved the cost was construction rather than lookup, and so what pointed the profiler at the one line that mattered. Architecture is the reward for a measurement, not a substitute for it.

The array era supplied the same lesson in a diagnostic register, and the shape is worth recording because it transfers. A defect that made a verified program unrunnable was attacked three times with three different mechanisms, and all three failed; each failure was read as evidence that the problem was *architectural* — that the mechanism was wrong in kind rather than incomplete. The investigation twice wrote down a conclusion a later probe refuted. What broke the loop was instrumenting the failure to *name the dying borrow* rather than reasoning further about the design, after which the fault turned out to be one normalization firing at four independent sites, each of them small. The rule extracted: **count sites rather than estimate effort — each is small, and there is reliably one more than you think.** "Architectural" is what a multi-site bug looks like from the inside when you are estimating instead of counting.

8.2 How the problem is posed is a performance decision

The lesson above is about the checker. Its sequel is about the *statement being checked*, and it was measured after the profiler had already done its work.

The mechanization contains two direct-proving quicksorts. Both sort in place through a `&mut List Nat` and both prove the same shape of postcondition — sortedness of the exit snapshot, paired with count-preservation against the entry. The earlier is *positional*: sortedness is a bounded Π-predicate `SortedR cnt lo` over range offsets, the pivot index is a pure `partIdxL`, every lemma is indexed by an offset into the whole list, and the partition is a swap-based Lomuto scan over a range. The later is *whole-list*: `Sorted` is a Σ-chain down the spine, the partition returns two lists, and the vocabulary is `count`, `append`, `take`, `drop`. The positional check takes **21.8 s**; the whole-list one takes **21 ms** — same machine, same checker, same build configuration, no performance work of any kind between them.

The honest scope of that number needs stating, because it is easy to overclaim. The two differ in *both* the specification's encoding and the partition's program, and the two differences are not independent: the positional specification exists because a swap-based scan mutates a range in place, and the whole-list specification is what makes a two-list partition natural. No experiment here isolates one from the other, so the factor of roughly a thousand is a property of the *pair* — of how the problem is posed, program and statement together — and not a measurement of encoding alone.

What can be said about the mechanism is what the first lesson established: cost in this checker is *construction*, and conversion is full normal-form equality. A predicate indexed by an offset carries that offset's arithmetic into every subterm, so positional normal forms are large; structural ones are small. Whatever the split between program and statement, the difference is in the size of the terms conversion must build, which is precisely the cost no cache and no addressing scheme repairs. The architected next step at the time — δ-constants for the pure library, so that proofs become constant-spines and checks scale with the program rather than the proof — would have been aimed past this.

The design corollary is worth more than the number. The standing rule for this calculus is *naturalness first*: write the program a systems programmer would write, and if the checker rejects it, fix the calculus rather than the program. That rule is usually assumed to trade efficiency for fidelity. Here it did the opposite — the formulation one would use to explain the algorithm aloud is also the one the checker finds cheap. When a verification effort is slow, how the problem has been posed is worth suspecting before the engine that checks it.

8.3 Run the differential at both extremes, and default to neither

The array era produced a testing rule sharp enough to state, and it came from noticing that every defect which hid until late hid the same way.

`Array` values have two regimes. Extents can be *concrete* — the lengths a first milestone naturally writes tests at, because they let you print the answer — or *symbolic*, which is what every program with a runtime-computed split point is made of. The two exercise disjoint machinery. A carve at concrete extents rejoins to a plain run and never needs the normalization that folds a segment list back to a value; a symbolic segment cannot merge and always needs it. Conversely, a residue is never *known* to be zero

symbolically, so the entire zero-extent path — four separate defects — is unreachable in a symbolic suite and constant in a concrete one.

The consequence is that **every late-hiding defect in the era was invisible at one regime and routine at the other**, in one direction or the other. The exit snapshot arriving in the wrong form: invisible concretely. The carve failing to collapse a suspended node: needs a recursive program, so invisible in a suite of straight-line tests. All four zero-extent sites: invisible symbolically. Each suite was green throughout while the other regime was broken, and neither suite was negligent.

So: **arrays need the differential run at both, and neither is the default.** The generalizable form is worth more than the instance. Whenever a representation admits a normalization that is justified on one class of values, the class it is *not* justified on is where the defects will be, and a test suite written in the natural style will sit entirely inside the safe class without anyone choosing that.

8.4 One negative test per rule branch, not per feature

Every genuine soundness bug the project has found was caught the same way: by a **lying twin** — a program written to be rejected — that the checker instead accepted, a negative test refusing to fail. There are three, and the lesson sharpened at each one.

The first was a signature-inferred "constrained wire": at a call whose loan group released a captured owner, the checker inferred the released value from the surrendered field payload. But `through(b) = b` and an `advance` that steps a cursor share a signature and differ only in body, so the inference let the checker refine an owner to its own tail — unsound. The fix removed the inference entirely; a captured release under an opaque call is now always a fresh existential, and `through`'s lost precision is recovered only through a **declared** backward spec.

The second bug exists because the first one's lesson was generalized. A declared `back` was checked at the audit only on the **borrow-returning** branch of the callee rule; a **value-returning** body's declared `back` was never checked at all, and a lying `partScanL` variant passed clean. The lesson from the first bug had been "test the feature"; the sharper statement it forced was **one negative test per rule branch, not per feature** — and applying that statement is what surfaced the unchecked value-returning branch, which was then closed as its own audit (the zero-hole case, converting the declared spec directly against the mutated argument's resolved suspension tree).

The third is the starkest, and it sharpened the statement once more. A call is checked against a signature alone — recursion forces that — so a **self**-call was admitted at the function's own declared return type with no side condition anywhere. That is Hoare's rule for recursion with its side condition deleted, and the lying twin is two lines long:

```
fn bad () -> Id Nat Z (S Z) { bad() }        // was ACCEPTED
```

Any false postcondition proved itself. The fix is the declared `[k]` guard of Section 10.5. What is instructive is **why the hole survived so long in a corpus with per-branch negative tests**: reaching it needs a declaration that is recursive **and** back-less, and the corpus had many of each but never one that was both — every recursive declaration carried a `back` whose conversion did the real work, and every back-less one was a straight-line delegator. The hole sat in the uncovered **square** of a two-feature grid, not on an untested branch of one rule. So the statement generalizes again: **one negative test per reachable combination of features**, which is a larger obligation than per-branch and the reason the architecture comparison of Section 7 was worth building twice over — a second architecture exercises combinations the first never forms.

The class has a fourth member, caught not by a test but by writing this paper. Extracting the boundary rules from the implementation for Section 10.5 surfaced that both back-checks take the **first** qualifying obligation and pass **vacuously** when none qualifies — so on such a path a declared `back` can go entirely unaudited. The rule extraction was, in effect, another kind of negative test, one that reads every branch of a rule by construction; it remains an open theory-line fix.

8.5 The document is the single source of truth

DLLBC's calculus document and its mechanization were developed as one artifact. The document's annotated environment traces **are** the test suite — each is a golden final state, checked up to loan and symbol renaming — and every milestone's report closed with a list of doc-vs-code ambiguities that were folded back into the document the same day. When a coordination conflict arose between a message and the document, the standing tiebreak was that **the document wins**.

Two disciplines make this trustworthy. First, retractions are **logged, never silently edited**. Three claims have been formally withdrawn in the progress record. The "write order forced by the types" argument did not survive the switch to a recursive-family encoding of vectors. A "convergence crux discharged inside the checker" claim was false because the check it relied on did not run at the time — its discovery is precisely the second bug above. And the **delegation discipline** was withdrawn at this pin: the finding that an in-place leaf's exit is opaque, so that every direct-proving body must delegate its mutation to a callee carrying a pure model, was half wrong. Opacity belongs to borrows **issued by a call** — the call rule mints their payloads as fresh existentials, having nothing in a signature to say what they hold — and not to inline mutation at all. A leaf that does its own cursor work through the body's **own** match-field borrows writes into a suspension the audit itself collapses, so its exit is a constructor tree over known snapshots and is fully provable. The retraction mattered: three features had been filed forward on the strength of the wrong half, and two of them evaporated with it. The inference had been reasonable from the evidence then available, which is the usual shape of a retraction worth logging.

The array era added two more, and they differ in kind from the three above in a way worth naming: they did not need *refining*, they *reversed*, and they are struck through in the design note where they stood rather than quietly rewritten. That note recommended carving inline and reaching for a function boundary only when abstraction was wanted; the truth is the opposite, and a body following the advice cannot be written at all. It claimed the migration would delete one stratum of reasoning, inherit one, and invent none; one *is* invented, at the partition's interface, because a signature must describe a split that only the caller's carve can name. Both are the same fact seen from two sides — carving is a within-body mechanism, and boundaries are where its guarantees stop and start again — and neither was visible from the desk. The discipline that makes them useful rather than embarrassing is the strike-through: a reader can see what was believed, what replaced it, and that the replacement came from running the thing. Second, the rule-figure extraction for **this paper** functioned as an audit in its own right. Reading the six figures out of the implementation, doc in hand, produced 20 recorded findings, several load-bearing: the vacuous back-audit paths above; a `seq`-discard that drops an owning value without running `drop`, a real semantics fork rather than the sugar the document claimed; and a comptime `match` that today holds only through the recursor constants it would elaborate to. Writing the paper found gaps the mechanization's own green test suite had not — which is the strongest argument we have for keeping specification and implementation close enough that either can audit the other.

The discipline has since been exercised a second time, and the second run is the better evidence. Re-extracting the figures against a later pin — the paper had to be brought current after a second verification architecture landed — closed two of those twenty and left the rest standing, and re-reading each *remaining* one against the implementation rather than carrying it forward is what made the difference between a ledger and a habit. The numbering is deliberately sealed to the first extraction: findings are not renumbered when they close, because a claim this paper made about its own audit should stay checkable against the audit it made it about.

One artifact of that second pass deserves to be stated plainly, because it is the cleanest evidence available that a paper about a moving mechanization decays silently. **The quicksort listing in Section 5 would have been rejected by the checker it describes.** Between the two pins the recursion guard arrived, every recursive declaration in the corpus acquired the `[k]` annotation that admits its self-calls — and the paper's transcription of that declaration, correct when it was made, quietly became a program the mechanization no longer accepts. Nothing failed. No test covered it, because the listing is prose; the extraction's findings ledger did not catch it, because the ledger tracks *rules* against the implementation and this was a *quotation* going stale. It was found by re-reading the source with the listing in hand. The

lesson is not that transcription is dangerous but that a document is only as current as its last *verification*, and the interval at which that must happen is set by the artifact, not by the writing.

9 Related Work

DLLBC sits at the intersection of two lines that rarely meet: the verification of Rust-style mutable borrowing, and full dependent type theory. We position it against the closest work in each, and against the handful of systems that, like DLLBC, try to hold both at once.

9.1 Aeneas and LLBC

The closest kin is Aeneas [5] and its underlying Low-Level Borrow Calculus [4]. DLLBC borrows LLBC's loan/borrow **value** semantics — mutable references modelled as loans and borrows in the operational state rather than as pointers into a heap — and its symbolic execution wholesale. The founding identification of this calculus, that matching a symbolic scrutinee **is** dependent pattern matching, is the observation that LLBC's symbolic interpreter already does, verbatim, what Agda-style dependent elimination does by unification and substitution; DLLBC's comptime write (↜) is that mechanism named and reused for types.

The inversion is where DLLBC departs. Aeneas **synthesizes** a backward function for each mutable borrow from the loan structure, as one stage of a translation pipeline (Rust → LLBC → a pure functional model → a proof assistant), and the program's correctness is then proved about the extracted model in Lean, Coq, or F*. DLLBC turns the synthesized artifact into a **declared and checked** signature component: a function writes `back = f` in its type, and the checker verifies it by converting the borrow's suspension tree against the declaration at the return audit (Section 10.5). Because the declaration is checked rather than derived, backward specifications **compose** along call chains — a caller resolves a sub-call's group-end through the callee's declared spec — which reconstructs LLBC's backward-function composition as a checking discipline internal to one language. That single-language stance is the second difference: the imperative program, its dependent types, and its pure model are all terms of the same calculus under one conversion relation, with no translation boundary between the code and the logic it is verified against.

9.2 Prophecy-based specification

RustHorn [8], RustHornBelt [9], Creusot [10], and Verus [11] specify mutable borrows through **prophecy** variables: a mutable reference's final value is a logical variable, constrained at the point the borrow ends, so a postcondition may speak about a value the program has not yet produced. DLLBC's snapshot discipline deliberately **refuses** prophecy. Its ⇝ obligation and exit-snapshot postconditions are the prophecy-free alternative — a borrow's contract is discharged against the value actually present when its loan ends, never against a promised future. The calculus doc makes the case for the refusal directly (doc §4.2): per-loan contracts on coupled data would have to state conditions like "a vector whose length matches whatever the **sibling** loan ends with" — a promise about another loan's future, which is exactly what prophecy formalizes. DLLBC dissolves the coupling instead of naming it: the sibling writes are unchecked strong updates, and the **joint** condition is checked once, at the boundary, when both final values are in hand and forming the result is a matter of conversion.

9.3 Refinement types for Rust

Flux [12] equips Rust with refinement types that support **strong updates** — a value's refinement may change as it is mutated in place — which makes it the nearest neighbour in spirit for in-place verification, and the direct descendant of Liquid-type refinement [13]. DLLBC differs in the strength and decidability of its type discipline. Flux's refinements are drawn from an SMT-decidable logic and discharged by an SMT solver; DLLBC carries full CIC-style dependency — indexed families via large elimination, arbitrary type-level computation — and its only decision procedure is conversion. The boundary money test — a forgotten length update leaving a stuck type that no constructor inhabits — is a conversion failure, not a solver query.

9.4 Disjoint mutable subslices

The carve (Section 10.7) lands in a crowded area, and the honest positioning is that the crowd already has every component. Three concerns are separable, and every system here separates them: an **interval-arithmetic disjointness predicate**, present in Low* [14], in VST, and derivable in Iris; an **exclusivity discipline** that makes coexistence a question worth asking, which is Rust's borrow checker, separation logic's separating conjunction, or Verus's [11] tracked ghost tokens; and a **recombination story**, which is Iris's magic wand, Aeneas's [5] backward function, Creusot's [10] prophecies, or RustBelt's [15] inheritance-at-lifetime-death.

What is claimed as new is the **conjunction**, not any component: ownership-transferring, exclusivity-enforcing range borrows whose coexistence is licensed by comptime evidence *the checker consumes*, over a heapless value-tree semantics. No system in the list routes the disjointness predicate through the program's own dependent proofs in order to license exclusivity inside the operational semantics. Rust hardwires it, and only for constant indices. Separation logic discharges it in the meta-logic, by a proof written in a proof mode rather than one the checker consumes. Low* has the predicate and the recombination but no exclusivity at all — its buffers may overlap freely, so disjointness is never a permission question, only a framing one.

The concession belongs in the same breath, because a reader will reach for it: **Rust already ships this API surface.** `get_disjoint_mut` takes range indices and returns several disjoint mutable subslices, checking disjointness at run time and returning a result that can fail. DLLBC is therefore not inventing a capability. It is moving an existing capability's check from run time to compile time and deleting the failure mode — a smaller claim than new expressive power, and the honest one. It is not a small claim either, because *why* Rust must check at run time is that it has no comptime fragment to pay a proof from, and having one is this calculus's premise.

9.5 Dependent and linear type theories

ATS [16] combines dependent and linear types with explicit **views**: stateful, separated resources threaded through the program to license mutation. DLLBC's value semantics needs no view threading — the loan and borrow machinery lives in the operational state, and the surface program reads like ordinary code, its mutation-through-aliases tracked by the checker rather than spelled out by the programmer. Quantitative Type Theory [17], as realized in Idris 2 [18], places linearity **in** the types by tracking usage multiplicities, but governs erasure and resource counting rather than borrowing: it has no reborrows and no strong updates through an alias, which are exactly the constructs DLLBC is built to type.

9.6 Deductive verification and the `old`/`ensures` correspondence

Dafny [7] pairs each method with an `ensures` clause in which `old(e)` names the value of `e` on entry — the design ancestor of DLLBC's exit-snapshot postconditions, where a return type's `old *v` reads the entry snapshot and its borrow-payload derefs read the exit. The difference is where the correspondence comes from. Dafny **decrees** the `old`/`ensures` semantics as a specification construct; DLLBC **derives** it from loan structure — the entry-pin and exit-snapshot readings fall out of **where** the $\rightsquigarrow$ obligation is consulted, not from a separate assertion language layered over the code. Low* [14] targets the same domain of verified low-level software, but verifies against an explicit heap model discharged by SMT, where DLLBC verifies against a heap-free loan/borrow value semantics discharged by conversion. The broader deductive landscape for Rust — Prusti [19], the semantic-soundness foundation of RustBelt [15], and the hybrid symbolic approach of Gillian-Rust [20] — shares DLLBC's target but not its ambition to make the specification language and the type theory one and the same.

9.7 Lineage

DLLBC subsumes the earlier Och staging — the core calculus that isolated the unification of terms and types, subtyping in the pure-subtype-systems line [6], and dependent elimination before any mutation was in scope — for the mutation question, and is the Ochre programme's [21] working answer to how full dependent types and in-place mutation inhabit a single system.

10 The Rules of DLLBC

The complete rule set, presented **nondeterministically**: reorganization (Section 10.2) may fire wherever its premises hold; the implementation's lazy, fuel-bounded strategy (fire a reorganization only when a rule's premise demands it) is one deterministic scheduling of these rules, discussed in Section 6. Each figure ends with a rule-to-implementation-to-test correspondence table. Six of the seven present rules that are, in one way or another, bookkeeping; Section 10.7 is the exception and says so — it is the only rule in the calculus that consumes a proof. References of the form "doc §N" point into the calculus's design document (`docs/dllbc-arrows.md` at the pin commit), the specification the figures were extracted against; `@`-references point within this paper.

10.1 F1: Runtime reads and writes (⇒/⇐)

The runtime fragment has two arrows. $\Omega \vdash t \Rightarrow v \dashv \Omega'$ is the **consume-read** (a move): it evaluates t to a value, threading the environment Ω to Ω'. $\Omega \vdash p \Leftarrow v \dashv \Omega'$ is the **destructive write**, and it is **fill-only** — its one premise is that the target holds the hole $\bot$. One grammar carries all variation (doc §1.1): the ⇐, take, and `&mut` rules are only **defined** on **places**, and the same construct means different things under different arrows.

Reorganization is presented nondeterministically. The implementation ends loans and drops values **lazily and deterministically** — a fuel-bounded retry fired exactly when a rule's premise is blocked. F1 factors that out into a single interleaving rule, R-Reorg, and every other rule takes clean premises. The reorganization judgment $\Omega \rightsquigarrow \Omega'$ is defined in F2 (rule prefix `G-`); F1 only cites it.

10.1.1 Places

The write, the *-take, and `&mut` act on a **place** (impl `placeToPos`):

$$p ::= x \quad | \quad *p$$

A place resolves to a root variable under n *-peels. Two conventions, matching `navRead`/`navWrite`: $\mathrm{loc}(\Omega, p)$ is the value reached by peeling n borrow_m -layers into the root slot (each peeled layer must be a borrow, else the step is stuck); and $\Omega[p \mapsto w]$ rethreads w back through those same borrow layers to the root. At $n = 0$ (a bare variable) $\mathrm{loc}(\Omega, x) = \Omega(x)$ and $\Omega[x \mapsto w]$ is the ordinary slot update. This one convention gives reborrow (`&mut *b`), take-at-depth, and write-through for free.

10.1.2 The ⇒ arrow (consume-read)

$$\frac{\Omega(x) = v \qquad v \text{ index-kind}}{\Omega \vdash x \Rightarrow v \dashv \Omega} \text{ R-Copy}$$

A read of an[3] index-kind value copies it: the owner slot is left intact (Ω unchanged), so the value stays re-readable. This is why a comptime index may be used more than once (doc §2.1).

$$\frac{\Omega(x) = v \qquad v \text{ data} \qquad v \text{ move-ready}}{\Omega \vdash x \Rightarrow v \dashv \Omega[x \mapsto \bot]} \text{ R-Move}$$

*A read of **data** (a `Cons`-tree, a pair, a user constructor — anything not index-kind) moves it out, leaving ⊥ behind; a second read is then stuck (no rule).[4] Data moves even when marker-free — silent aggregate duplication is the cost-opacity Rust's move discipline prevents, and this calculus keeps that line (doc §2.1).*

[3] **Index-kind** (impl `indexKindV`): a concrete `Nat`/`Bool`/`Unit` tree, any pure-former value (a proof — `Refl` or a neutral proof spine — a type, a Π-type, or a λ), or a symbolic σ whose σ-context type is one of these. Decided by value **shape** (the σ-context type for a σ), never a declared trait; a σ with no σ-context entry is not index-kind and moves. Index-kind values are marker-free, so R-Copy never needs a reorganization.

[4] **Move-ready**: no loan marker sits in v's owned position (the constructor spine), and if v is itself a borrow its payload hides no suspended reborrow. Both blockers are ended by R-Reorg first — an owned-position marker via G-EndMut (doc §2.2), a carried suspended reborrow via doc §19′s flow-back. The implementation checks these inline (`firstLoanMarker`; the carried-borrow case at `readR` var, doc §19) and retries; F1 makes them R-Reorg preconditions.

$$\frac{\Omega_0 \vdash t_1 \Rightarrow v_1 \dashv \Omega_1 \quad \ldots \quad \Omega_{n-1} \vdash t_n \Rightarrow v_n \dashv \Omega_n}{\Omega_0 \vdash C(t_1 \ \ldots \ t_n) \Rightarrow C(v_1 \ \ldots \ v_n) \dashv \Omega_n}\ \text{R-Ctor}$$

A constructor application $\Rightarrow$-consumes each field left to right, threading Ω, and assembles the constructor value. A nullary `Nil/Z` is the $n = 0$ case with no field premises.

$$\frac{\mathrm{loc}(\Omega, *p) = w \qquad w \neq \bot \qquad \text{own-free}(w)}{\Omega \vdash *p \Rightarrow w \dashv \Omega[*p \mapsto \bot]}\ \text{R-Take}$$

*`*p` in read position is a **take**: it moves the located payload out **through** the borrow, leaving a hole $\bot$ at that place. The borrow layers above are untouched — `b` itself is not consumed — and the hole's only legal successor is a $\Leftarrow$-fill through it (doc §2.4). This is the update-in-place idiom the calculus exists to make routine. The* own-free(w) *premise is a **demand**, discharged the way every demand is: a payload still holding a parked loan — a `&mut *v` handed to a call that has returned, or a field reborrow from a borrow-mode match — is a **suspension**, not a value, and G-EndMut (Section 10.2) collapses it innermost-first before the take proceeds. Without the premise the **marker** is taken and travels on as though it were a value.*

$$\frac{\mathrm{loc}(\Omega, p) = v \qquad v \notin \{\bot, \mathrm{loan}_m \cdot\} \qquad \ell \text{ fresh}}{\Omega \vdash \&\mathrm{mut}\ \ p \Rightarrow \mathrm{borrow}_m \ell\ v \dashv \Omega[p \mapsto \mathrm{loan}_m \ell]}\ \text{R-Mint}$$

`&mut` mints a fresh loan ℓ: ownership of the located value moves into the borrow $\mathrm{borrow}_m \ell\ v$*, and the marker* $\mathrm{loan}_m \ell$ *parks where it sat, rendering the source unusable (not vacant — **owed**). At a bare place x this is a plain borrow (doc §2.2); at `*b` it is a **reborrow** — the parent `b` is suspended, holding* $\mathrm{loan}_m \ell$ *where its payload was, recovering when ℓ ends (doc §2.5).*

$$\frac{\Omega \vdash t \Rightarrow v \dashv \Omega_1 \qquad \Omega_1, x \mapsto v \vdash t' \Rightarrow v' \dashv \Omega_2}{\Omega \vdash \mathrm{let}\ x = t; t' \Rightarrow v' \dashv \Omega_2}\ \text{R-Let}$$

`let` reads its right-hand side by $\Rightarrow$, binds it to a fresh slot, and reads the continuation — the sequencing form of the runtime fragment.

$$\frac{\Omega \vdash t \Rightarrow \_ \dashv \Omega_1 \qquad \Omega_1 \vdash t' \Rightarrow v \dashv \Omega_2}{\Omega \vdash t; t' \Rightarrow v \dashv \Omega_2}\ \text{R-Seq}$$

A sequenced statement is read for its Ω-effect and its value discarded, then the continuation is read. (Doc §1.1 calls this sugar for `let _ = t ; t'`; the implementation keeps a distinct `seq` former that discards the value without dropping it — see the gaps note.)

$$\frac{\Omega \vdash t \Rightarrow v \dashv \Omega_1 \qquad \Omega_1 \vdash p \Leftarrow v \dashv \Omega_2 \qquad \Omega_2 \vdash t' \Rightarrow v' \dashv \Omega_3}{\Omega \vdash p := t; t' \Rightarrow v' \dashv \Omega_3}\ \text{R-Assign}$$

*Assignment evaluates the RHS by $\Rightarrow$ **first**, then fills the target place by $\Leftarrow$ (a live target is vacated by a drop via R-Reorg), then reads the continuation. The RHS-first order is doc §2.5′s, and it is exactly what makes the self-reborrow `b := c` stuck: the RHS consumes `c`, so the value drop needs is in flight.*

$$\frac{}{\Omega \vdash () \Rightarrow \mathrm{unit}\ () \dashv \Omega}\ \text{R-Unit}$$

The unit literal reads to the nullary `unit` value with no effect on Ω; it is what a `let`/`:=`-terminated program ends in.

$$\frac{\Omega \vdash t \rightsquigarrow v \qquad t \text{ a comptime-only former}}{\Omega \vdash t \Rightarrow v \dashv \Omega} \text{ R-Lift}$$

*On the borrow-free fragment $\Rightarrow$ coincides with the comptime read $\rightsquigarrow$ up to consumption (doc §1.3): a comptime-only former (*Type, Π, λ, *an application,* Id *A a b, a constant) is an* ***unrestricted*** *value, so $\Rightarrow$ delegates to $\rightsquigarrow$ (impl* `readC`*) and hands the result back as an ordinary runtime datum — storable in a field, passable to a call, returnable. The lift is non-destructive (these values are copyable/erasable), so Ω is unchanged, modulo the sanctioned pure-*`let`* entries a $\rightsquigarrow$ may introduce (doc §1.3).*

10.1.3 The ⇐ arrow (destructive write)

$$\frac{\text{loc}(\Omega, p) = \bot}{\Omega \vdash p \Leftarrow v \dashv \Omega[p \mapsto v]} \text{ W-Fill}$$

$\Leftarrow$ is ***fill-only****: its single premise is that the target place holds $\bot$, and the value drops in. Writing onto a* ***live*** *place is not a separate rule — a* ***drop*** *reorganization (R-Reorg, doc §2.3) vacates it to $\bot$ first. The place convention makes* `x := v`*,* `*b := v` *(write-through), and* `**x := v` *the same rule at* 0, 1, 2 *peels. A non-place target (*`Pair(1) := v`*) has no* `loc`*, so it is stuck — this is how $\Leftarrow$ rejects writes to arbitrary terms.*

10.1.4 Interleaving

$$\frac{\Omega \overset{*}{\rightsquigarrow} \Omega' \qquad \Omega' \vdash t \Rightarrow v \dashv \Omega''}{\Omega \vdash t \Rightarrow v \dashv \Omega''} \text{ R-Reorg}$$

Before any $\Rightarrow$ (or $\Leftarrow$) step, the environment may reorganize — $\Omega \rightsquigarrow \Omega'$, whose rules live in F2 — to unblock the step: end a loan sitting in a value about to move (G-EndMut, doc §2.2), collapse a match's field-loan chain when its owner is demanded (doc §3.3), end a carried suspended reborrow before its carrier moves (doc §19), or ***drop*** *a live value at a fill target, vacating it to $\bot$ (doc §2.3). The implementation fires these lazily and deterministically (a fuel-bounded retry) exactly when a premise is blocked; F1 states the freedom nondeterministically. The identical rule with $\Leftarrow$ in place of $\Rightarrow$ supplies the drop-before-fill that W-Fill and R-Assign rely on.*

10.1.5 Rule ↔ implementation ↔ test

Implementations are in `Dllbc/Machine.lean` unless prefixed; tests are in `Dllbc/Tests/`. F2′s `endLoan`/`drop` implement R-Reorg's premise. Cells name the **function and case** rather than a line: an earlier version of this table carried line numbers, and every one of them was wrong within a few milestones. Test-side anchors are line numbers because those files are stable.

Rule	Implementation	Test
R-Copy	`readR .var` (index-kind arm); `indexKindV`	S2.lean:35 doc §2.1 copy-on-read
R-Move	`readR .var` (move arm, and its doc §19 carried-borrow retry)	S2.lean:101 take-and-refill (`tail` is data → moved)
R-Ctor	`readR .ctorApp`; `readArgs`	S2.lean:88 `Cons(3, Nil)`
R-Take	`readR .deref` (`firstLoanMarker` + `endLoan` + retry); `navRead`	S2 doc §2.4 `let tail = *b`; `S23Direct`
R-Mint	`readR .borrow`	S2.lean:45 `&mut x`; :117 `&mut *b` reborrow
R-Let	`readR .letIn`	S2.lean:26 (every trace)
R-Seq	`readR .seq`	S8Diff.lean:65 `e ; ()`; S19Partition
R-Assign	`readR .assign`; `writeR`	S2.lean:53 `*b := 7`; :76 `b := 9`
R-Unit	`readR .unit`	S2.lean:30 trailing `()`
R-Lift	`readR` (pure formers) → `collapseCDerefs`, then `readC`	S11Lib.lean:72 `botElim` ⇒-lifted; :61 proof into Σ

Rule	Implementation	Test
W-Fill	`writeR` (fill arm); `setAtPos`; `navWrite`; `placeToPos`	S2.lean:101 fill the hole; :150 non-place reject
R-Reorg	`readR` (the retry arms); `writeR` (drop-before-fill); F2′s `endLoan/drop`	S2.lean:63 End-Mut before read; :76 drop before fill; :129 chain collapse

10.2 F2: Reorganization ($\rightsquigarrow$)

Reorganization is the judgment that rewrites the borrow bookkeeping: ending loans, vacating displaced values, collapsing the loan groups calls leave behind. No surface syntax elaborates to it — its rules fire *between* evaluation steps, and they are presented here **nondeterministically**: a rule may fire at any point where its premises hold, and $\Omega \overset{*}{\rightsquigarrow} \Omega'$ is the reflexive–transitive closure of single steps. The implementation is lazy — it fires a reorganization only when a $\Rightarrow/\Leftarrow$ premise demands one — and carries a fuel bound; both are scheduling, not semantics (the fuel is a totality device for the mechanization, and the demand sites are catalogued before the correspondence table below).

The judgment has two levels. End-Mut and drop touch only the environment and are stated as $\Omega \rightsquigarrow \Omega'$ — such a step lifts to the checker state $\langle \Omega; \Gamma_\sigma; O; G; R\rangle$ with the other components fixed. Group ends delete their group node and mint existentials into Γ_σ, so they are stated over the full state, $S \rightsquigarrow S'$. Two auxiliary judgments — dropping, $\Omega \vdash \mathrm{drop}\, v \dashv \Omega'$, and the issued-borrow surrender, $\Gamma_\sigma; \Omega \vdash \ell_{[\text{owed } \tau]} \Downarrow v \dashv \Omega'$ — are not $\rightsquigarrow$ steps of their own: they occur only as premises (of $\Leftarrow$'s overwrite rule in Section 10.1, and of the group ends here).

Positions. $\hat{\Omega}[\cdot]$ (and the two-hole $\hat{\Omega}[\cdot,\cdot]$, and its m-hole extension) ranges over environment contexts: a marked position is a whole entry of Ω or a position inside an entry's value tree, and $\hat{\Omega}[v]$ is the environment with v at that position. $\hat{v}[\cdot]$ is the same over a single value. A loan id occurs at most once in each role across the state, so these decompositions are unique. own-free(v) says v contains no loan_m or borrow_m node at any depth.

10.2.1 Ending a loan

$$\frac{\Omega = \hat{\Omega}[\mathrm{loan}_m \ell, \mathrm{borrow}_m \ell\ v] \qquad \ell \notin \mathrm{captured}(G)}{\Omega \rightsquigarrow \hat{\Omega}[v, \bot]}\ \text{G-EndMut}$$

The borrow's current payload is plugged back where the marker sits and the borrow node is killed to $\bot$ — a $\Leftarrow$-fill aimed at Ω's own bookkeeping, followed by the kill (doc §2.2). Either half may sit at a whole slot or buried inside a value tree: a marker buried in a constructor field (the field loans a borrow-mode match parks, doc §3.3) and a marker directly under a borrow's payload (a suspended reborrow) end by this same rule. The payload v travels as-is — if it is itself suspended, its markers relocate with it, and the chain collapses by further G-EndMut steps; if it is a hole, the plug-back merely leaves the owner's position vacant and fillable, not corrupt (doc §2.4). The side condition routes captured loans to the group rules: ending a loan a call captured is the group's business. (In reachable states the premise is anyway unsatisfiable for a captured ℓ — its borrow half was consumed into the call and is no longer an Ω position — but the guard is what the implementation checks first, and the rule states it.)

Doc §2.2′s "generalized owned-position rule" is a fact about Section 10.1, not a separate reorganization: a $\Rightarrow$-move demands that the value it is about to move carry no marker in *owned position* — on the constructor spine, not under a borrow_m payload, since a carried borrow relocates with its suspensions intact — and G-EndMut, at whatever depth the marker sits, is the reorganization that clears one. Restricting the rule itself to owned position would be wrong: the demand that arrives *through* a borrow (matching or moving a suspended payload) ends a marker that is not in owned position, by this same rule.

10.2.2 Drop

Fill demands a vacant target; when a $\Leftarrow$ aims at a live value, the value is displaced and must be dropped (doc §2.3). The judgment $\Omega \vdash \mathrm{drop}\, v \dashv \Omega'$ reads: the displaced value v may be discarded, reorganizing Ω

to Ω'. Note the form: v is an operand of the judgment, not an entry of Ω — the implementation vacates the slot *before* dropping, so no stale copy of v is ever visible to the ends' Ω-searches.

$$\frac{\text{own-free}(v)}{\Omega \vdash \text{drop}\, v \dashv \Omega}\ \text{G-DropFree}$$

$$\frac{v = \hat{v}[\text{loan}_m \ell] \qquad \Omega = \hat{\Omega}[\text{borrow}_m \ell\ p] \qquad \hat{\Omega}[\bot] \vdash \text{drop}\, \hat{v}[p] \dashv \Omega'}{\Omega \vdash \text{drop}\, v \dashv \Omega'}\ \text{G-DropLoan}$$

$$\frac{v = \hat{v}[\text{borrow}_m \ell\ p] \qquad \text{own-free}(p) \qquad \Omega = \hat{\Omega}[\text{loan}_m \ell] \qquad \hat{\Omega}[p] \vdash \text{drop}\, \hat{v}[\bot] \dashv \Omega'}{\Omega \vdash \text{drop}\, v \dashv \Omega'}\ \text{G-DropBorrow}$$

Drop is a total procedure, not a choice: end every ownership node the displaced value carries, then discard the remainder. An ownership-free value discards immediately (G-DropFree), so ordinary code pays nothing. A marker in the value ends against its borrow in Ω — the borrow is killed and the payload returns *into the value being dropped*, where the recursive premise continues on it (G-DropLoan: a chain-link value dies with nothing stranded). A borrow node in the value sends its payload home to its marker in Ω and becomes a hole (G-DropBorrow). Doc §2.3′s locatability condition is exactly the premises' shape: the half the rule matched is a position within the value being dropped, and the complementary half must be an Ω position.

The own-free(p) premise on G-DropBorrow is doc §2.3′s "innermost first" made local, and it is load-bearing. Without it, G-DropBorrow could fire on $\text{borrow}_m \ell$ ($\text{loan}_m \ell'$) by sending the still-suspended payload home — rewiring the owner's loan from ℓ to ℓ' and *accepting* the self-reborrow with no ghost machinery, precisely the "twisted" program this calculus rejects (doc §2.5). With it, a borrow node ends only after its payload is clean, so the buried marker must end first — and if it cannot, no rule applies:

Non-derivation — the self-reborrow (doc §2.5). After `let c = &mut *b`, the assignment `b := c` $\Longrightarrow$-consumes `c`, so $\Omega = x \mapsto \text{loan}_m \ell, b \mapsto \text{borrow}_m \ell$ ($\text{loan}_m \ell'$) with $\text{borrow}_m \ell'$ 3 *in flight*, and the fill must first derive $\Omega \vdash \text{drop}\, \text{borrow}_m \ell$ ($\text{loan}_m \ell'$) $\dashv \Omega'$. G-DropLoan on ℓ' needs $\text{borrow}_m \ell'\ p$ as an Ω position — it is the in-flight value, not an entry; G-DropBorrow on ℓ needs own-free($\text{loan}_m \ell'$) — false; G-DropFree needs an ownership-free value — false. No rule applies; the program is rejected. The stuckness *is* the rejection: no rule was added to say so.

10.2.3 Ending a group

A call mints a loan group $A(\rho)\{\bar{C}; \bar{I}\}$ (Section 10.5): the node ties the loans it **captured** (the argument borrows' loans, each annotated with the owed type S from the signature, instantiated at the call) to the borrows it **issued** (the loans of the borrows in its result, owed-typed from the return type). The group's whole content is its ending discipline — issued first, then captured, atomically.

An issued borrow surrenders by the auxiliary judgment: located anywhere in Ω, audited against its owed type, killed.

$$\frac{\Omega = \hat{\Omega}[\text{borrow}_m \ell\ p] \qquad \Gamma_\sigma \vdash p : \tau}{\Gamma_\sigma; \Omega \vdash \ell_{[\text{owed } \tau]} \Downarrow p \dashv \hat{\Omega}[\bot]}\ \text{G-EndIssued}$$

A hole cannot surrender: $\bot$ inhabits no type, so the typing premise already excludes it (the implementation's distinct "nothing surrendered" error is diagnostic, not a separate condition). A payload suspended by a reborrow of the result likewise fails the typing premise; G-EndMut on the parked loan restores it first.

$$\frac{\begin{array}{c}A(\rho)\{\bar{C};\bar{I}\} \in G \quad \text{(no declared back)} \quad \bar{C} = \ell_{1[\text{owed } \tau_1]}, ..., \ell_{m[\text{owed } \tau_m]} \quad \bar{I} = \ell'_{1[\text{owed } S_1]}, ..., \ell'_{n[\text{owed } S_n]} \\ \Gamma_\sigma; \Omega_{i-1} \vdash \ell'_{i[\text{owed } S_i]} \Downarrow v_i \dashv \Omega_i \quad (i = 1, ..., n, \text{ in issued order}) \\ \Omega_n = \hat{\Omega}[\text{loan}_m \ell_1, ..., \text{loan}_m \ell_m] \quad \sigma_1, ..., \sigma_m \text{ fresh} \quad \Gamma'_\sigma = \Gamma_\sigma, \sigma_1 : \tau_1, ..., \sigma_m : \tau_m\end{array}}{\langle \Omega_0; \Gamma_\sigma; O; G; R\rangle \rightsquigarrow \langle \hat{\Omega}[\sigma_1, ..., \sigma_m]; \Gamma'_\sigma; O; G \setminus A(\rho); R\rangle} \text{ G-EndGroupOpaque}$$

$$\frac{\begin{array}{c}A(\rho)\left\{\ell_{c[\text{owed } \tau_c]}; \bar{I}; \text{back } f\right\} \in G \quad \bar{I} = \ell'_{1[\text{owed } S_1]}, ..., \ell'_{n[\text{owed } S_n]} \\ \Gamma_\sigma; \Omega_{i-1} \vdash \ell'_{i[\text{owed } S_i]} \Downarrow v_i \dashv \Omega_i \quad (i = 1, ..., n, \text{ in issued order}) \\ \Omega_n = \hat{\Omega}[\text{loan}_m \ell_c] \quad w = \text{nf}(f\, v_1 \ldots v_n)\end{array}}{\langle \Omega_0; \Gamma_\sigma; O; G; R\rangle \rightsquigarrow \langle \hat{\Omega}[w]; \Gamma_\sigma; O; G \setminus A(\rho); R\rangle} \text{ G-EndGroupBack}$$

Every issued borrow surrenders first; then the group ends *atomically*, every captured loan releasing in the same step. The ordering is the soundness argument made structural (doc §6.1): a captured owner cannot recover while an issued borrow lives, because the release conclusion is unreachable until every surrender premise is derivable — and no other rule can touch a captured loan (G-EndMut's side condition). The opaque end *discards* the surrendered payloads and releases each captured loan with a fresh existential at its owed type, extending Γ_σ — the caller learns exactly what the signature says, and the forgetting is doc §6.2′s deliberate precision cost. The computed end instead applies the declared backward spec f — reflected and instantiated at the call, and checked against the callee's body at its audit (Section 10.5) — to the surrendered values, in issued order, and releases the computed value; no existential is minted. Its $n = 0$ instance is the value-returning mutator (doc §6.2′s dual): no issued borrows, and the release is $\text{nf}(f)$, a pure function of the entry snapshots. The atomic captured release over-approximates the concrete demand-driven ending — doc §6.1 records the simulation obligation — and one concrete demand suffices to fire the whole cascade.

The wire (doc §5.3) is the degenerate group, displayed for the shape doc §5.3′s traces use — it is an instance, not a separate mechanism:

$$\frac{\begin{array}{c}A(\rho)\left\{\ell_{[\text{owed } \tau]}; \emptyset\right\} \in G \quad \text{(no declared back)} \\ \Omega = \hat{\Omega}[\text{loan}_m \ell] \quad \sigma \text{ fresh}\end{array}}{\langle \Omega; \Gamma_\sigma; O; G; R\rangle \rightsquigarrow \langle \hat{\Omega}[\sigma]; (\Gamma_\sigma, \sigma : \tau); O; G \setminus A(\rho); R\rangle} \text{ G-EndOwed } (= \text{G-EndGroupOpaque with } \bar{I} = \emptyset)$$

Ending a call-annotated loan is where the caller learns what the callee did: a fresh value arrives at the owed type — an existential, opened at loan end. Under `&mut List T` the caller learns only that a list came back; under a type-changing $\rightsquigarrow$ obligation, its precise new index.

Global side condition: knowledge vs. state (doc §3.2). Every conclusion in this figure rewrites Ω's value trees *locally* — a plug-back, a kill, a release lands at a loan's position and nowhere else. No reorganization ever records its effect by substituting for a σ: the σ's name *knowledge* (constructor shapes, equation solutions — facts about a value true at entry and forever), while a hole, a marker, or a mutation's result is *state*, a fact about a slot at a moment. The one Γ_σ-effect a reorganization has runs the other way — group ends **extend** Γ_σ with fresh existentials at the owed types; no existing entry is rewritten. Dually, refinement ($\leftarrow$, Section 10.3, Section 10.4) substitutes into every σ-bearing component of the state — including the owed types stored in O and in the group nodes of G, which is why groups live in S — and its replacement must be marker-free. The implementation asserts both directions at the single substitution site (`refineSym`); instrumenting the whole mechanization found zero violations (doc §3.2).

Where the implementation fires these. The rules above carry no laziness; the checker's demand-driven scheduling (doc §2′s standing convention) is one strategy over them, discussed in Section 6. The governing statement is one line — **every demand collapses first** — and the sites are its instances rather than separate rules. A reorganization fires exactly when a $\Rightarrow/\Leftarrow/\rightsquigarrow$ premise needs one: a $\Rightarrow$-read of an owner whose value carries an owned-position marker ends it (`readR`, variable case); a move of a borrow whose payload is suspended ends the parked reborrow first (`readR`, same case); a $\Rightarrow$-**take** through

a borrow whose payload is a suspension ends it before taking (`readR`, deref case — R-Take's own-free premise, Section 10.1); `&mut` of a place holding a parked marker demand-ends its loan — a prior call's whole group, in the sequential-reborrow shape (`readR`, borrow case); a match reorganizes its scrutinee the same way (`reorgScrut`); a **comptime** read on the pure lift collapses the places it is about to project through, a bare variable whose own loan a call still holds included (`collapseCDerefs`); a $\Leftarrow$ onto a live value vacates it by drop (`writeR`); and the return audit collapses argument-borrow payloads with the same ends (`collapseArg`), the boundary being the canonical demander (doc §3.3).

That the list is **closed** is not decoration. Each of the last three entries was added after a body reached a state the earlier entries did not cover, and in every case the symptom was identical: the marker itself was read as though it were a value, and surfaced layers downstream as an untypeable term. The calculus document now states the rule once with the sites as instances, rather than per-site, for exactly that reason — an unlisted site is not a missing convenience but a live silent-marker bug.

Rule	Implementation	Test
G-EndMut	`endLoan` (ungrouped arm): `killBorrowInΩ` then `sendPayloadToLoan`	S2 (forced End-Mut before a move; doc §2.5 chain collapse), S3 (field-loan collapse on owner read)
G-DropFree / G-DropLoan / G-DropBorrow	`drop`, node selection via `firstOwnNode`	S2 (overwrite forces drop, doc §2.3), S3 (variant change ends the field loans, doc §3.4)
self-reborrow non-derivation	`killBorrowInΩ`'s in-flight error	S2 (`b := c` rejected, "in flight")
G-EndIssued	`endIssued`	S7Group (holed issued payload: "nothing surrendered")
G-EndGroupOpaque	`endGroup` (default arm)	S7Group (`choose` cascade: fresh σ's for both owners; `through`: precision deliberately lost)
G-EndGroupBack	`endGroup` (back-spec arm), `Val.rebuildSpine`	S17Spec (`spcCaller` recovers the computed release), S19Partition (`twoRec` sequential reborrow; quicksort conformance)
G-EndOwed	`endGroup` with `issued = []`	S6Call (the doc §5.3 wire: "the promise is collected")

10.3 F3: The comptime arrows ($\rightsquigarrow$/$\leftsquigarrow$)

The comptime pair is the runtime pair (Section 10.1) restricted to the borrow-free, assignment-free fragment: $\rightsquigarrow$ is the $\Longrightarrow$ column minus the borrowing rows, minus `:=`, and minus consumption (doc §1.3). Two shape facts encode that restriction and are worth stating before the rules. First, **$\rightsquigarrow$ carries no output environment** — the judgment is $\Omega \vdash t \rightsquigarrow v$, with no Ω'. That shape **is** the effect-freedom claim: a comptime read cannot move, mint, or assign, so it has nothing to hand back (the sole footprint any $\rightsquigarrow$-evaluation leaves is the fresh pure entries its `let`s bind, and those are confined to the premise that uses them — rule C-Let). Second, $\leftsquigarrow$ is the checker's **refinement** judgment $S \vdash x \leftsquigarrow t \dashv S'$: it has no surface syntax (no program text elaborates to $\leftsquigarrow$), fires only at match-branch entry, and rewrites the whole checker state by substitution. Throughout this figure the state abbreviates $S = \langle \Omega; \Gamma_\sigma; O; G; R \rangle$ — the environment, the σ-context, the boundary obligations (Section 10.5), the loan groups (Section 10.5), and the pinned return type.

10.3.1 The $\rightsquigarrow$ arrow: reduction (β and ι)

`readC` is reflect-then-normalize: `reflectC` maps a term into a value (resolving Ω snapshot reads, below) and `nfV` normalizes it by β and ι. The reduction rules read directly off `whnfV`.

$$\frac{\Omega \vdash t[x := u] \rightsquigarrow v}{\Omega \vdash (\lambda(x : \tau).t)\ u \rightsquigarrow v}\ \text{C-Beta}$$

$$\frac{\Omega \vdash n \rightsquigarrow \mathrm{Z}\ () \qquad \Omega \vdash z \rightsquigarrow v}{\Omega \vdash \mathrm{natRec}\ \ P\ z\ s\ n \rightsquigarrow v}\ \text{C-IotaNatZ} \qquad \frac{\Omega \vdash n \rightsquigarrow \mathrm{S}\ (m) \qquad \Omega \vdash s\ m\ (\mathrm{natRec}\ \ P\ z\ s\ m) \rightsquigarrow v}{\Omega \vdash \mathrm{natRec}\ \ P\ z\ s\ n \rightsquigarrow v}\ \text{C-IotaNatS}$$

$$\frac{\Omega \vdash p \rightsquigarrow \mathrm{Refl}\ () \qquad \Omega \vdash d \rightsquigarrow v}{\Omega \vdash \mathrm{j}\ \ A\ a\ P\ d\ b\ p \rightsquigarrow v}\ \text{C-IotaJ} \qquad \frac{\Omega \vdash p \rightsquigarrow \mathrm{Refl}\ () \qquad \Omega \vdash d \rightsquigarrow v}{\Omega \vdash \mathrm{k}\ \ A\ a\ P\ d\ p \rightsquigarrow v}\ \text{C-IotaK}$$

Gloss. C-Beta is ordinary β: a `lam`-headed spine substitutes its argument (`substPure 0 u b` in the implementation — the delayed-lift substitution of the perf note). C-IotaNatZ/C-IotaNatS are the two `natRec` ι-rules: reduce the target, and fire on its constructor — `Z` selects the base, `S m` unfolds one step and recurs. **`boolRec` and `listRec` follow the identical scheme** (reduce the scrutinee; one rule per constructor — `True`/`False` for `boolRec`, `Nil`/`Cons h t` for `listRec`, the `Cons` case recurring on the tail exactly as `S m` recurs on the predecessor); they are elided here for space. **`sigmaRec` does not follow that scheme** and is worth its own sentence: Σ is not inductive in its own right, so its eliminator is non-recursive — one arm, no `ih`, firing on `Pair` with sigmaRec $A\ B\ P\ f$ (Pair $a\ b$) $\rightsquigarrow f\ a\ b$, and leaving a legal stuck spine on a neutral target like every other recursor. It completes the basis's one-recursor-per-former, and it is what lets a caller take a returned Σ certificate **apart** rather than merely build one — the prerequisite the direct architecture of Section 7 rests on. C-IotaJ and C-IotaK are the fording kit's eliminators (doc §10) and get explicit rules: Paulin-Mohring `j` and Streicher `k` both fire **only** on `Refl`, collapsing to the `Refl`-case witness `d`. `botElim` has no ι-rule at all — ⊥ has no constructors, so a `botElim` spine is always a stuck value.

10.3.2 The $\rightsquigarrow$ arrow: reflection (the Ω-facing rules)

These three are `reflectC`: how a comptime read consults Ω. Each is non-destructive — Ω is read, never written.

$$\frac{\Omega(x) = v \qquad v \neq \bot}{\Omega \vdash x \rightsquigarrow v}\ \text{C-Snap} \qquad \frac{\Omega \vdash t \rightsquigarrow \mathrm{borrow}_m \ell\ v \qquad v\ \text{proper}}{\Omega \vdash *\,t \rightsquigarrow v}\ \text{C-Deref}$$

$$\frac{\Omega \vdash \mathrm{rhs} \rightsquigarrow v \qquad \Omega, x \mapsto v \vdash \mathrm{rest} \rightsquigarrow w}{\Omega \vdash (\mathrm{let}\ x = \mathrm{rhs}; \mathrm{rest}) \rightsquigarrow w}\ \text{C-Let}$$

Gloss. C-Snap is the snapshot read: a variable resolves to the value its slot **currently** holds, and — crucially — does not consume it (no Ω′; the owner keeps the value). The side-condition $v \neq \bot$ is the `reflectC` guard (doc §2.1): every read-shaped rule excludes the vacant slot, so a comptime read of a moved or uninitialized variable is **stuck**, not a silent ⊥. C-Deref is the comptime deref (doc §5.2): $*\,(\mathrm{borrow}_m \ell\ v)$ projects the payload snapshot v — a pure projection, never a store lookup, so a type mentioning `*b` is never stale. The proviso "v proper" is load-bearing: a **suspended** borrow whose payload carries loan markers (mid-match, Section 10.4) has no meaningful snapshot, and comptime deref is undefined on it until the reborrows end. C-Let is the pure `let`: reflect the right-hand side, bind it as a **fresh** Ω entry, and read the body under the extension. The extension $\Omega, x \mapsto v$ appears only in the premise — it is the one sanctioned footprint of a comptime read (doc §1.3), scoped to the body's evaluation and invisible in the conclusion's no-Ω′ shape.

> **Footnote (kernel gap, doc'd in the milestones record).** C-Let presents what $\rightsquigarrow$ does **today**: `reflectC` eliminates the `let` at reflection time by binding a fresh Ω entry. The value-level normalizer `nfV` has **no** `letIn` rule — `Val` has no `let` former at all — so `let` reduces only along the reflection pass, never during pure normalization or conversion. `let` is thus one of the two bimodal surface forms (with `&mut`); a single kernel `nfV letIn`/β rule would make it mode-free. Whether to add it (kernel simplicity) or keep the surface split is an open decision, deliberately left to the user.

10.3.3 Stuckness is the absence of a rule

One prose note covers every $\rightsquigarrow$ dead-end, since none is a side effect. A recursor whose target reduces to a **neutral** (a bare σ, or a stuck spine) matches no ι-rule and is returned as a stuck value — this is exactly the "type computes under a stuck index" property the `Vec` encoding relies on (VecF Nat S (σ_1)

unfolds while σ_1 is unknown; doc §4.2). `botElim` never fires. A comptime read of $\bot$ is stuck (C-Snap's side-condition). And the refl-match below is stuck when neither endpoint is a flexible σ. In each case there is simply no rule, and the value stands as a legal stuck neutral.

10.3.4 The $\twoheadleftarrow$ arrow: refinement

Refinement fires at branch entry, once the scrutinee has been read ($\rightsquigarrow$) to expose what it holds. It substitutes globally, subject to a single deep invariant:

Global knowledge/state constraint on every X-rule (doc §3.2 — the calculus's deepest invariant). Write $S[\sigma := t]$ for substituting t for σ in **every** σ-bearing component of the checker state: Ω, the σ-context Γ_σ, the boundary obligations O, every loan group in G (its captured and issued owed types, and its backward spec), the pinned return type R, and the self-back spec.

This has a **dual** that bounds what a refinement may carry. $\twoheadleftarrow$ propagates **knowledge** — a constructor shape true of the value timelessly, or an equation solution — and never **state**. The replacement t must be marker-free: no hole $\bot$, no $\text{loan}_m \ell$, no $\text{borrow}_m \ell$ (enforced at `refineSym` by `hasStateMarker`). Consequently the **only** σ-substitutions in the entire calculus are match-shape refinement (X-Shape) and equation solutions (X-Sol) — **never** a mutation's result. Substituting a take, a fill, or an End-Mut plug-back for a σ would make the snapshot track the present, which is the staleness doc §0 promises snapshots can never have. (Audited across the whole mechanization: zero violations.)

$$\frac{\Omega \vdash p \rightsquigarrow \sigma \qquad \Gamma_\sigma(\sigma) = \tau \qquad C \in \text{ctors}(\tau) \qquad \overline{\sigma} \text{ fresh, typed by } C\text{'s telescope}}{S \vdash p \twoheadleftarrow C(\overline{\sigma}) \dashv S[\sigma := C(\overline{\sigma})]}\ \text{X-Shape}$$

$$\frac{p : \&\text{mut } (A : a \hookrightarrow b) \quad \Omega \vdash a \rightsquigarrow \sigma \qquad \Omega \vdash b \rightsquigarrow b' \qquad \sigma \notin b'}{S \vdash p \twoheadleftarrow \text{Refl } () \dashv S[\sigma := b']}\ \text{X-Sol}$$

$$\frac{p : \&\text{mut } (A : a \hookrightarrow b) \quad \Omega \vdash a \rightsquigarrow a' \qquad \Omega \vdash b \rightsquigarrow b' \qquad a' \equiv b'}{S \vdash p \twoheadleftarrow \text{Refl } () \dashv S}\ \text{X-SolRefl}$$

$$\frac{\Omega \vdash x \rightsquigarrow e \qquad e \text{ a stuck spine } (e \neq \sigma) \qquad \sigma_\text{b} : \text{Bool fresh}}{S \vdash x \twoheadleftarrow \sigma_\text{b} \dashv S[e := \sigma_\text{b}]}\ \text{X-Gen}$$

Gloss. X-Shape is the branch-entry refinement of doc §3.2/doc §3.3. Once the scrutinee place p reads to a symbolic σ (owned mode: $p = x$; borrow mode: $p = *\,x$, the deref cell of the $\twoheadleftarrow$ column), the branch for constructor C mints fresh field σ's — typed by C's field telescope in Γ_σ, which is what discharges the obligation that C actually inhabits σ's type — and substitutes $C(\overline{\sigma})$ for σ **everywhere**. In owned mode the scrutinee slot then goes to $\bot$ and the binders take the fresh σ's; in borrow mode the payload becomes $C(...)$ over fresh loan markers (a suspended parent) and the binders become reborrows — but the **refinement** is this one rule either way, run first.

X-Sol and X-SolRefl are the refl-match **solution transition** (doc §10, `reflUnify`): matching a symbolic $p : \&\text{mut } (A : a \hookrightarrow b)$ against `Refl` unifies the endpoints. Whnf both; if one side is a flexible σ not occurring in the other (X-Sol the occurs-check side-condition $\sigma \notin b'$ is mandatory), refine it to the other side everywhere; if the endpoints already convert (X-SolRefl), the refinement is the identity on state. This is **solution only** — no injectivity, conflict, or cycle: when **both** endpoints are rigid the match is stuck (no rule), naming `j/k` as the elimination route the library must take.

X-Gen is the stuck-spine split (doc §19, `generalizeStuck`), the machine-level inverse of substitution. When a `match/if` scrutinee reads to a stuck neutral spine e (e.g. `leb` $\sigma\ \sigma_\text{p}$ — an application, **not** a bare σ), no X-Shape can fire, since refinement needs a substitutable σ. So normalize e, mint a fresh $\sigma_\text{b} : \text{Bool}$, and abstract every occurrence of e to σ_b across the same all-of-state targets $[\sigma := t]$ reaches. The ordinary X-Shape on σ_b (`True/False`) then refines the spine per branch, in values **and** types alike.

The rule has a second output, and it is load-bearing elsewhere: the **normalized spine** e itself. Abstraction is exactly what makes e unnameable in the branch — nothing in the refined state mentions it any more, and a body that recomputes it writes a term the refinement never saw — so the branch equation of Section 10.4 can only be built from the value this rule hands back. The pairing is not incidental: X-Gen

is the one split where knowledge has to be reified rather than substituted, and it is the one split that returns the material for doing so.

Rule-gap notes (implementation vs. this figure / doc).

- **C-Deref proviso.** `reflectC`'s deref case projects a `borrowM`'s payload **unconditionally** — it does not test the payload for loan markers. What changed at this pin is who arranges that the condition **holds**. A comptime read is a **demand** like any other, so where a body reads live state — the pure lift — a parked loan is **ended first**, and the projection then meets a proper payload (`collapseCDerefs`, which treats a bare variable whose own loan a call still holds as a place too). `readC` proper, which also reads types and specs, stays the read-only projection it is documented as. So the proviso is discharged by the lazy-ending discipline rather than by making the peel stuck, and what remains is presentational: the figure states as a side-condition something a premise elsewhere establishes.
- **Comptime `match` deferred.** `reflectC` errors on a surface `.matchE` ("not implemented in the comptime fragment"). ι-reduction of case analysis is therefore reached **only** through the recursor constants (C-IotaNat… etc.), never a `match` form — the doc's §3.1 note that the v0 mechanization "defers comptime match entirely." So the doc's arrow table entry marking `match` as $\rightsquigarrow$-defined is aspirational at the surface; it holds today only via the recursors it elaborates to (doc §9).
- **X-Gen is Bool-only.** `generalizeStuck` mints a σ_{b} : Bool specifically; generalizing a stuck spine of another type is not implemented (nothing needs it yet — the only stuck-scrutinee split in the corpus is `leb/if`).

10.3.5 Rule ↔ implementation ↔ test

Rule	Implementation	Test
C-Beta	`Pure.whnfV` (β case)	`S4Pure`
C-IotaNatZ/S	`Pure.whnfV` (`natRec`)	`S4Pure`
C-IotaBool/List	`Pure.whnfV` (`boolRec`/`listRec`)	`S4Pure`, `S11Lib`
C-IotaSigma	`Pure.whnfV` (`sigmaRec`)	`S23Direct`
C-IotaJ/K	`Pure.whnfV` (`j`/`k`)	`S10Ford`
C-Snap	`Machine.reflectC` (`.var`)	`S4Pure`, `S3Sym`
C-Deref	`Machine.reflectC` (`.deref`)	`S5Bound`
C-Let	`Machine.reflectC` (`.letIn`)	`S4Pure`
X-Shape	`Machine.refineSym` via `symOwnedSetup`/ `symBorrowSetup`	`S3Sym`
X-Sol/Refl	`Machine.reflUnify` → `refineSym`	`S10Ford`
X-Gen	`Machine.generalizeStuck` (`abstractInto`)	`S19Partition` (`stuckProbe`)

10.4 F4: Match

Match is the only eliminator, and its symbolic rule is the calculus's founding identification: **entering a branch on a symbolic scrutinee is dependent pattern matching** in the Agda style — refinement by unification and substitution (the $\leftsquigarrow$ of Section 10.3), never by threading equality proofs. The concrete modes are the two ways a branch's binders meet the scrutinee — an owned value is **destructured** (doc §3.1), a mutable borrow is matched **through** (doc §3.3) — and the symbolic split (doc §3.2) is where type-checking becomes case analysis.

The scrutinee is a **variable** (a substitutable identity is what refinement needs), and branch patterns bind constructor fields one level deep; `overline(x)` denotes the field-binder vector, `overline("br")` the branch list, `Δ_C` the field telescope of constructor `C`, and `"ctors"(T)` the constructor set of type `T`. Inside a `match` term a branch is written `C(overline(x)) => t`. The form carries one further, **optional** binder — the **branch equation** `match h : x { … }`, marked $[h :]^{?}$ in the conclusions below. Present, it is bound in every branch by the shared premise of the third subsection; absent, every eqn(·) premise is empty and the rules read exactly as they did before it existed. Its purpose and its cost profile are Section 3′s subject; the rules here only say what it binds.

10.4.1 Scrutinee reorganization (a shared premise)

Every rule below assumes the scrutinee slot has first been **reorganized**, lazily and innermost-first, until it exposes a constructor or a symbolic value (`reorgScrut`). Three reorganizations can fire, each owned by

another figure and cited, not restated: G-EndMut on a **suspended owner** or a **reborrowed payload** (Section 10.2); and, when the scrutinee whnf-reduces to a **stuck spine** — a neutral `Bool` like $\mathrm{leb}\,\sigma\,\sigma'$, not a bare σ — its generalization to a fresh $\sigma_b : \mathrm{Bool}$ across the whole state (Section 10.3, X-Gen), after which it dispatches as an owned-symbolic scrutinee. The outcome is one of four dispatches: owned or borrow mode, concrete or symbolic. X-Gen yields **two** things — the fresh σ_b and the normalized spine it abstracted — because the abstraction is what makes that spine unnameable in the branch, and the shared premise below is the only place it can be recovered.

10.4.2 The branch equation (a shared premise)

`match h : x { … }` binds `h`, in every branch, to a proof that the scrutinee's **pre-split** value equals this branch's constructor. Write p for that pre-split value — the value the slot (or, in borrow mode, the payload) held before the split, **before** any generalization abstracted it — and define the environment extension

$$\mathrm{eqn}(h, \tau, p, C\ \overline{\sigma}) = \begin{cases} \emptyset & \texttt{h} \text{ absent} \\ \{h \mapsto \mathrm{Refl}\} & p \equiv C\ \overline{\sigma} \\ \{h \mapsto \sigma_e\} & \text{otherwise, with } \sigma_e : \mathrm{Id}\ \ \tau\ p\ (C\ \overline{\sigma}) \text{ added to } \Gamma_\sigma \end{cases}$$

The three cases are not three behaviours but one, read off what happened to p. On a **concrete** scrutinee p is the constructor value itself. On a **plain symbolic** scrutinee the $\leftsquigarrow$ refinement of the same rule has just rewritten p to $C\ \overline{\sigma}$ **everywhere**, so the endpoints are identical. Both give `Refl`, and nothing is minted. Only where the scrutinee was a **stuck spine** is the third case reached, because X-Gen **abstracted** p rather than refining it — which is precisely what leaves p unnameable afterwards, and so what makes an equation about it worth binding. The fresh σ_e is registered in Γ_σ **before** the refinement fires, so it is swept by the same $\leftsquigarrow$ that sweeps every other σ-bearing component (Section 10.3); it needs no update path of its own. `Refl` here is not interchangeable with the third case: `Refl` inhabits an identity only when its endpoints convert (Section 10.6, T-Ctor), and a stuck spine does not convert with a constructor.

10.4.3 Concrete modes

$$\frac{\Omega(x) = C(\overline{v}) \qquad C(\overline{x}) \Rightarrow t \in \overline{\mathrm{br}} \qquad \Omega[x \mapsto \bot, \overline{x} \mapsto \overline{v}] \cup \mathrm{eqn}(h, T, C\ \overline{v}, C\ \overline{v}) \vdash t \Rightarrow v \dashv \Omega'}{\Omega \vdash \mathrm{match}\ \ [h :]^?\ x\ \left\{\overline{\mathrm{br}}\right\} \Rightarrow v \dashv \Omega'}\ \text{M-Owned}$$

Owned destructuring (doc §3.1): the scrutinee is $\Rightarrow$-consumed (its slot $\leftarrow \bot$), its fields move into the binders, and the selected branch body is evaluated. On concrete values this is ι-reduction wearing binder syntax.

$$\frac{\begin{array}{c}\Omega(x) = \mathrm{borrow}_m \ell\ C(\overline{v}) \qquad C(\overline{x}) \Rightarrow t \in \overline{\mathrm{br}} \quad \overline{\ell'} \text{ fresh} \\ \Omega' = \Omega\left[x \mapsto \mathrm{borrow}_m \ell\ C\left(\overline{\mathrm{loan}_m \ell'}\right), \overline{x}_i \mapsto \mathrm{borrow}_m \ell'_i\ v_i\right] \cup \mathrm{eqn}(h, T, C\ \overline{v}, C\ \overline{v}) \\ \Omega' \vdash t \Rightarrow v \dashv \Omega''\end{array}}{\Omega \vdash \mathrm{match}\ \ [h :]^?\ x\ \left\{\overline{\mathrm{br}}\right\} \Rightarrow v \dashv \Omega''}\ \text{M-Borrow}$$

Matching **through** a mutable borrow (doc §3.3): each field binder becomes a whole-value **reborrow** of its component (fresh loan ℓ'_i), and the parent's payload becomes a `C` of loan markers — the parent is **suspended**, since no rule reads through a loan, so the children's intermediate states are unobservable through `x`. The field writes inside the branch are **contract-free strong updates**. The field loans do **not** end at branch exit; they collapse only when a demand arrives through the parent chain — the owner is read (G-EndMut, Section 10.2) or the boundary audit fires (Section 10.5) — doc §3.3′s precision on the collapse trigger. The nullary case (`Nil()`) is degenerate: no binders, no loans issued, the refinement alone updates the payload.

10.4.4 Symbolic modes: the split

Inside a body, arguments are symbolic. A match on a symbolic scrutinee does not reduce to one successor — it **forks**, and the judgment is **set-valued**:

$$\Omega \vdash \mathrm{match}\ \ [h :]^?\ x\ \left\{\overline{\mathrm{br}}\right\} \Rightarrow \{(v_i, \Omega_i)\}_i$$

— one (result, state) pair per branch, checked independently. Duplication, not a join, is the baseline (doc §3.5): the continuation is re-checked under each branch (see the elaboration note below), so there is no merge step and each path carries its own refined state. The runs are up to renaming of loan and symbolic ids, which branches mint independently. Nondeterministic, no fuel in the judgment (the implementation bounds spine depth only).

$$\frac{\sigma : T \in \Gamma_\sigma \qquad \mathrm{ctors}(T) \subseteq \{\overline{C}\}}{\mathrm{exhaustive}(\sigma, \overline{C})}\ \text{M-Exhaust}$$

A side-condition on both symbolic rules: the branch heads $\overline{C}$ must cover the scrutinee type's constructor set (`checkExhaustive`). A missing branch is an accepted-but-concretely-stuck program, so exhaustiveness is the simulation theorem's precondition made syntactic. `Bot`'s constructor set is empty, so the empty match on a `Bot` scrutinee is vacuously exhaustive.

$$\frac{\begin{array}{c}\Omega(x) = \sigma \quad \sigma : T \in \Gamma_\sigma \quad \mathrm{exhaustive}(\sigma, \overline{C}) \qquad \text{for each } C_i(\overline{x}_i) \Rightarrow t_i \in \overline{\mathrm{br}} : \quad \overline{\sigma}_i : \Delta_{C_i} \text{ fresh} \\ S \vdash x \leftsquigarrow C_i(\overline{\sigma}_i) \dashv S_i \quad \text{(X-Shape)} \\ \Omega_i[x \mapsto \bot, \overline{x}_i \mapsto \overline{\sigma}_i] \cup \mathrm{eqn}(h, T, p, C_i\ \overline{\sigma}_i) \vdash t_i \Rightarrow v_i \dashv \Omega_i'\end{array}}{\Omega \vdash \mathrm{match}\ \ [h :]^?\ x\ \{\overline{\mathrm{br}}\} \Rightarrow \{(v_i, \Omega_i')\}_i}\ \text{M-OwnedSym}$$

The symbolic owned split (doc §3.2) — **the founding identification made a rule**. Branch entry is a $\leftsquigarrow$ refinement (X-Shape, Section 10.3): the constructor shape $C_{i(\overline{\sigma}_i)}$ is substituted for σ **everywhere** in the whole checker state S (Ω, the σ-context, obligations, groups, the pinned return type — not Ω alone), and only then are the field σ's minted, **typed by the instantiated field telescope** Δ_{C_i} (dependent positions instantiated at the earlier fresh σ's — `typeFieldSyms`), the scrutinee consumed, and the branch body checked. Ω_i is the Ω-projection of the refined state S_i.

$$\frac{\begin{array}{c}\Omega(x) = \mathrm{borrow}_m \ell\ \sigma \quad \sigma : T \in \Gamma_\sigma \quad \mathrm{exhaustive}(\sigma, \overline{C}) \\ \text{for each } C_i(\overline{x}_i) \Rightarrow t_i \in \overline{\mathrm{br}} : \quad \overline{\sigma}_i : \Delta_{C_i} \text{ fresh} \quad \overline{\ell'}_i \text{ fresh} \\ S \vdash * x \leftsquigarrow C_i(\overline{\sigma}_i) \dashv S_i \quad \text{(X-Shape, at the payload)} \\ \Omega_i' = \Omega_i[x \mapsto \mathrm{borrow}_m \ell\ C_i(\mathrm{loan}_m \ell'_i), \overline{x}_{i,j} \mapsto \mathrm{borrow}_m \ell'_{i,j}\ \sigma_{i,j}] \cup \mathrm{eqn}(h, T, \sigma, C_i\ \overline{\sigma}_i) \\ \Omega_i' \vdash t_i \Rightarrow v_i \dashv \Omega_i''\end{array}}{\Omega \vdash \mathrm{match}\ \ [h :]^?\ x\ \{\overline{\mathrm{br}}\} \Rightarrow \{(v_i, \Omega_i'')\}_i}\ \text{M-BorrowSym}$$

Borrow mode on a symbolic payload (doc §3.3): refine **at the payload** ($\leftsquigarrow$ at `*x`, substituting σ everywhere) **then** reborrow the fields, suspending the parent — the same field-loan / suspension structure as M-Borrow, entered per path.

$$\frac{\begin{array}{c}\Omega(x) = \sigma \quad \sigma : \mathrm{Id}\ A\, a\, b \in \Gamma_\sigma \qquad a \Downarrow a' \quad b \Downarrow b' \quad a' = \sigma_f \quad \sigma_f \notin \mathrm{syms}(b') \\ S \vdash \sigma_f \leftsquigarrow b' \dashv S_1 \quad \text{(X-Sol)} \qquad S_1 \vdash x \leftsquigarrow \mathrm{Refl} \dashv S_2 \qquad \Omega_2[x \mapsto \bot] \vdash t \Rightarrow v \dashv \Omega'\end{array}}{\Omega \vdash \mathrm{match}\ x\{\mathrm{Refl} \Rightarrow t\} \Rightarrow \{(v, \Omega')\}}\ \text{M-Refl-Flex}$$

The refl-match — the **solution** transition (`reflUnify`, via `mintFieldSyms`). Matching `Refl` at an identity type whnf-unifies the endpoints: a **flex** endpoint (a substitutable σ not occurring on the other side — the occurs check is real) is solved to the other by X-Sol (Section 10.3), symmetrically in a/b; the scrutinee's own σ is then refined to `Refl` (a second $\leftsquigarrow$), and the `Refl` branch, being field-free, consumes and continues. The borrow case is identical at `*x`.

When both endpoints whnf to rigid heads, the match is **stuck** — there is no rule. The kernel performs no injectivity, conflict, or cycle reasoning; such identities are eliminated by the `j`/`k` fording eliminators (Section 10.3), not by a match rule.

10.4.5 The σ-typing obligation

The premise $\overline{\sigma}_i : \Delta_{C_i}$ hides an obligation: the branch constructors must be constructors **of the scrutinee's type** — C_i must belong to $\mathrm{ctors}(T)$, and Δ_{C_i} is read from the signature table with its dependent positions instantiated (`mintFieldSyms` / `typeFieldSyms`). The symbolic machinery cannot see this alone; it is discharged by the σ-context Γ_σ of the comptime fragment (doc §9). A stray constructor is rejected ("does not belong"), distinct from the exhaustiveness rejection above.

10.4.6 Elaboration note (not a rule)

`match` is a statement-position form. A syntactic pre-pass pushes each match's continuation into every branch (`let y = match … ; k` and `match … ; k` become **terminal** matches, duplicating `k`), so every match splits in tail position and the continuation is re-checked once per branch (`pushContinuations`, then `explore`). This is the doc §3.5 **duplication baseline**: maximally precise, no generalization step, worst-case exponential in match-nesting depth. A **symbolic** match left in **expression** position (a constructor argument) cannot split and is rejected ("expression position"); a concrete expression-position match is fine.

The doc §3.5 **join** (reorganize branches to a common shape, then generalize the values that still differ — the $\leftsquigarrow$ of refinement run backwards) is a documented future **widening** of this duplication; it is not implemented, and no rule depends on it. Comptime match (the $\rightsquigarrow$ column, doc §3.1) is likewise deferred.

10.4.7 Correspondence

Rule	Implementation	Test
M-Owned	`ownedSelect`, `readR`/`exploreMatch`	`S3` doc §3.1
M-Borrow	`borrowSelect`	`S3` doc §3.3–3.4
M-OwnedSym	`symOwnedSetup`, `exploreSymBranches`	`S3Sym`, `S5Bound`
M-BorrowSym	`symBorrowSetup`	`S3Sym`, `S5Bound`
M-Refl-Flex	`reflUnify` (via `mintFieldSyms`)	`S10Ford`
M-Exhaust	`checkExhaustive`	`S5Bound`
σ-typing	`typeFieldSyms`, `mintFieldSyms`	`S5Bound`
X-Gen (cited)	`generalizeStuck`, `reorgScrut`	`S19Partition`
eqn$(\cdot)$, `Refl` cases	`bindEqnRefl`	`S23Direct`
eqn$(\cdot)$, stuck case	`mintStuckEqn`, `symOwnedSetup`	`S23Direct`

10.5 F5: Boundaries, calls, and the audit

Where the Aeneas toolchain [5] *synthesizes* a backward function per borrow — a pure function describing what flows back through it — DLLBC moves that description into the signature and checks it: the $\hookrightarrow$ obligation is the backward function's *type* (B-Seed, B-Audit), and, at the precise end of the doc §6.2 spectrum, a declared `back` is the backward function *itself*, audited against the body it claims to describe (B-Back0/B-BackN) rather than synthesized from it. This figure is the contract layer: how a signature is entered (seeding), how it is used (the call), and how it is discharged (the audit at return — the *only* check in the whole borrow story).

Reading conventions. All rules are nondeterministic and fuel-free; the implementation's fuel bounds its *search*, not the relations. Φ is the ambient declaration table; d ranges over declarations $\mathrm{fn}\ f(\Delta) \to T$ with telescope $\Delta = (x_1 : \tau_1)...(x_n : \tau_n)$ and an optional declared backward spec $d.\mathrm{back} = b$. The checker state is $\mathcal{S} = \langle \Omega; \Gamma_\sigma; O; G; R \rangle$ (environment, σ-context, obligations, groups, pinned return type); rules display only the components they touch, the rest ride along unchanged. We write $v \in \Omega$ (and $\mathrm{loan}_m \ell \in p$) for "occurs anywhere inside some slot's value", not merely at top level. $\mathcal{S}[\Omega_1]$ (and likewise for other components) is $\mathcal{S}$ with that component replaced; $\mathcal{S}[\Gamma_\sigma, \sigma : \hat{\tau}]$ extends a component in place. Substitution instances are normalized; nf is left implicit except where the normalization *is* the check. O, R, and the reflected backward specs are inside refinement's reach: an F3 $\leftsquigarrow$ substitutes into them exactly as into Ω — a `Refl`-match that refines a σ rewrites the owed types that mention it, and the audit sees the refined obligation. Premises draw freely on the other figures: $\Rightarrow$ (F1), reorganization $\rightsquigarrow\!\!\!\!\rightsquigarrow$ (F2), $\rightsquigarrow$ (F3), the symbolic match fork (F4), and typing $v : \tau$ / conversion $\equiv$ (F6).

The auxiliary judgment forms, top-down: $\mathcal{S} \vdash \mathrm{seed}(\Delta) \dashv \mathcal{S}'$ seeds a telescope at function entry; $\mathcal{S}\,;\theta \vdash \overline{a} : \Delta \Rightarrow C \dashv \mathcal{S}'\,;\theta'$ checks a call's arguments against a telescope, accumulating the instantiation θ (parameter variable $\mapsto$ checked actual) and the captured loans C; $\theta \vdash T \underset{\mathrm{res}}{\Rightarrow} (v\,; I) \dashv \mathcal{S}'$ builds a call's fresh result from

the instantiated return type, collecting the issued loans I; $T \rhd v \Rightarrow B$ collects, at return, the result's borrow positions against the return type; $\ell \twoheadrightarrow \ell'$ is transitive reachability through reborrow chains and group links; $\mathcal{S}\,; I \vDash \text{ob}$ audits one obligation given issued loans I; $\mathcal{S} \vDash \text{back}_I(d)$ checks a declared backward spec against the body's suspension tree; and the two pinned forms $\mathcal{S} \vDash \text{audit}(v)$ and $\vdash \text{fn } d$ ok close the figure.

10.5.1 Seeding the boundary

$$\frac{}{\mathcal{S} \vdash \text{seed}(\cdot) \dashv \mathcal{S}}\ \text{B-Seed-Nil}$$

$$\frac{\text{no \&mut type occurs in } T \qquad \overline{\sigma_{\text{exit}}} \text{ fresh, one per borrow param} \qquad \Omega \vdash \text{markExit}(T) \rightsquigarrow \hat{R}}{\langle \Omega; \Gamma_\sigma; O; G; \cdot \rangle \vdash \text{pin}(T) \dashv \langle \Omega; \Gamma_\sigma; O; G; \hat{R} \rangle}\ \text{B-Pin}$$

$$\frac{\tau \text{ not a \&mut type} \qquad \Omega \vdash \tau \rightsquigarrow \hat{\tau} \qquad \sigma \text{ fresh} \qquad \langle \Omega[x \mapsto \sigma]; (\Gamma_\sigma, \sigma : \hat{\tau}); O; G; R \rangle \vdash \text{seed}(\Delta) \dashv \mathcal{S}'}{\langle \Omega; \Gamma_\sigma; O; G; R \rangle \vdash \text{seed}((x : \tau), \Delta) \dashv \mathcal{S}'}\ \text{B-Seed-Pure}$$

$$\frac{\Omega \vdash \tau \rightsquigarrow \hat{\tau} \qquad \Omega \vdash S \rightsquigarrow \hat{S} \qquad \sigma, \ell \text{ fresh} \qquad \langle \Omega[x \mapsto \text{borrow}_m \ell\ \sigma]; (\Gamma_\sigma, \sigma : \hat{\tau}); \big(O, \big(x, \ell, \hat{S}[s := \sigma]\big)\big); G; R \rangle \vdash \text{seed}(\Delta) \dashv \mathcal{S}'}{\langle \Omega; \Gamma_\sigma; O; G; R \rangle \vdash \text{seed}((x : \text{\&mut } (s : \tau \hookrightarrow S)), \Delta) \dashv \mathcal{S}'}\ \text{B-Seed-Borrow}$$

Seeding is the callee's half of the telescope discipline. A pure parameter becomes a fresh symbolic value typed in Γ_σ; a borrow parameter becomes a fresh argument borrow $\text{borrow}_m \ell\ \sigma$ over a fresh snapshot, plus an *obligation* $\big(x, \ell, \hat{S}[s := \sigma]\big)$ — the owed type with the $\hookrightarrow$ binder instantiated at the entry snapshot, the one position that must outlive its moment (doc §5.2). Two deliberate absences. There is *no* owner entry for ℓ: the caller holds that loan, so nothing in the body can collapse the argument borrow by owner-demand — only the audit judges it, which is what makes the audit the single point where judging is possible. And there is *no cross-parameter substitution*: a later parameter's type mentions an earlier parameter by its runtime *variable* (x, resolved to the snapshot by $\rightsquigarrow$ each time the type is consulted — for a borrow parameter, $* x$ peels to the payload snapshot). A pure binder substituted at seed time would sit unsubstituted under whnf, a rigid neutral, and read as rigid at a refl-match that should have been flex. B-Pin fixes the return type at entry, while the parameters it may mention are still live: a dependent return type over a consumed parameter means its *entry* value (re-reading at return would find $\bot$). A return type carrying a borrow anywhere is *not* pinned ($R = \cdot$): it is audited structurally at return (B-Coll-Borrow) rather than reflected.

The pin is not uniform, and the exception is the propositional architecture's central move (doc §5.4, Section 7). Before reflecting T, the rule mints one fresh σ_{exit} per borrow parameter and markExit rewrites T so that a **bare** $* v$ pins to that parameter's σ_{exit} — the **exit** snapshot — while `old` $* v$ pins to the entry σ. So a value return type is entry-pinned in every position except a borrow payload's deref, which reads the end. The exit σ's live **only** in the pin and in an `exitSyms` side-table — never in Γ_σ, never in an obligation — until the audit **defines** them by substituting each borrow's collapsed final payload, which is why the substitution is a dedicated audit-local pass and not a $\leftsquigarrow$: it carries a mutation's result, and $\leftsquigarrow$ is knowledge-only (Section 10.3). Earlier versions of these rules pinned entry wholesale and flagged the difference as an unimplemented decision; it is implemented at this pin.

10.5.2 The call: consume and promise

Caller-side telescope discipline, symmetric to seeding: arguments are $\Rightarrow$-consumed left to right, and each checked actual is bound into the instantiation θ before the remaining parameter types are consulted — so at a call site $\text{Fin}(\text{len} * b)$ means the b just passed. The $\rightsquigarrow$ premises below read a declaration type against θ, Ω: the parameter variables resolve through θ (which shadows any caller slot), everything else through Ω (implementation: `readCWith`).

$$\frac{}{\mathcal{S}\,; \theta \vdash \cdot : \cdot \Rightarrow \cdot \dashv \mathcal{S}\,; \theta}\ \text{B-Inst-Nil}$$

$$\frac{\Omega \vdash a \Rightarrow v \dashv \Omega_1 \qquad \theta, \Omega_1 \vdash \tau \rightsquigarrow \hat{\tau} \qquad v : \hat{\tau} \qquad \mathcal{S}[\Omega_1]\,;\theta[x \mapsto v] \vdash \overline{a} : \Delta \Rightarrow C \dashv \mathcal{S}'\,;\theta'}{\mathcal{S}\,;\theta \vdash a, \overline{a} : (x : \tau), \Delta \Rightarrow C \dashv \mathcal{S}'\,;\theta'}\ \text{B-Inst-Pure}$$

$$\frac{\begin{array}{c}\Omega \vdash a \Rightarrow \text{borrow}_m \ell\ w \dashv \Omega_1 \qquad \theta, \Omega_1 \vdash \tau \rightsquigarrow \hat{\tau} \qquad w : \hat{\tau} \qquad \theta, \Omega_1 \vdash S \rightsquigarrow \hat{S} \\ \mathcal{S}[\Omega_1]\,;\theta[x \mapsto \text{borrow}_m \ell\ w] \vdash \overline{a} : \Delta \Rightarrow C \dashv \mathcal{S}'\,;\theta'\end{array}}{\mathcal{S}\,;\theta \vdash a, \overline{a} : (x : \&\text{mut}\ (s : \tau \hookrightarrow S)), \Delta \Rightarrow \ell_{\left[\text{owed}\ \hat{S}[s:=w]\right]}, C \dashv \mathcal{S}'\,;\theta'}\ \text{B-Inst-Borrow}$$

A pure argument is consumed and checked at its (instantiated) parameter type. A borrow argument must evaluate to a borrow; its payload is checked at τ, and its loan is *captured*, annotated with the owed type instantiated at the payload snapshot just passed ($\hat{S}[s := w]$ — the caller-side mirror of B-Seed-Borrow's $\hat{S}[s := \sigma]$). The parameter is bound in θ to the actual *borrow*, so a later $*x$ in a type peels to the payload (doc §5.2 at the call site); a bare x of borrow type in an index position is outside the unrestricted fragment and unspecified. Left-to-right consumption has a practical corollary (doc §5.3): a later argument expression may not mention a borrow an earlier argument consumed — a bounds proof about $*v$ must be `let`-bound *before* the call that consumes v.

$$\frac{T \text{ neither an \&mut type nor a } \Sigma \qquad \theta, \Omega \vdash T \rightsquigarrow \hat{R} \qquad \sigma \text{ fresh}}{\theta \vdash T \underset{\text{res}}{\Rightarrow} (\sigma\,;\cdot) \dashv \mathcal{S}\left[\Gamma_\sigma, \sigma : \hat{R}\right]}\ \text{B-Res-Val}$$

$$\frac{\theta, \Omega \vdash \tau \rightsquigarrow \hat{\tau} \qquad \theta, \Omega \vdash S \rightsquigarrow \hat{S} \qquad \sigma, \ell_r \text{ fresh}}{\theta \vdash \&\text{mut}\ (s : \tau \hookrightarrow S) \underset{\text{res}}{\Rightarrow} \left(\text{borrow}_m \ell_r\ \sigma\,;\ell_{r\left[\text{owed}\ \hat{S}[s:=\sigma]\right]}\right) \dashv \mathcal{S}[\Gamma_\sigma, \sigma : \hat{\tau}]}\ \text{B-Res-Borrow}$$

$$\frac{\theta \vdash A \underset{\text{res}}{\Rightarrow} (v_1\,;I_1) \dashv \mathcal{S}_1 \qquad \theta \vdash B[x := v_1] \underset{\text{res}}{\Rightarrow} (v_2\,;I_2) \dashv \mathcal{S}_2}{\theta \vdash \Sigma(x : A).B \underset{\text{res}}{\Rightarrow} (\text{Pair}(v_1, v_2)\,;I_1, I_2) \dashv \mathcal{S}_2}\ \text{B-Res-Pair}$$

B-Res-Pair is **dependent**: the tail is built at $B[x := v_1]$, the head's freshly-minted component substituted in, one binder at a time from the outside so that each substitution shifts the remaining binders down as it goes. Earlier versions of this rule built the two components independently and carried a rule-gap note saying so — harmless while no caller ever **consumed** a pinned component, and immediately fatal once a callee's only description is a Σ-pinned postcondition, since the tail's type reached the caller with a dangling binder.

$$\frac{\Phi(f) = \text{fn}\ f(\Delta) \to T \qquad \mathcal{S}\,;\cdot \vdash \overline{a} : \Delta \Rightarrow C \dashv \mathcal{S}_1\,;\theta \qquad \theta \vdash T \underset{\text{res}}{\Rightarrow} (v\,;I) \dashv \mathcal{S}_2 \qquad \rho \text{ fresh} \\ \left(\theta, \Omega_2 \vdash b \rightsquigarrow \hat{b}\ \text{ iff }\ \Phi(f).\text{back} = b\right)}{\mathcal{S} \vdash f(\overline{a}) \Rightarrow v \dashv \mathcal{S}_2\left[G, A(\rho)\left\{C; I; \text{back}\,\hat{b}\right\}\right]}\ \text{B-Call}$$

$$\frac{\begin{array}{c}\mathcal{S}.\text{selfRec} = (f, (k, c)) \qquad a_k = \text{the actual at the declared decreasing position} \\ a_k \sqsubset c \quad \text{(strict structural subterm)} \qquad \mathcal{S} \vdash f(\overline{a}) \Rightarrow v \dashv \mathcal{S}' \quad \text{(B-Call)}\end{array}}{\mathcal{S} \vdash f(\overline{a}) \Rightarrow v \dashv \mathcal{S}'}\ \text{B-Call-Self}$$

$$\frac{\mathcal{S}.\text{selfRec} = (g, \cdot) \quad g \neq f \qquad f \twoheadrightarrow g \text{ in the call graph of } \Phi}{\text{no rule}}\ \text{B-Call-Mutual}$$

A call is checked against the signature alone — recursion forces this, and all calls get it uniformly. That uniformity has one consequence a **self**-call must pay for. Admitting `f`'s own call at `f`'s declared return type, with nothing else required, is Hoare's recursion rule without its side condition, and under a return type that carries real content every false postcondition proves itself (Section 8). B-Call-Self is that side condition. The declaration names a decreasing position `[k]` (doc §1.2); the checker holds that parameter's **current snapshot** c — seeded at entry and thereafter carried by $\rightsquigarrow$ along with every other σ-bearing component, so inside `match n { S(m) => … }` it reads `S` σ_m — and the call is admitted only if the actual at k is a strict structural subterm of it. Because the checker is a symbolic interpreter, that test is ordinary structural comparison; no measure and no termination checker appears anywhere. A borrow parameter decreases through its **payload** snapshot, which is the only thing that shrinks in a list cursor. B-Call-

Mutual is the absence of a rule: the guard is per-declaration, so `f` → … → `f` through another declaration would let each admit the other's postcondition with nothing decreasing anywhere, and reachability in the call graph rejects it outright rather than approximating it. `[k]` is **declared** and not inferred, for two reasons that are both fatal to inference: "some argument decreases at each call" is unsound, since two branches can each shrink a different coordinate while growing the other forever; and paths are explored independently, so no single inferred index could be committed to across them. B-Res builds the fresh result from the instantiated return type: a non-borrow leaf is a fresh existential at the return type (the doc §5.3 *wire*); each &mut position mints a fresh *issued* reborrow over a fresh snapshot, owed-annotated from the return type's own $\hookrightarrow$; a Σ of borrows issues one loan per position (`nth2`, the multi-issued group — this calculus's `split_at_mut`). B-Call mints one loan group $A(\rho)\{C; I\}$ tying the captured loans to the issued ones; when the callee declares a backward spec, the group carries it, instantiated at the actuals ($\hat{b}$). The group's *ending* discipline belongs to F2 (Section 10.2), cited here rather than restated: each issued borrow surrenders first (G-EndIssued — located anywhere in Ω, audited against its owed type, killed), then the group ends atomically. G-EndGroupOpaque releases each captured loan with a fresh existential at its owed type — "the promise is collected", and how much the caller learns is exactly how much the signature says; the doc §5.3 wire is its $I = \cdot$ instance, G-EndOwed. G-EndGroupBack consumes exactly what this figure supplies: the $\hat{b}$ minted at B-Call and audited against the callee's body at B-BackN/B-Back0 below, applied to the surrendered payloads in issued order — the captured loan releases the *computed* value, no existential minted. Sound because declared and checked; the once-tried signature-*inferred* version was unsound (`through` and `advance` share a signature) and is gone. Two boundary-relevant provisos are F2′s ledger, not re-derived here: G-EndGroupBack exists only for a single captured loan (a multi-captured declared back silently degrades to the opaque end), and surrender performs *no* collapse — the collapse step lives exclusively in this figure's audit, next.

10.5.3 The audit at return

The callee's side, per explored path: the result value v against the return type, every argument-borrow obligation discharged. The audit is itself a demand, and it *collapses first*: the $\mathcal{S} \overset{*}{\rightsquigarrow} \mathcal{S}'$ premise in the two B-Audit rules lets F2′s reorganizations End-Mut the payloads' field loans at the boundary — doc §3.3′s suspension ending here rather than at an owner, an argument borrow having none — before conversion judges the collapsed payloads. Rejection is rule-absence: an orphaned marker (its borrow missing) can be ended by no G-rule, a hole survives every reorganization and $\bot$ satisfies no type, and an obligation that is neither locatable nor continued matches no B-Oblig/B-Exempt rule — the implementation errors distinctively in each case ("take without refill", "it was lost").

First, collecting the result's borrow positions against the return type ($B = \cdot$ is a value-returning body; a borrow-returning type whose result is not a borrow admits no rule):

$$\frac{\Omega \vdash S \rightsquigarrow \hat{S}}{\text{\&mut } (s : \tau \hookrightarrow S) \rhd \text{borrow}_m \ell\ p \Rightarrow \left(\ell, p, \hat{S}[s := p]\right)}\ \text{B-Coll-Borrow}$$

$$\frac{T \text{ neither an \&mut type nor a } \Sigma}{T \rhd v \Rightarrow \cdot}\ \text{B-Coll-Val}$$

$$\frac{A \rhd v_1 \Rightarrow B_1 \qquad B \rhd v_2 \Rightarrow B_2}{\Sigma(x : A).B \rhd \text{Pair}(v_1, v_2) \Rightarrow B_1, B_2}\ \text{B-Coll-Pair}$$

Reachability — is this loan connected to that one, through reborrow chains (loan markers parked in a borrow's payload) or through a group (captured in, issued out)? "Consumed into the result" is *transitive*: a recursive cursor's outer result is the inner call's issued borrow, reached from the argument through the group link.

$$\frac{}{\ell \twoheadrightarrow \ell}\ \text{B-Reach-Refl}$$

$$\frac{\text{borrow}_m \ell\ p \in \Omega \qquad \text{loan}_m \ell_c \in p \qquad \ell_c \twoheadrightarrow \ell'}{\ell \twoheadrightarrow \ell'}\ \text{B-Reach-Chain}$$

$$\frac{A(\rho)\{C; I\} \in G \qquad \ell \in C \qquad \ell_i \in I \qquad \ell_i \twoheadrightarrow \ell'}{\ell \twoheadrightarrow \ell'}\ \text{B-Reach-Group}$$

One obligation, given the issued loans I: exempt if consumed into the result (directly, or as the captured owner of a field reborrow that became the result — B-Reach-Refl folds the direct case in), exempt if consumed onward into another call (its story continues through *that* call's group); otherwise it must be locatable — as a live borrow *anywhere* in Ω, it may have been moved into a local value — with a marker-free, hole-free payload of its owed type.

$$\frac{\ell \twoheadrightarrow \ell' \qquad \ell' \in I}{\mathcal{S}\,; I \vDash (x, \ell, S)}\ \text{B-Exempt-Result}$$

$$\frac{A(\rho)\{C; I'\} \in G \qquad \ell \in C}{\mathcal{S}\,; I \vDash (x, \ell, S)}\ \text{B-Exempt-Call}$$

$$\frac{\text{borrow}_m \ell\ p \in \Omega \qquad \text{no loan}_m \cdot \text{ and no } \bot \text{ in } p \qquad p : S}{\mathcal{S}\,; I \vDash (x, \ell, S)}\ \text{B-Oblig-Conv}$$

The audit rules themselves. A dead branch is admitted first: a result that is an eliminator applied to a verified inhabitant of $\bot$ is unreachable, and the audit admits it at *any* return type — the $\bot$-witness is the only thing checked, and no obligation is audited (this is how a bounds-proof cursor's `Nil` branch "returns" a borrow it cannot have: it doesn't, and needn't; the `botElim` motive need not be the unreflectable borrow return type).

$$\frac{v = \text{botElim}\ \ T'\ w \qquad w : \text{Bot}}{\mathcal{S} \vDash \text{audit}(v)}\ \text{B-ExFalso}$$

$$\frac{\mathcal{S} \overset{*}{\rightsquigarrow} \mathcal{S}' \qquad T \rhd v \Rightarrow \cdot \qquad \forall\ \text{ob} \in O' :\ \mathcal{S}'\,; \cdot \vDash \text{ob} \qquad \mathcal{S}' \vDash \text{back}_{\cdot}(d) \qquad v : \hat{R}}{\mathcal{S} \vDash \text{audit}(v)}\ \text{B-Audit-Val}$$

$$\frac{\mathcal{S} \overset{*}{\rightsquigarrow} \mathcal{S}' \qquad T \rhd v \Rightarrow B \qquad B \neq \cdot \qquad I = \text{loans}(B) \qquad \forall\ \text{ob} \in O' :\ \mathcal{S}'\,; I \vDash \text{ob} \qquad \forall (\ell, p, S) \in B :\ p : S \qquad \mathcal{S}' \vDash \text{back}_I(d)}{\mathcal{S} \vDash \text{audit}(v)}\ \text{B-Audit-Borrow}$$

B-Audit-Val is doc §5.4′s audit exactly: obligations, then the result against the entry-pinned return type $\hat{R}$. B-Audit-Borrow is its doc §6.1 reshaping for a borrow-returning path: the issued borrows' payloads are audited against the return type's own owed types, obligations are audited with the exemptions in force, and no separate result check remains — being issued *is* the result's typing, its story continuing caller-side through the group. The back premise is vacuous for a spec-less declaration (B-Back-None below); for a spec-carrying one it is the doc §6.2 callee check.

10.5.4 Declared backward specs: the callee check

The doc §6.2 callee side. A declared back is checked against the *suspension tree* the body actually built: the captured borrow's payload with the issued loans' markers read as holes $\%\text{r}_\text{i}$ (pure de Bruijn, in issued order), resolved down reborrow chains and *through sub-calls' declared backs* — the tree *is* the backward function the body implements, and LLBC's backward functions compose exactly here. Resolution (first matching clause applies; $\hat{f}$ is a sub-group's instantiated spec):

$$
\begin{aligned}
\mathrm{res}_I(\mathrm{loan}_m\,\ell) &= \%\mathrm{r}_i && \text{if } \ell = I_i \\
&= \mathrm{nf}\big(\hat{f}\ \mathrm{res}_I(\mathrm{loan}_m\ell'_1)\ \ldots\ \mathrm{res}_I(\mathrm{loan}_m\ell'_m)\big) && \text{if } \ell \in C \text{ for some } A(\rho)\big\{C; (\ell'_1 \ldots \ell'_m); \mathrm{back}\,\hat{f}\big\} \in G \\
&= \mathrm{res}_I(p) && \text{if } \mathrm{borrow}_m\ell\ p \in \Omega \\
&= \mathrm{loan}_m\,\ell && \text{otherwise} \\
\mathrm{res}_I(\mathrm{borrow}_m\ell\ p) &= \%\mathrm{r}_i && \text{if } \ell = I_i \\
&= \mathrm{borrow}_m\ell\ \mathrm{res}_I(p) && \text{otherwise} \\
\mathrm{res}_I(c(\overline{v})) &= c(\mathrm{res}_I(\overline{v})) \qquad \mathrm{res}_I(v) = v \text{ otherwise}
\end{aligned}
$$

The second clause is composition: a loan captured by a sub-call's group resolves to *that* call's declared back applied to its (recursively resolved) issued borrows. Composition is all-or-nothing — a spec-less sub-group falls to the fourth clause, an unresolvable marker no spec ever converts against, so a spec-carrying function's whole call tree must be spec'd. Recursion is fine: a recursive call resolves through the function's own declaration.

$$
\frac{d.\mathrm{back} = \cdot}{\mathcal{S} \vDash \mathrm{back}_I(d)}\ \text{B-Back-None}
$$

$$
\frac{d.\mathrm{back} = b \qquad (x, \ell_c, S) \in O \qquad \ell_c \twoheadrightarrow \ell' \qquad \ell' \in I \qquad \mathrm{raw} = \begin{cases} p & \text{if } \mathrm{borrow}_m\ell_c\ p \in \Omega \\ \mathrm{loan}_m\ell_c & \text{else} \end{cases} \qquad \mathrm{res}_I(\mathrm{raw}) \equiv \mathrm{nf}\big(\hat{b}\ \%\mathrm{r}_0\ \ldots\ \%\mathrm{r}_{k-1}\big)}{\mathcal{S} \vDash \mathrm{back}_I(d)}\ \text{B-BackN}
$$

$$
\frac{d.\mathrm{back} = b \qquad (x, \ell, S) \in O \qquad \mathrm{borrow}_m\ell\ p \in \Omega \qquad \mathrm{res}_\cdot(p) \equiv \mathrm{nf}\big(\hat{b}\big)}{\mathcal{S} \vDash \mathrm{back}_\cdot(d)}\ \text{B-Back0}
$$

B-BackN ($k = |I| > 0$) checks a borrow-returning body: the captured borrow is the obligation consumed into the result (directly — its loan *is* a result loan, `through` — or as the owner of a field reborrow that became the result, `advance`/`nth2`); its payload, or the hole itself if it was returned directly, resolves to the tree, which must convert with the spec applied to the holes. B-Back0 is the value-returning dual: an in-place mutator issues no borrows, so the spec is applied to *no* holes and checked against the argument borrow's resolved tree directly. Until the dual was implemented a value-returning body's back went unchecked at its own audit (a lying `partScanL` variant passed; the borrow-returning branch was the only one that looked) — the lesson generalizes: a negative test per rule *branch*, not per feature. The dual's reach is precise, and stated as a proviso rather than a premise: it checks a back authored *as the raw tree* — `partScanL`, being literally the composition of its sub-calls' declared backs, converts — while a back that *reformulates* the tree (`swapS`'s swapL $ij * v$ against the set-based composition it actually implements: semantically equal, never definitionally so) is outside conversion's reach and is the differential's to validate. Complementary, not redundant. The principled close, deferred: admit a reformulated back with a *cited bridging equation* — the audit doing rewrite-by-Id along a proved $\mathrm{Id}(\mathrm{swapL}\ ijl, \mathrm{set}\ i\ldots(\mathrm{set}\ j\ldots l))$ — so reformulation becomes checked rather than trusted.

10.5.5 The top rule

$$
\frac{d = \mathrm{fn}\ f(\Delta) \to T \in \Phi \qquad \mathrm{init}(\Phi) \vdash \mathrm{seed}(\Delta) \dashv \mathcal{S}_0 \qquad \mathcal{S}_0 \vdash \mathrm{pin}(T) \dashv \mathcal{S}_1\ (\text{else } \mathcal{S}_1 = \mathcal{S}_0) \qquad \big(\Omega_0 \vdash b \rightsquigarrow \hat{b}\ \text{ iff }\ d.\mathrm{back} = b\big) \qquad \forall\ \mathcal{S}_1 \vdash \mathrm{body}(d) \Rightarrow v \dashv \mathcal{S}' :\ \ \mathcal{S}' \vDash \mathrm{audit}(v)}{\vdash \mathrm{fn}\ d\ \mathrm{ok}}\ \text{B-CheckFn}
$$

Seed the telescope, pin the return type while the parameters are live (B-Pin, skipped for a borrow-carrying T), reflect the declaration's own back over the entry snapshots, then *quantify over every $\Rightarrow$-derivation of the body* and audit each. The quantification is where nondeterminism earns its keep: F4′s symbolic match makes $\Rightarrow$ a genuine relation — one derivation per constructor of a symbolic scrutinee's type, exhaustiveness enforced at the fork — so "explore all paths, audit each" is a single $\forall$ over derivations. Each derivation carries its own state: the obligations, pinned return type, and reflected back it audits against are the ones *that path* refined. Throughout the audit rules, T, d, and O are the checked declaration's,

ambient; Φ contains d itself, so a recursive call resolves — against the signature, never the body — without unrolling. The implementation determinizes exactly this figure: a continuation-pushing pre-pass makes every match terminal, `explore` enumerates the derivations, and the lazy collapse (`collapseArg`, firing End-Mut innermost-first when the audit demands a marker-free payload) is one scheduling of the $\overset{*}{\rightsquigarrow}$ premise.

10.5.6 Correspondence

Rule	Implementation	Test
B-Seed-Nil/Pure/Borrow	`Boundary.seedTelescope`	`S5Bound`
B-Pin	`Boundary.checkFn` (entry pin), `hasBorrowT`	`S12Inst`
B-Inst-Nil/Pure/Borrow	`Machine.processArgs`, `readCWith`	`S6Call`, `S12Inst`
B-Res-Val/Borrow/Pair	`Machine.buildResult`	`S7Group` (`nth2`)
B-Call	`Machine.readR` (`.call`, checking mode)	`S6Call`
B-Call-Self	`Machine.strictSubterm`, `St.selfRec`	`S23Direct`
B-Call-Mutual	`Machine.reachesFn`	`S23Direct`
B-Coll-Borrow/Pair/Val	`Boundary.collectResultBorrows`	`S7Group`
B-Reach-Refl/Chain/Group	`Boundary.reachesLoan`	`S14Bounds` (`advance`)
B-Exempt-Result/Call, B-Oblig-Conv	`Boundary.auditObligation`, `collapseArg`	`S7Group` (`choose`), `S5Bound`
B-ExFalso	`Boundary.auditAction` (`botElim` spine)	`S14Bounds`
B-Audit-Val/Borrow	`Boundary.auditAction`	`S5Bound`, `S7Group`
B-Back-None/Back0	`Boundary.auditAction` (value dual)	`S19Partition` (`partScan`)
B-BackN, res	`Boundary.auditAction`, `resolveTree`	`S17Spec`
B-CheckFn	`Boundary.checkFn`, `Machine.explore`	`S5Bound`, `SDeclUnified`

10.6 F6: Value typing and conversion

The comptime fragment (Section 10.3) is a small dependent type theory, and this figure is its judgment of **inhabitation**: when does a value v inhabit a type τ, written $v : \tau$? The check is the mechanism behind the money test of the introduction (doc §4.2) — inhabitation is tested against a type that must first **compute** to canonical form, and a stuck type (an index that never reduced, say Vec $T\ \sigma_l$ with σ_l unknown) exposes no constructor, so conversion has nothing to say yes to and the audit rejects. Two judgments do the work: value typing $v : \tau$ (`hasType`) and definitional conversion $\tau \equiv \tau'$ (`convert`). They are mutually entangled — every typing leaf ends in a conversion, and `Refl` is well-typed only up to one.

Global conventions. (i) The subject is **weak-head normalized before any rule fires** (`hasType` calls `whnfV` on v first): a β-redex or a stuck recursor spine reduces to its weak head, so the rules below read on canonical subjects. (ii) The rules are stated **nondeterministically and fuel-free**; conversion is factored out into a single coercion rule T-Conv, and the implementation's fusion of a convert-check into each synthesis leaf is one deterministic reading of it. (iii) There is **no context turnstile** in the judgment: the σ-context Γ_σ (each symbolic value's type, `St.sctx`) is ambient checker state, consulted where a rule names it.

> **Algorithm note (stated once).** `convert` decides $\tau \equiv \tau'$ by normalizing both sides to normal form (β and ι, de Bruijn, **no η**) under an explicit fuel and comparing syntactically ($\tau \equiv \tau' := \mathrm{nfV}(\tau) = \mathrm{nfV}(\tau')$). Because v0 collapses the universe hierarchy to type-in-type (doc §1.1), strong normalization fails (Girard's paradox lives there), so conversion — and hence typing — is a **fuel-bounded partial**

decision procedure, correct on the normalizing fragment where all v0 programs live. The fuel is the algorithm's, not the relation's; the rules carry none.

10.6.1 The conversion judgment $\tau \equiv \tau'$

Definitional equality is the least congruence containing β and ι, closed under the equivalence laws. Presented as axioms (the normal-form equality `convert` computes is exactly this relation for a confluent, normalizing fragment):

$$\frac{}{\tau \equiv \tau}\ \text{C}{\equiv}\text{-Refl} \qquad \frac{\tau \equiv \tau'}{\tau' \equiv \tau}\ \text{C}{\equiv}\text{-Sym} \qquad \frac{\tau \equiv \tau' \quad \tau' \equiv \tau''}{\tau \equiv \tau''}\ \text{C}{\equiv}\text{-Trans}$$

$$\frac{\tau_i \equiv \tau_i'}{F(\cdots \tau_i \cdots) \equiv F(\cdots \tau_i' \cdots)}\ \text{C}{\equiv}\text{-Cong}$$

C≡-Cong closes conversion under every term former $F \in \{\text{app}, \text{ctor}, \Pi, \Sigma, \lambda, \text{Id}\}$, componentwise — normal-form equality is a congruence by construction.

The computation rules, β and the ι-rules for the recursor basis (`whnfV` in `Pure.lean`), as directed equations read as conversions:

(C≡-Beta) $(\lambda(x : \tau).t)\ u \equiv t[x := u]$

(C≡-IotaNat) $\text{natRec}\ P\ z\ s\ \ \text{Z} \equiv z \qquad \text{natRec}\ P\ z\ s\ (\text{S}\ \ m) \equiv s\ m\ (\text{natRec}\ P\ z\ s\ m)$

(C≡-IotaBool) $\text{boolRec}\ P\ t\ f\ \ \text{True} \equiv t \qquad \text{boolRec}\ P\ t\ f\ \ \text{False} \equiv f$

(C≡-IotaSigma) $\text{sigmaRec}\ A\ B\ P\ f\ (\text{Pair}\ \ a\ b) \equiv f\ a\ b$

(C≡-IotaList) $\text{listRec}\ AP\ p_n\ p_c\ \ \text{Nil} \equiv p_n \qquad \text{listRec}\ AP\ p_n\ p_c\ (\text{Cons}\ \ h\ t) \equiv p_c\ h\ t\ (\text{listRec}\ AP\ p_n\ p_c\ t)$

(C≡-IotaJ) $\text{j}\ A\ a\ P\ d\ b\ \ \text{Refl} \equiv d$ (C≡-IotaK) $\text{k}\ A\ a\ P\ d\ \ \text{Refl} \equiv d$

A recursor whose target normalizes to a **neutral** (a `sym` or a bound variable, never a constructor) does not reduce: the spine is a stuck value and convertible only to itself and its congruent variants — this is what makes `add σ 1` and `add σ 2` inconvertible (S4Pure). `botElim` has **no** ι-rule: $\bot$ has no constructor, so `botElim T x` is always stuck.

Deliberately absent: η. There is no rule $\lambda(x : \tau).f\ x \equiv f$. Two λ's with the same domain differing only in their bodies are **not** convertible (S4Pure: $\lambda(u : \text{Nat}).u \not\equiv \lambda(u : \text{Nat}).\ \text{Z}$ is `false`). `convert` compares normal forms structurally; η would require comparing under an applied/abstracted context, which it does not do.

10.6.2 The value-typing judgment $v : \tau$

10.6.2.1 Universe and type formers

$$\frac{\tau \equiv \text{Type}}{\text{Type} : \tau}\ \text{T-Type} \qquad \frac{F \in \{\Pi(x : A) \to B,\ \Sigma(x : A).B,\ \ \text{Id}\ A\ a\ b\} \quad \tau \equiv \text{Type}}{F : \tau}\ \text{T-Former}$$

Type-in-type: $\text{Type} : \text{Type}$, and every type former inhabits Type (doc §1.1; the hierarchy returns with doc §9). Both are checking rules — the target τ need only **convert** to Type, which is where T-Conv is folded in.

10.6.2.2 Symbolic values

$$\frac{\Gamma_\sigma(\sigma) = \tau_\sigma}{\sigma : \tau_\sigma}\ \text{T-Sym}$$

A symbolic value synthesizes the type the σ-context records for it; other types follow by T-Conv. (In code, the two are fused: the `.sym` case returns $\tau_\sigma \equiv \tau$ directly.) A σ absent from Γ_σ is a checker-internal error, not a `false`.

10.6.2.3 Constructors

$$\frac{\text{ctorSig}(c,\ \text{whnf}\ \tau) = [\tau_1, ..., \tau_n] \quad v_1 : \tau_1 \quad v_2 : \tau_2[x_1 := v_1] \quad ... \quad v_n : \tau_{n[x_1:=v_1,...,x_{n-1}:=v_{n-1}]}}{c(v_1, ..., v_n) : \tau}\ \text{T-Ctor}$$

The expected type is **whnf'd first**, then matched against the signature table below to recover the constructor's field-type telescope; a table miss (the constructor does not inhabit this type former, or the type is stuck) yields `false`, not an error — this **is** the money-test rejection. Fields are checked left to right, each subsequent field type instantiated at the **values** of the earlier fields (`checkFields` threads `substPure 0 vᵢ`): this is exactly how `Pair`'s second field is typed against the first field's value — dependent Σ.

constructor	expected type (whnf)	field telescope
`unit`	Unit	[]
`True` / `False`	Bool	[]
`Z`	Nat	[]
`S`	Nat	[Nat]
`Nil`	List T	[]
`Cons`	List T	[T, List T]
`Pair`	$\Sigma(x : A).B$	[A, B] (2nd dependent on 1st)
`Refl`	Id A a b	[] if $a \equiv b$, else miss

`Refl` is the one signature entry carrying a **side condition**: it inhabits Id A a b only when the endpoints convert ($a \equiv b$), which is how `Id`'s reflexive constructor witnesses definitional equality. The table has no `inductive`-declaration machinery behind it — it is a fixed basis (Unit, Bool, Nat, List, Σ, Id), the v0 kernel's constructor set (doc §7.1).

10.6.2.4 Functions against a Π

$$\frac{\text{whnf}\ \tau = \Pi(x : A').C \quad A \equiv A' \quad \Gamma_\sigma,\ \sigma : A' \ \vdash\ b[x := \sigma] : C[x := \sigma]}{\lambda(x : A).b : \tau}\ \text{T-Lam}$$

A λ is checked against a Π: the declared domain must **convert** to the Π's domain, then the body is checked under a **fresh symbolic witness** $\sigma : A'$ added to Γ_σ — the binder opened with a hypothesis of the Π's domain type. This is what types a Π-typed lemma (`id_sym`, S16Spec) and the recursors' λ-shaped step arguments (`s`, `pc`). It is the elaboration of dependent elimination (doc §9/doc §10), with no arrow of its own.

10.6.2.5 Eliminator-neutral synthesis

A neutral application spine `head args` synthesizes a type only when `head` is one of the eliminator constants. The fixed prefix is typed to a **base result** — the motive P applied to the target(s) — its premises checked; any **extra** arguments (an over-applied eliminator, `natRec … n` at a function motive then given its argument) are then absorbed by T-Spine's spine synthesis. One schema, instantiated per eliminator by the table:

$$\frac{\text{premises}(e, \bar{m}, \bar{t}) \quad \text{base} = P(\bar{t}) \quad \text{per the table} \quad \text{synthSpine}(\text{base}, \bar{\text{rest}}) = \tau'' \quad \tau' \equiv \tau''}{e\ \bar{p}\ \bar{m}\ \bar{t}\ \bar{\text{rest}} : \tau'}\ \text{T-ElimSynth}$$

applied form	premises	base result
natRec P z s n	$z : P$ Z; $s : \Pi(m : \text{Nat}) \to P\ m \to P(\text{S}\ m)$; n : Nat	P n
boolRec P t f b	$t : P$ True; $f : P$ False; b : Bool	P b

applied form	premises	base result
listRec $A\ P\ p_n\ p_c\ l$	$p_n : P\ \text{Nil}$; $p_c : \Pi(h : A) \to \Pi(t : \text{List}\ A) \to P\ t \to P(\text{Cons}\ h\ t)$; $l : \text{List}\ A$	$P\ l$
sigmaRec $A\ B\ P\ f\ s$	$f : \Pi(x : A) \to \Pi(y : B\ x) \to P(\text{Pair}\ x\ y)$; $s : \Sigma(x : A).B\ x$	$P\ s$
botElim $T\ x$	$x : \text{Bot}$	T

Σ's parameters are a type A and a **family** B, so — unlike `List`'s uniform parameter — both of `sigmaRec`'s premises cross a binder and B and P are shifted to be read under it. The eliminator is non-recursive, hence the single arm with no inductive hypothesis. Its absence was the basis's one missing recursor-performer: a Σ could be **built** and never **projected**, which is survivable while conjunctions are only ever assembled and fatal once a caller's whole knowledge of a callee is a returned certificate it must take apart (Section 7).

The two `Id`-eliminators, written explicitly (Paulin-Mohring J and Streicher K, doc §10):

$$\frac{d : P\ a \quad \text{Refl} \qquad p : \text{Id}\ A\ a\ b}{\text{j}\ A\ a\ P\ d\ b\ p : P\ b\ p}\ \text{T-Elim-J} \qquad \frac{d : P \quad \text{Refl} \qquad p : \text{Id}\ A\ a\ a}{\text{k}\ A\ a\ P\ d\ p : P\ p}\ \text{T-Elim-K}$$

These are what let **library** fording terms type-check as ordinary values — `natNoConf` (a `j`-spine landing in a reducing `NatCode`, hence in $\bot$), constructor injectivity (`injProof` via `j`), UIP (`uipProof` via `k`) — with no machine rule beyond the eliminators' own typing (S10Ford).

10.6.2.6 Application spines for bound function variables

When the neutral head is a σ (a bound function variable applied — `ih b c hab hbc`), its type is looked up and the spine is synthesized by iterated Π-instantiation:

$$\frac{\Gamma_\sigma(\sigma) = \tau_\sigma \qquad \text{synthSpine}(\tau_\sigma, [a_1, ..., a_n]) = \tau_{\text{res}}}{\sigma\ a_1\ ...\ a_n : \tau_{\text{res}}}\ \text{T-Spine} \qquad \frac{\text{whnf}\ \tau = \Pi(x : A).B \qquad a : A \qquad \text{synthSpine}(B[x := a], \bar{a}) = \tau_{\text{res}}}{\text{synthSpine}(\tau, a :: \bar{a}) = \tau_{\text{res}}}\ \text{Spine-}\Pi$$

`synthSpine` on the empty argument list is the identity ($\text{synthSpine}(\tau, []) = \tau$); each argument checks against the current domain and instantiates the codomain at the argument's **value**. It is shared with T-ElimSynth (the over-application tail), so ordinary application typing and eliminator over-application are one mechanism.

10.6.2.7 Conversion coercion

$$\frac{v : \tau \qquad \tau \equiv \tau'}{v : \tau'}\ \text{T-Conv}$$

The single coercion rule: a value keeps its type up to definitional equality. Nondeterministic here; in code it is **not** a standalone rule but is fused into every synthesis leaf (`.sym`, `.type`, formers, and each `finish`/`synthSpine` result all end in `Val.convert fuel _ ty`). There is **no subsumption and no subtyping** — the calculus has conversion only, never a $\tau \leq \tau'$.

10.6.3 What no rule does

No rule types a hole. There is no $\bot : \tau$: $\bot$ is the absence of a value, inhabiting nothing (doc §2.1). `hasType` never receives a $\bot$ that a rule accepts, and the comptime read that would produce one is rejected upstream (`reflectC` on a moved slot). This is the same fact the boundary audit leans on — a function cannot return a hole.

Ex-falso admission lives in F5, not here. T-Elim above types `botElim T x` **when `x` is a genuine $\bot$-inhabitant** — an ordinary eliminator application. The **audit-level** rule that admits a dead branch at **any** return type on the strength of a $\bot$-witness (a cursor's `Nil` branch "returning" a borrow it cannot have) is a boundary rule, `B-ExFalso` in Section 10.5 — excluded from this figure by design. The distinction: here the $\bot$-witness types a **term**; there it discharges a **return obligation**.

Rule	Implementation	Test
T-Ctor	`hasType .ctor`, `checkFields`, `ctorSig`	S4Pure (dependent `Pair`), S5Bound `vecPush`
T-Sym	`hasType .sym`, `St.sctx`	S10Ford `learn`
T-Lam	`hasType .lam` (fresh-σ body)	S16Spec `id_sym`
T-ElimSynth	`hasType .app`, `finish`, `synthSpine`	S10Ford `j`/`k`/`botElim`/`natNoConf`
T-Spine	`hasType .sym`-applied, `synthSpine`	S10Ford `injProof`/`uipProof`; S16Spec
T-Type / T-Former	`hasType .type`/`.pi`/`.sigmaT`/`.idT`	S4Pure
T-Conv	`Val.convert` at each leaf	S5Bound money test; S4Pure `VecF`
C≡-Beta / -Iota	`convert`, `nfV`, `whnfV`	S4Pure (stuck neutrals, `VecF`); S10Ford `natCode`
C≡ (no η)	`convert` = nf-equality	S4Pure (two λ's)

10.7 F7: The carve

A **carve** splits an array node into segments so that a sub-range becomes a subterm. It is a reorganization in Section 10.2′s sense — lazy, fired by a demand, rewriting Ω and nothing else — with one difference that is the array era's whole point: **it is the only rule in the calculus that consumes evidence.** Section 10.2 can say that the borrow machinery has no rules of its own beyond bookkeeping because every rule there is bookkeeping; this one is not, and that is why it is a separate figure rather than a seventh reorganization.

What it buys is stated most precisely by comparison. Rust's `get_disjoint_mut` takes several disjoint mutable subslices at once and checks their disjointness **at run time**, returning a result that can fail. The carve moves that check to compile time and deletes the failure mode — and afterwards there is no disjointness judgment in the system at all, because the two sub-borrows are **different subterms of one value tree** and Section 10.4′s suspension machinery already governs those.

10.7.1 The extent map

A carved array value is a **segment list**, written Arr $\langle c_1 \rhd b_1, ..., c_k \rhd b_k \rangle$: k bodies with their extents, in order. Its **extent map** is the derived list of **leaves** — each a base, a count, and a body — obtained by summing the extents left to right. An uncarved array carries no wrapper at all: a single-segment value and a bare value are the same state, so the wrapper appears only when $k \geq 2$. Two consequences the rules below rely on: a **merge** (adjacent runs concatenated) is run first, so that Section 10.7′s premise (1) sees maximal leaves; and **rejoin is merge at the moment a payload plugs back into its marker**, which every End-Mut path funnels through, so no trigger site can be forgotten because none is listed.

10.7.2 The rule

$$\frac{\begin{array}{l}\Omega(p) \overset{*}{\rightsquigarrow} \text{merge} \quad \text{leaves}(\Omega(p)) \ni (b, m, v) \qquad \text{lo} = \text{add}\ \ b\ \ \text{lo'} \quad \text{own-free}(v) \\ \quad e : \text{Le}\ \ (\text{add}\ \ \text{lo'}\ \ \text{cnt})\ m \qquad m \equiv \text{add}\ \ \text{lo'}\ \ (\text{add}\ \ \text{cnt}\ \ \text{rest}) \\ \quad \Omega' = \Omega[p \mapsto \text{Arr}\ \langle \text{lo'} \rhd v_1, \text{cnt} \rhd v_2, \text{rest} \rhd v_3 \rangle]\end{array}}{\Omega \vdash p\ \mathbf{x}\ [\text{lo}; \text{cnt}] \dashv \Omega'}\ \text{K-Carve}$$

Read the premises in order; each is doing one job.

(1) Containment — a leaf, not an overlap test. Some leaf must contain the request. Because two segments cannot overlap, a range that crosses a segment boundary has no leaf **at all**, whatever its ownership: the rejection is "no leaf contains it", and no owned-versus-loaned comparison appears. A request **contained** in a loaned leaf is a different case and is not rejected — it demand-ends that loan and retries, which is Section 10.2′s rule reaching its sixth site.

(2) The obligation, and whose term discharges it. e is a comptime proof the **program supplies**; there is no inference and no decision procedure. Two spellings are forced, and both matter more than they look. The bound is stated **leaf-relatively** — `Le (add lo' cnt) m` and not `Le (add lo cnt) n`

— because `Le` computes by double recursion and is stuck on a symbolic base, so the global form never converts with anything a program can supply. And a **concrete** count is unrolled into successors, so that the obligation at `a[i]` reads `Le (S i) n`. That is not a convenience: it is character for character the cursor bound the list development has been threading since its two-cursor swap. The design note claimed those were "on the nose, the containment obligation the carve demands"; written the obvious way the obligation would have been `Le (add i (S Z)) n` and the claim would have been false in the one place it was meant to be exact.

(3) The residue transition, and the rule that constrains it. The leaf's extent must decompose as offset-plus-count-plus-remainder. Solved as an ordinary refl-match solution transition (Section 10.4, M-Refl-Flex) — no subtraction is introduced anywhere, which is the decision that keeps the audit's later conversion definitional. `rest` may be **minted** by the checker, or **supplied** by the program as an optional surface slot. Supplying it is what makes a residue nameable downstream, and it often deletes the obligation outright rather than merely making it writable: a pivot carve's `Le 1 rest` is unwritable while `rest` is an unnamed σ, and computes to ⊤ the moment `rest := S j` is supplied.

$$\frac{m \equiv \text{add} \quad \text{lo'} \quad (\text{add} \quad \text{cnt} \quad \text{rest})}{\text{licensed}(m, \text{cnt}, \text{rest})}\ \text{K-Cite}$$

$$\frac{h_{\text{eq}} : \text{Id} \quad \text{Nat} \quad m \ (\text{add} \quad \text{cnt} \quad \text{rest})}{\text{licensed}(m, \text{cnt}, \text{rest})}\ \text{K-Cite-Eq}$$

K-Cite asserts nothing: the decomposition already holds by conversion, and no evidence is demanded. K-Cite-Eq is the other case, and the **only** other case — there is no third rule that refines the extent directly.

Premise (3) may not refine a telescope parameter's σ. This is a soundness rule and it was found, not designed. When the leaf's extent is a telescope parameter's σ — which is the case in every program the supplied-residue form exists for — solving the decomposition by refinement is a refinement of a **universally quantified** length against terms that may be unrelated, and nothing records the induced constraint in the signature. Callers are then never held to it: a body carving `[Z ; i ; S j]` out of `Array n` checked, and so did a caller passing $n = 2$ with $i = j = 5$, which the concrete machine then got stuck on. That is the differential property **failing** rather than over-approximating.

The precedent settles it. A body imposing an unrecorded constraint on its callers is exactly the signature-inferred backward flow of Section 4, whose lesson is that cross-boundary constraints are **declared and checked, never inferred**. So the decomposition must either hold by conversion — K-Cite, the common case, asserting nothing — or be **cited** as an ordinary `Id`, checked by Section 10.6 before the same solution transition runs (K-Cite-Eq). The measurement that says this is not a tax: across the three shipped array declarations there are thirteen carve sites, and **three** need a citation — one each, at the split that the program's own returned equation is about. Every other carve converts and asserts nothing. A decomposition a program cannot prove is one it had no business asserting, and the programs were already returning the equations as honesty decoration before the rule made them load-bearing.

10.7.3 Degenerate cases, and why they are separate

A **whole-leaf** request needs no split and no obligation — `Le m m` computes — and a **zero-width** request must disturb nothing at all. Both skip the ownership test, because between a take and its refill a segment holds a hole and the refill is its one legal successor; testing ownership first rejects the refill. The zero-width case needs its own segment inserted rather than selecting the leaf it abuts, since `Le (add lo Z) (add b m)` holds at a leaf's **far end** — a subtlety that is unreachable symbolically (a residue σ is never **known** to be zero) and routine concretely, and which is one of the four sites of the polarity finding Section 6 reports.

10.7.4 Two restrictions, stated

Rigid extents cannot be sub-carved. The residue transition needs the leaf's extent to be a **flex** σ. A segment whose extent is a constructor tree (`S i`) or a compound neutral (`add p q`) equates two rigid terms and is stuck. This bites wider

than a declared length: it applies to **every** segment's extent, and it is why a three-way split cannot be reached by re-associating a two-way one.

Recursion cannot decrease through a carved payload. Section 10.5′s guard counts constructor fields as subterms and refuses application spines; a carve refines the payload to a concatenation **spine**, so no sub-slice is a structural predecessor of its parent. "Recurse on the sub-slice, no fuel needed" is therefore closed, and the array sort recurses on fuel like its list counterpart. Teaching the guard that a concatenation's arguments are subterms of it would open it, and is filed rather than built. PROPOSED

10.7.5 Correspondence

Rule	Implementation	Test
K-Carve	`carveAt` (premises in order); `extentMap`	`S24Arrays`
premise (1), merge	`Val.mergeArrays`, `segsNode`	`S24Arrays`
premise (2)	`carveAt` obligation formation; `hasType`	`S24Arrays`
K-Cite / K-Cite-Eq	`carveAt` (cited-equation arm)	`S24Arrays` (`threeWayUncited`, `threeWayWrongEq`)
rejoin	`sendPayloadToLoan` → merge	`S24Arrays`
the array library	`StdLemmas` (M24 section)	`S24Arrays`, `S25ArrSort`

Bibliography

[1] G. Klein *et al.*, "seL4: Formal Verification of an OS Kernel," in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, ACM, 2009, pp. 207–220. doi: 10.1145/1629575.1629596.

[2] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, "HACL*: A Verified Modern Cryptographic Library," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2017, pp. 1789–1806. doi: 10.1145/3133956.3134043.

[3] K. Bhargavan *et al.*, "Everest: Towards a Verified, Drop-in Replacement of HTTPS," in *2nd Summit on Advances in Programming Languages (SNAPL)*, 2017. doi: 10.4230/LIPIcs.SNAPL.2017.1.

[4] S. Ho, A. Fromherz, and J. Protzenko, "Sound Borrow-Checking for Rust via Symbolic Semantics," in *Proceedings of the ACM on Programming Languages (ICFP)*, 2024. doi: 10.1145/3674640.

[5] S. Ho and J. Protzenko, "Aeneas: Rust Verification by Functional Translation," in *Proceedings of the ACM on Programming Languages (ICFP)*, 2022. doi: 10.1145/3547647.

[6] D. S. Hutchins, "Pure Subtype Systems," in *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2010, pp. 287–298. doi: 10.1145/1706299.1706334.

[7] K. R. M. Leino, "Dafny: An Automatic Program Verifier for Functional Correctness," in *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, in Lecture Notes in Computer Science, vol. 6355. Springer, 2010, pp. 348–370. doi: 10.1007/978-3-642-17511-4_20.

[8] Y. Matsushita, T. Tsukada, and N. Kobayashi, "RustHorn: CHC-Based Verification for Rust Programs," in *Programming Languages and Systems (ESOP)*, in Lecture Notes in Computer Science, vol. 12075. Springer, 2020. doi: 10.1007/978-3-030-44914-8_18.

[9] Y. Matsushita, X. Denis, J.-H. Jourdan, and D. Dreyer, "RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2022. doi: 10.1145/3519939.3523704.

[10] X. Denis, J.-H. Jourdan, and C. Marché, "Creusot: A Foundry for the Deductive Verification of Rust Programs," in *Formal Methods and Software Engineering (ICFEM)*, Springer, 2022. doi: 10.1007/978-3-031-17244-1_6.

[11] A. Lattuada *et al.*, "Verus: Verifying Rust Programs using Linear Ghost Types," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2023. doi: 10.1145/3586037.

[12] N. Lehmann, A. T. Geller, N. Vazou, and R. Jhala, "Flux: Liquid Types for Rust," in *Proceedings of the ACM on Programming Languages (PLDI)*, 2023. doi: 10.1145/3591283.

[13] N. Vazou, E. L. Seidel, R. Jhala, D. Vytiniotis, and S. P. Jones, "Refinement Types for Haskell," in *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2014. doi: 10.1145/2628136.2628161.

[14] J. Protzenko *et al.*, "Verified Low-Level Programming Embedded in F*," in *Proceedings of the ACM on Programming Languages (ICFP)*, 2017. doi: 10.1145/3110261.

[15] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "RustBelt: Securing the Foundations of the Rust Programming Language," in *Proceedings of the ACM on Programming Languages (POPL)*, 2018. doi: 10.1145/3158154.

[16] H. Xi, "Applied Type System: An Approach to Practical Programming with Theorem-Proving," in *Journal of Functional Programming*, 2017. [Online]. Available: https://www.cs.bu.edu/~hwxi/academic/papers/JFPats.pdf

[17] R. Atkey, "Syntax and Semantics of Quantitative Type Theory," in *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, ACM, 2018. doi: 10.1145/3209108.3209189.

[18] E. Brady, "Idris 2: Quantitative Type Theory in Practice," in *35th European Conference on Object-Oriented Programming (ECOOP)*, in Leibniz International Proceedings in Informatics (LIPIcs), vol. 194. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 9:1–9:26. doi: 10.4230/LIPIcs.ECOOP.2021.9.

[19] V. Astrauskas *et al.*, "The Prusti Project: Formal Verification for Rust," in *NASA Formal Methods (NFM)*, Springer, 2022. doi: 10.1007/978-3-031-06773-0_5.

[20] S.-É. Ayoun, X. Denis, P. Maksimović, and P. Gardner, "A Hybrid Approach to Semi-automated Rust Verification," in *Proceedings of the ACM on Programming Languages (PLDI)*, 2025. doi: 10.1145/3729289.

[21] C. Lidbury, "Ochre: A Dependently Typed Systems Programming Language," MEng Individual Project, 2024. [Online]. Available: https://assets.super.so/f890b173-9aa0-4184-8dbd-d0ee94de3ebb/files/00463014-8f18-41dc-81f5-2dccbb91ceae/Ochre_Thesis_Main.pdf